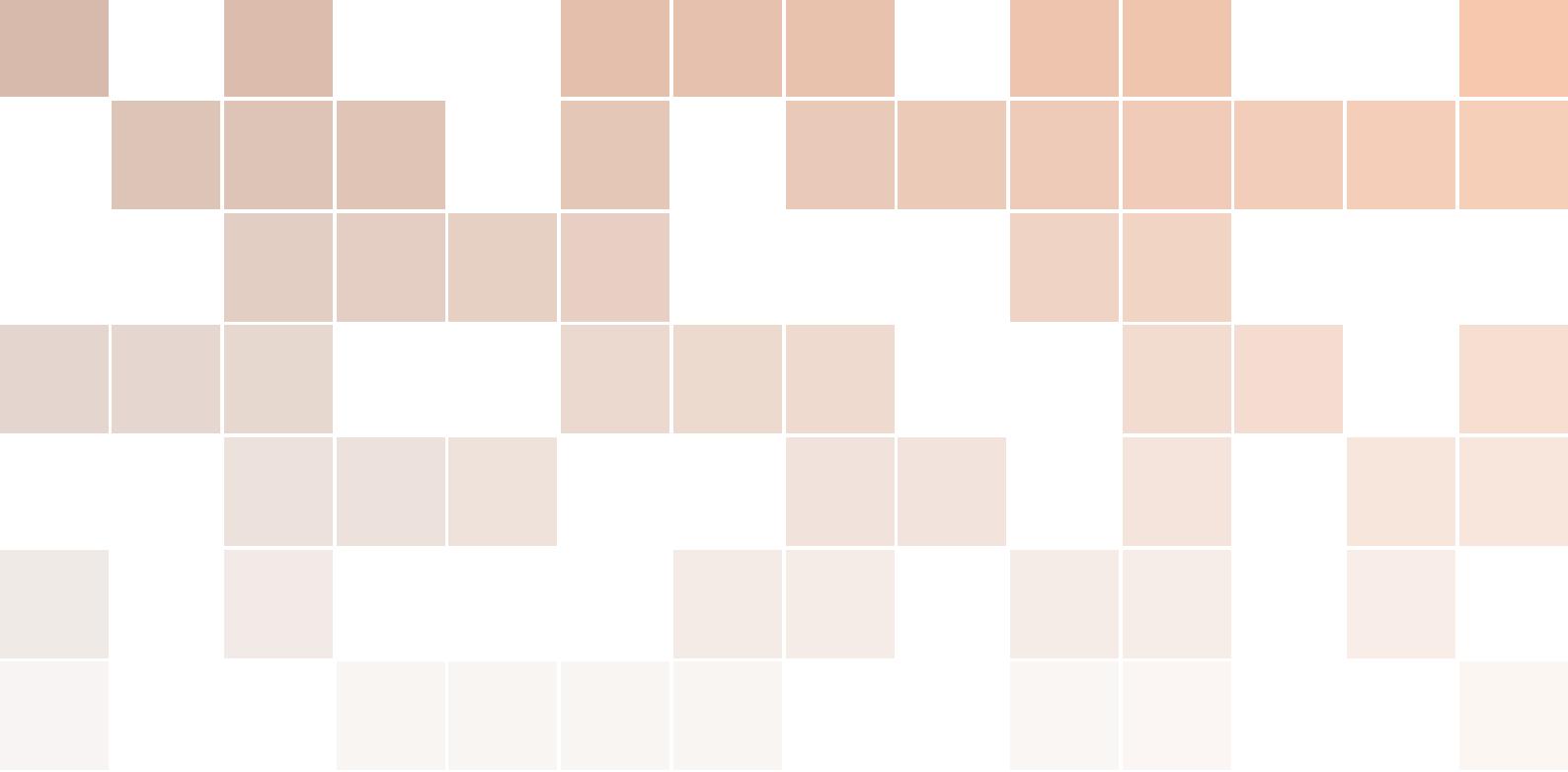


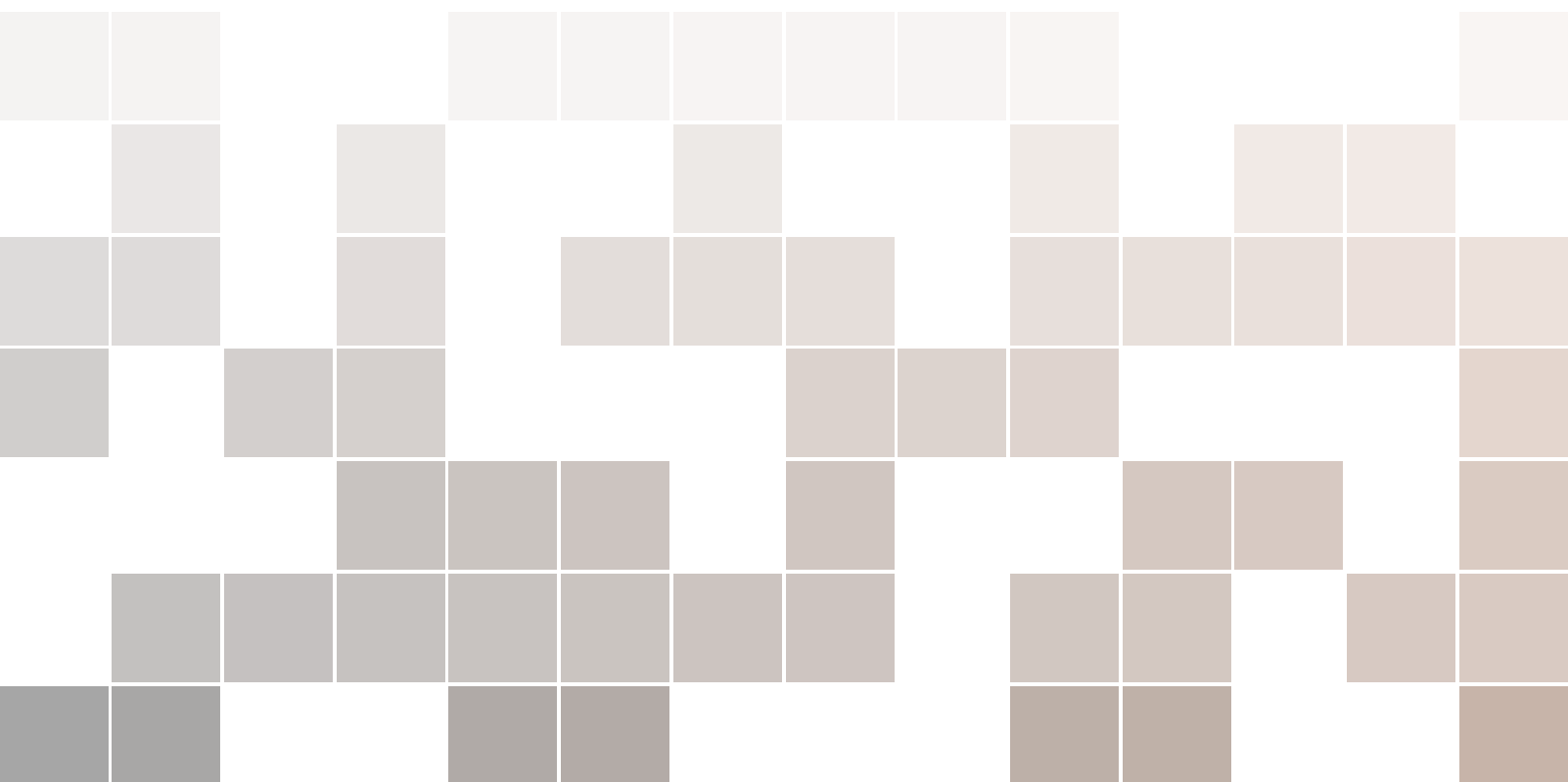


Data Structures Andi Toce

Student Lecture Workbook

Based primarily on material from the course textbook:
Data Structures and Algorithms in Java - 2nd Edition

by Robert Lafore



Last update: Tuesday 13th May, 2025

Contents

1	Introduction	7
1.1	Pre-requisites	7
1.2	What are Data Structures and Algorithms	7
1.3	Object Oriented Programming	8
1.4	Review of Java Programming	8
1.5	Practice Questions	10
2	Arrays	11
2.1	Array Basics	11
2.2	Arrays and OOP	13
2.3	Ordered Arrays	17
2.4	Practice Questions	20
3	Simple Sorting	25
3.1	Introduction	25
3.2	Bubble Sort	25
3.3	Selection Sort	28
3.4	Insertion Sort	29
3.5	Quick Sort	30
3.6	Practice Questions	34
4	Stacks and Queues	39
4.1	Stacks	39

4.2	Queues	44
4.3	Priority Queues	46
4.4	Practice Questions	48
5	Linked Lists	53
5.1	Simple Linked Lists	54
5.2	Double Ended Lists	57
5.3	A Stack Implemented by a Linked List	58
5.4	Doubly Linked Lists	62
5.5	Iterators	67
5.6	Practice Questions	68
6	Binary Trees	73
6.1	Tree Terminology	73
6.2	Java Code for a Binary Tree	75
6.3	Traversing a Tree	83
6.3.1	In Order Traversal	83
6.3.2	Pre Order Traversal	83
6.3.3	Post Order Traversal	83
6.4	Deleting a Node from a Tree	84
6.4.1	Case 1: No Children	84
6.4.2	Case 2: One Child	84
6.4.3	Case 3: Two Children	85
6.5	Trees Represented as Arrays	87
6.6	Practice Questions	88
7	Hash Tables	93
7.1	Introduction	93
7.2	Open Addressing	95
7.2.1	Linear Probing	95
7.2.2	Quadratic Probing	100
7.2.3	Double Hashing	100
7.3	Separate Chaining	105
8	Heaps	115
8.1	Introduction	115
8.2	Heap Operations	117
8.2.1	Removal	117
8.2.2	Insertion	118
8.3	Java Code	119
8.4	Heapify Methods: Top-Down vs Bottom-Up	125
8.4.1	Top-Down Heapify (Successive Insertions)	125
8.4.2	Bottom-Up Heapify	125

8.5	Heapsort	127
8.6	Practice Questions	131
9	Graphs	135
9.1	Introduction and Review from MAC281	135
9.2	Java Graph Code	136
9.3	Searches	139
9.3.1	Depth-First Search	139
9.3.2	Breadth-First Search	144
9.4	Minimum Spanning Trees	149
9.5	Practice Questions	154
10	Weighted Graphs	159
10.1	Minimum Spanning Trees for Weighted Graphs	160
10.1.1	Practice Questions	168
10.2	The Shortest-Path Problem	170
10.2.1	Practice Questions	177
11	When to Use What?	181
11.1	General Purpose Data Structures	181
11.2	Special Purpose Data Structures	182
11.3	Sorting	182
12	Sample Chapter	183
12.1	Sample section	183
A	Answers to Exercises	185
A.1	Chapter 1	185
A.2	Chapter 2	188
A.3	Chapter 1	189
	Index	191

1. Introduction

1.1 Pre-requisites

Before taking this class you must have completed **MAC190** (Object Oriented Programming with Java) and **MAC281** (Discrete Structures). Knowledge of the content from these two classes is essential in fully understanding the topics in this class.

You need to have an **IDE** (Integrated Development Environment) capable to running Java code. I will use Eclipse, but you can use any IDE you prefer.

1.2 What are Data Structures and Algorithms

A **data structure** is an arrangement of data in a computer's memory (or sometimes on a disk). Data structures include arrays, linked lists, stacks, binary trees, and hash tables, among others. **Algorithms** manipulate the data in these structures in various ways, such as searching for a particular data item and sorting the data.

Typical questions we will address when working with data (information):

1. How to arrange (save) data in computer memory for later access?
2. How to add new data to an existing arrangement in memory?
3. How to remove data from an existing arrangement?
4. How to search for information stored via a particular data structure?
5. How to sort data stored in computer memory?

1.3 Object Oriented Programming

- Objects in a program can represent real-world things or abstractions.
- An object is an instance of a class.
- A class is like a blueprint for creating objects.
- A class specifies an object's attributes and defines its behaviors as methods.
- An object contains both methods (behavior) and variables (attributes, characteristics, properties).

1.4 Review of Java Programming

Based on the content from your previous Java classes you should be able to understand and apply the following topics/concepts:

- Variables and data types
- Conditionals and loops
- Methods
- Basic arrays and strings
- Basic OOP, classes, constructors, accessors, mutators, access specifiers, overloading.
- Basic inheritance and polymorphism.

Java OOP practice example. Define a class named **Person**. The class has the following components:

- A variable 'name' of type string
- Default and parameterized constructors
- A method toString(), that returns the information about the object as a string
- Accessor and mutator methods



Then, define a class named **Employee** whose objects are records for employees. Derive this class from the class **Person**. An employee record inherits an employee's name from the class **Person**. In addition, an employee record contains an annual salary represented as a single value of type **double**. Give your class a reasonable complement of constructors, accessor methods, and mutator methods. Finally, use a driver program to test the various features of each class.

BankAccount class example from book:

Code 1.1: BankAccount class (from book)

```
1 class BankAccount {
2     private double balance;           // account balance
3
4     public BankAccount(double openingBalance) { // constructor
5         balance = openingBalance;
6     }
7
8     public void deposit(double amount) {    // makes deposit
9         balance = balance + amount;
10    }
11
12    public void withdraw(double amount) {    // makes withdrawal
13        balance = balance - amount;
14    }
15
16    public void display() {                 // displays balance
17        System.out.println("balance=" + balance);
18    }
19 } // end class BankAccount
```



<— Get code here.

Code 1.2: BankApp test class (from book)

```
1 class BankApp
2 {
3     public static void main(String[] args)
4     {
5         BankAccount ba1 = new BankAccount(100.00); // create acct
6
7         System.out.print("Before transactions , ");
8         ba1.display();                             // display balance
9
10        ba1.deposit(74.35);                          // make deposit
11        ba1.withdraw(20.00);                          // make withdrawal
12
13        System.out.print("After transactions , ");
14        ba1.display();                                // display balance
15    } // end main()
16 } // end class BankApp
```



<— Get code here.

1.5 Practice Questions

1. **Java OOP practice problem:** Implement a class named **BankAccount** that has the following attributes:

- A variable 'balance' of type double
- Default and parameterized constructors
- A method to deposit and withdraw money from the account
- A method to get the current balance of the account

Then, define a class named **SavingsAccount** that inherits from **BankAccount**. A savings account has an interest rate represented as a single value of type double. Implement methods to calculate and add interest to the account, and to get the current interest rate. Finally, use a driver program to test the various features of each class.

2. **Java OOP exercise:** Write a program that defines a class named **Rectangle**. The class has the following attributes:

- A variable 'length' of type double
- A variable 'width' of type double
- Default and parameterized constructors
- A method to calculate and return the area of the rectangle
- A method to calculate and return the perimeter of the rectangle
- Accessor and mutator methods

Then, define a class named **Square** that inherits from **Rectangle**. A square has all sides equal. Implement a method to set the side of the square, and override the methods to calculate the area and perimeter accordingly. Finally, use a driver program to test the various features of each class.

Click [here](#) for answer. (Try to code it first yourself!)

2. Arrays

2.1 Array Basics

An array is a data structure that can store a fixed-size sequential collection of elements of the same type. It is the most commonly used data storage structure and is built into most programming languages.

Recall from your previous classes the following terminology related to arrays:

- We use the *new* operator to create an array
- The name of the array is the reference to the actual array
- We use indices, which start from 0 in most programming languages, to access the elements of an array
- Array elements should be initialized before being used
- The length or size of an array is typically fixed at the time of array creation

The example that follows shows some basic operations on an array of integers. We create an array, store 10 values in it, search for specific values, and delete an element from the array.

Code 2.1: ArrayApp procedural example (from book)

```

1  class ArrayApp {
2  public static void main( String [] args) {
3      long [] arr = new long [100];           // make array
4      int nElems = 0;                         // number of items
5      int j;                                 // loop counter
6      long searchKey;                        // key of item to search for
7  // -----
8      arr [0] = 77;                           // insert 10 items
9      arr [1] = 99;
10     arr [2] = 44;
11     arr [3] = 55;
12     arr [4] = 22;
13     arr [5] = 88;
14     arr [6] = 11;
15     arr [7] = 00;
16     arr [8] = 66;
17     arr [9] = 33;
18     nElems = 10;                            // now 10 items in array
19 // -----
20     for (j=0; j<nElems; j++)                // display items
21         System.out.print( arr[j] + " ");
22     System.out.println( "" );
23 // -----
24     searchKey = 66;                          // find item with key 66
25     for (j=0; j<nElems; j++)                // for each element,
26         if (arr[j] == searchKey)            // found item?
27             break;                          // yes, exit before end
28         if (j == nElems)                    // at the end?
29             System.out.println( "Can't find " + searchKey ); // yes
30     else
31         System.out.println( "Found " + searchKey ); // no
32 // -----
33     searchKey = 55;                          // delete item with key 55
34     for (j=0; j<nElems; j++)                // look for it
35         if (arr[j] == searchKey)
36             break;
37     for (int k=j; k<nElems-1; k++)            // move higher ones down
38         arr[k] = arr[k+1];
39     nElems--;                               // decrement size
40 // -----
41     for (j=0; j<nElems; j++)                // display items
42         System.out.print( arr[j] + " ");
43     System.out.println( "" );
44 } // end main()
45 } // end class ArrayApp

```



← Get code here.

2.2 Arrays and OOP

It is possible to use an OOP approach to design a similar program. Now we'll encapsulate the array in a class, called LowArray. We'll also provide class methods by which objects of other classes (the LowArrayApp class in this case) can access the array. These methods allow communication between LowArray and LowArrayApp.

Code 2.2: Class LowArray (from book)

```
1  class LowArray
2  {
3  private long[] a;           // ref to array a
4  //-----
5  public LowArray(int size)   // constructor
6  { a = new long[size]; }    // create array
7  //-----
8  public void setElem(int index, long value) // set value
9  { a[index] = value; }
10 //-----
11 public long getElem(int index) // get value
12 { return a[index]; }
13 //-----
14 } // end class LowArray
```



← Get code here.

A class used to store data objects, as is LowArray, is sometimes called a container class. Typically, a container class not only stores the data but also provides methods for accessing the data and perhaps also sorting it and performing other complex actions on it.

Code 2.3: Class LowArrayApp (from book)

```

1  class LowArrayApp {
2  public static void main( String [] args) {
3      LowArray arr = new LowArray(100); // create LowArray object
4      int nElems = 0;                    // number of items in array
5      int j;                             // loop variable
6
7      arr.setElem(0, 77);                 // insert 10 items
8      arr.setElem(1, 99);
9      arr.setElem(2, 44);
10     arr.setElem(3, 55);
11     arr.setElem(4, 22);
12     arr.setElem(5, 88);
13     arr.setElem(6, 11);
14     arr.setElem(7, 00);
15     arr.setElem(8, 66);
16     arr.setElem(9, 33);
17     nElems = 10;                        // now 10 items in array
18
19     for(j=0; j<nElems; j++)             // display items
20         System.out.print(arr.getElem(j) + " ");
21     System.out.println("");
22
23     int searchKey = 26;                 // search for data item
24     for(j=0; j<nElems; j++)             // for each element,
25         if(arr.getElem(j) == searchKey) // found item?
26             break;
27     if(j == nElems)                     // no
28         System.out.println("Can't find " + searchKey);
29     else                                // yes
30         System.out.println("Found " + searchKey);
31
32                                     // delete value 55
33     for(j=0; j<nElems; j++)             // look for it
34         if(arr.getElem(j) == 55)
35             break;
36     for(int k=j; k<nElems; k++)         // higher ones down
37         arr.setElem(k, arr.getElem(k+1));
38     nElems--;                           // decrement size
39
40     for(j=0; j<nElems; j++)             // display items
41         System.out.print( arr.getElem(j) + " ");
42     System.out.println("");
43     } // end main()
44 } // end class LowArrayApp

```



← Get code here.

The LowArray class design, uses OOP techniques, but does not take full advantage of the OOP capabilities. Some of the low-level operations with the array are still performed in the main method, instead of being handled eternally by the LowArray class. An improved version is shown below in class HighArray.

Code 2.4: Class HighArray (from book)

```

1  class HighArray {
2      private long[] a;                // ref to array a
3      private int nElems;              // number of data items
4      //-----
5      public HighArray(int max) {      // constructor
6          a = new long[max];          // create the array
7          nElems = 0;                  // no items yet
8      }
9      //-----
10     public boolean find(long searchKey) { // find specified value
11         int j;
12         for(j=0; j<nElems; j++)        // for each element,
13             if(a[j] == searchKey)      // found item?
14                 break;                 // exit loop before end
15         if(j == nElems)                 // gone to end?
16             return false;              // yes, can't find it
17         else
18             return true;                // no, found it
19     } // end find()
20     //-----
21     public void insert(long value) {    // put element into array
22         a[nElems] = value;             // insert it
23         nElems++;                      // increment size
24     }
25     //-----
26     public boolean delete(long value) {
27         int j;
28         for(j=0; j<nElems; j++)        // look for it
29             if( value == a[j] )
30                 break;
31         if(j==nElems)                   // can't find it
32             return false;
33         else                            // found it
34             {
35                 for(int k=j; k<nElems; k++) // move higher ones down
36                     a[k] = a[k+1];
37                 nElems--;                // decrement size
38                 return true;
39             }
40     } // end delete()
41     //-----
42 
```

```

43 public void display() {           // displays array contents
44     for(int j=0; j<nElems; j++)    // for each element,
45         System.out.print(a[j] + " "); // display it
46     System.out.println("");
47 }
48 //-----
49 } // end class HighArray

```



<— Get code here.

Code 2.5: Class HighArrayApp (from book)

```

1  class HighArrayApp
2  {
3  public static void main( String [] args)
4  {
5      int maxSize = 100;           // array size
6      HighArray arr;               // reference to array
7      arr = new HighArray(maxSize); // create the array
8
9      arr.insert(77);               // insert 10 items
10     arr.insert(99);
11     arr.insert(44);
12     arr.insert(55);
13     arr.insert(22);
14     arr.insert(88);
15     arr.insert(11);
16     arr.insert(00);
17     arr.insert(66);
18     arr.insert(33);
19
20     arr.display();                // display items
21
22     int searchKey = 35;           // search for item
23     if( arr.find(searchKey) )
24         System.out.println("Found " + searchKey);
25     else
26         System.out.println("Can't find " + searchKey);
27
28     arr.delete(00);               // delete 3 items
29     arr.delete(55);
30     arr.delete(99);
31
32     arr.display();                // display items again
33 } // end main()
34 } // end class HighArrayApp

```



<— Get code here.

2.3 Ordered Arrays

If the values of elements in an array are arranged in increasing (or decreasing) order, then we have an **ordered array**. Many algorithms are designed specifically for such arrays.

Ordered arrays allow for more efficient way to search for values. Previously, we implemented a linear search algorithm which works the same way for any type of array. The OrderedArray class below uses a binary search algorithm.

Code 2.6: Class OrdArray (from book)

```

1  class OrdArray {
2  private long[] a;           // ref to array a
3  private int nElems;         // number of data items
4  //-----
5  public OrdArray(int max) {   // constructor
6      a = new long[max];      // create array
7      nElems = 0;
8  }
9  //-----
10 public int size()
11     { return nElems; }
12 //-----
13 public int find(long searchKey){
14     int lowerBound = 0;
15     int upperBound = nElems-1;
16     int curIn;
17
18     while(true) {
19         curIn = (lowerBound + upperBound) / 2;
20         if(a[curIn]==searchKey)
21             return curIn;           // found it
22         else if(lowerBound > upperBound)
23             return nElems;         // can't find it
24         else
25             // divide range
26             {
27                 if(a[curIn] < searchKey)
28                     lowerBound = curIn + 1; // it's in upper half
29                 else
30                     upperBound = curIn - 1; // it's in lower half
31             } // end else divide range
32     } // end while
33 } // end find()
34 //-----
35 public void insert(long value) { // put element into array
36     int j;
37     for(j=0; j<nElems; j++)        // find where it goes
38         if(a[j] > value)           // (linear search)
39             break;
40     for(int k=nElems; k>j; k--)    // move bigger ones up

```

```
40     a[k] = a[k-1];
41     a[j] = value;           // insert it
42     nElems++;              // increment size
43 } // end insert()
44 //-----
45 public boolean delete(long value){
46     int j = find(value);
47     if(j==nElems)           // can't find it
48         return false;
49     else {                  // found it
50         for(int k=j; k<nElems; k++) // move bigger ones down
51             a[k] = a[k+1];
52         nElems--;           // decrement size
53         return true;
54     }
55 } // end delete()
56 //-----
57 public void display() {    // displays array contents
58     for(int j=0; j<nElems; j++) // for each element,
59         System.out.print(a[j] + " "); // display it
60     System.out.println("");
61 }
62 //-----
63 } // end class OrdArray
```



← Get code here.

Code 2.7: Class OrderedApp (from book)

```

1  class OrderedApp
2  {
3  public static void main( String [] args )
4  {
5      int maxSize = 100;           // array size
6      OrdArray arr;               // reference to array
7      arr = new OrdArray( maxSize ); // create the array
8
9      arr.insert(77);              // insert 10 items
10     arr.insert(99);
11     arr.insert(44);
12     arr.insert(55);
13     arr.insert(22);
14     arr.insert(88);
15     arr.insert(11);
16     arr.insert(00);
17     arr.insert(66);
18     arr.insert(33);
19
20     int searchKey = 55;          // search for item
21     if( arr.find(searchKey) != arr.size() )
22         System.out.println( "Found " + searchKey );
23     else
24         System.out.println( "Can't find " + searchKey );
25
26     arr.display();              // display items
27
28     arr.delete(00);             // delete 3 items
29     arr.delete(55);
30     arr.delete(99);
31
32     arr.display();              // display items again
33 } // end main()
34 } // end class OrderedApp

```



← Get code here.



What are the advantages and disadvantages of using ordered arrays?



Analyze the efficiency of various operations, including searching, insertion, and deletion, in both randomly ordered and unordered arrays. Use Big O notation to express the time complexity of these operations. Discuss the factors that might affect the performance of these operations in different scenarios.

2.4 Practice Questions

1. (from book) **True or False:** When you delete an item from an unordered array, in most cases you shift other items to fill in the gap.
2. (from book) **True or False:** In an unordered array, it's generally faster to find out an item is not in the array than to find out it is.
3. **True or False:** When you add an item to an unordered array, you can simply insert it at the end of the array without shifting any other items.
4. **True or False:** In an unordered array, the time complexity of searching for an item is always $O(n)$, where n is the number of elements in the array.
5. **True or False:** In an unordered array, the time complexity of deleting the last item in the array is $O(1)$, regardless of the number of elements in the array.
6. **True or False:** In an ordered array, searching for an item using binary search is generally faster than linear search, especially for large arrays.
7. **True or False:** In an ordered array, finding the maximum and minimum items can be done in constant time.
8. **True or False:** In an ordered array, deleting the maximum or minimum item can be done in constant time using the first or last element of the array, respectively.
9. **True or False:** In an ordered array, the time complexity of searching for an item is always $O(\log n)$, where n is the number of elements in the array, assuming binary search is used.
10. (from book) Inserting an item into an unordered array...
 - (a) takes time proportional to the size of the array.
 - (b) requires multiple comparisons.
 - (c) requires shifting other items to make room.
 - (d) takes the same time no matter how many items there are.

Click [here](#) for answer. (Try it first yourself!)

11. (from book) In an unordered array, allowing duplicates
 - (a) increases times for all operations.
 - (b) increases search times in some situations.
 - (c) always increases insertion times.
 - (d) sometimes decreases insertion times.

12. (from book) If class A is going to use class B for something, then
 - (a) class A's methods should be easy to understand.
 - (b) it's preferable if class B communicates with the program's user.
 - (c) the more complex operations should be placed in class A.
 - (d) the more work that class B can do, the better.

13. (from book) The maximum number of elements that must be examined to complete a binary search in an array of 200 elements is
 - (a) 200.
 - (b) 8.
 - (c) 1.
 - (d) 13.

14. (from book) To the HighArray class add a method called **getMax()** that returns the value of the highest key in the array, or -1 if the array is empty. Add some code in **main()** to exercise this method. You can assume all the keys are positive numbers.

Click [here](#) for answer. (Try to code it first yourself!)

15. (from book) Modify the method in the previous exercise so that the item with the highest key is not only returned by the method, but also removed from the array. Call the method **removeMax()**.

Click [here](#) for answer. (Try to code it first yourself!)

16. Modify the **HighArray** class to include a method called **getMin()** that returns the value of the lowest key in the array, or -1 if the array is empty. Add some code in **main()** to exercise this method.
17. Write a method called **reverse()** in the **HighArray** class that reverses the order of the elements in the array. Add some code in **main()** to exercise this method.
18. Write a method called **removeDuplicates()** in the **HighArray** class that removes any duplicates from the array. Add some code in **main()** to exercise this method.
19. Add a method called **shuffle()** to the **HighArray** class that randomly shuffles the elements in the array. Add some code in **main()** to exercise this method.
20. (from book) Modify the `OrdArray.java` program (Listing 2.4) so that the **insert()** and **delete()** routines, as well as **find()**, use a binary search.

Click [here](#) for answer. (Try to code it first yourself!)

21. (from book) Add a **merge()** method to the `OrdArray` class so that you can merge two ordered source arrays into an ordered destination array. Write code in **main()** that inserts some random numbers into the two source arrays, invokes **merge()**, and displays the contents of the resulting destination array. The source arrays may hold different numbers of data items. In your algorithm you will need to compare the keys of the source arrays, picking the smallest one to copy to the destination. You'll also need to handle the situation when one source array exhausts its contents before the other.
22. (from book) Write a **noDups()** method for the `HighArray` class. This method should remove all duplicates from the array. That is, if three items with the key 17 appear in the array, **noDups()** should remove two of them. Don't worry about maintaining the order of the items. One approach is to first compare every item with all the other items and overwrite any duplicates with a null (or a distinctive value that isn't used for real keys). Then remove all the nulls. Of course, the array size will be reduced.
23. Consider the class **Array** below that encapsulates an array of integers. Add a member method *reverse()* to the class that reverses the order of the elements in the array. Write a small program **ArrayApp.java** that creates an object of type `Array`, inserts a few values, and tests the method *reverse()*.

```
1 class Array {  
2     private int[] a; // ref to array a  
3     private int nElems; // number of data items
```

```
4      //-----  
5      public Array(int max) {  
6          a = new int[max]; // create the array  
7          nElems = 0; // no items yet  
8      }  
9      //-----  
10     public void insert(int value) {  
11         a[nElems] = value; // insert it  
12         nElems++; // increment size  
13     }  
14     //-----  
15 } // end class Array
```


3. Simple Sorting

3.1 Introduction

Sorting algorithms are systematic procedures that reorder items in a collection in some specific way. In Computer Science in particular, sorting algorithms are important and have a large array of applications.

There are many sorting algorithms. Some are simple but often less efficient, but are quite often used as examples for learning purposes. Examples include Bubble sort, Insertion sort and Selection sort. More sophisticated algorithms typically perform better in applications. A typical list would include Quick sort, Merge sort, Heap sort, etc.

Each of these algorithms has its own advantages and disadvantages in terms of speed, memory usage, and efficiency, and the choice of algorithm depends on the specific requirements of the application.

3.2 Bubble Sort

The bubble sort is notoriously slow, but it's conceptually the simplest of the sorting algorithms and for that reason is a good beginning for our exploration of sorting techniques.

Code 3.1: Class ArrayBub (from book)

```

1  class ArrayBub
2  {
3      private long [] a;           // ref to array a
4      private int nElems;         // number of data items
5      //-----
6      public ArrayBub(int max)     // constructor
7      {
8          a = new long[max];       // create the array
9          nElems = 0;              // no items yet
10     }
11     //-----
12     public void insert(long value) // put element into array
13     {
14         a[nElems] = value;        // insert it
15         nElems++;                 // increment size
16     }
17     //-----
18     public void display()         // displays array contents
19     {
20         for(int j=0; j<nElems; j++) // for each element,
21             System.out.print(a[j] + " "); // display it
22         System.out.println("");
23     }
24     //-----
25     public void bubbleSort()
26     {
27         int out, in;
28
29         for(out=nElems-1; out>1; out--) // outer loop (backward)
30             for(in=0; in<out; in++)    // inner loop (forward)
31                 if( a[in] > a[in+1] ) // out of order?
32                     swap(in, in+1);    // swap them
33     } // end bubbleSort()
34     //-----
35     private void swap(int one, int two)
36     {
37         long temp = a[one];
38         a[one] = a[two];
39         a[two] = temp;
40     }
41     //-----
42 } // end class ArrayBub

```



← Get code here.



← Get A C++ version of Bubble sort here.

Code 3.2: Class BubbleSortApp (from book)

```
1  class BubbleSortApp
2  {
3      public static void main(String [] args)
4      {
5          int maxSize = 100;           // array size
6          ArrayBub arr;                // reference to array
7          arr = new ArrayBub(maxSize); // create the array
8
9          arr.insert(77);               // insert 10 items
10         arr.insert(99);
11         arr.insert(44);
12         arr.insert(55);
13         arr.insert(22);
14         arr.insert(88);
15         arr.insert(11);
16         arr.insert(00);
17         arr.insert(66);
18         arr.insert(33);
19
20         arr.display();                // display items
21
22         arr.bubbleSort();              // bubble sort them
23
24         arr.display();                // display them again
25     } // end main()
26 } // end class BubbleSortApp
```



← Get code here.



Analyze the efficiency of Bubble Sort.

3.3 Selection Sort

The selection sort improves on the bubble sort by reducing the number of swaps necessary from $O(N^2)$ to $O(N)$. Unfortunately, the number of comparisons remains $O(N^2)$. However, the selection sort can still offer a significant improvement for large records that must be physically moved around in memory, causing the swap time to be much more important than the comparison time. (Typically, this isn't the case in Java, where references are moved around, not entire objects.)

Code 3.3: SelectionSort method (from book)

```

1 public void selectionSort ()
2 {
3     int out , in , min;
4
5     for (out=0; out<nElems-1; out++)    // outer loop
6     {
7         min = out;                      // minimum
8         for (in=out+1; in<nElems; in++) // inner loop
9             if (a[in] < a[min] )        // if min greater ,
10                min = in;                // we have a new min
11        swap(out , min);                 // swap them
12    } // end for(out)
13 } // end selectionSort ()

```



Use the **selectionSort()** method instead of the **bubbleSort()** method in the previous section to test it.



← Get a C++ version of Selection Sort here.



Analyze the efficiency of Selection Sort.

3.4 Insertion Sort

In most cases the insertion sort is the best of the elementary sorts described in this chapter. It still executes in $O(N^2)$ time, but it's about twice as fast as the bubble sort and somewhat faster than the selection sort in normal situations. It's also not too complex, although it's slightly more involved than the bubble and selection sorts. It's often used as the final stage of more sophisticated sorts, such as quicksort.

Code 3.4: InsertionSort method (from book)

```
1 public void insertionSort ()
2 {
3     int in , out;
4
5     for (out=1; out<nElems; out++)    // out is dividing line
6     {
7         long temp = a[out];           // remove marked item
8         in = out;                     // start shifts at out
9         while (in>0 && a[in-1] >= temp) // until one is smaller,
10            {
11                a[in] = a[in-1];       // shift item right,
12                --in;                  // go left one position
13            }
14         a[in] = temp;                 // insert marked item
15     } // end for
16 } // end insertionSort ()
```



Test the **insertionSort()** method in a Java program.



← Get a C++ version of Insertion Sort here.



Analyze the efficiency of Insertion Sort.

3.5 Quick Sort

Quick Sort is a highly efficient and widely used sorting algorithm that operates on the principle of divide and conquer. Developed by Tony Hoare in 1959, Quick Sort exhibits remarkable performance by leveraging a recursive strategy to rapidly sort an array or list of elements. By intelligently partitioning the input into smaller subsets and recursively sorting them, Quick Sort efficiently and effectively rearranges elements, bringing order with remarkable speed.

Here's the list of steps for the Quick Sort algorithm:

- Choose a pivot element from the array.
- Rearrange the array such that all elements smaller than the pivot come before it, and all elements larger than the pivot come after it.
- Recursively apply the above steps to the subarray of elements smaller than the pivot.
- Recursively apply the above steps to the subarray of elements larger than the pivot.
- Repeat the process until each subarray contains only one element (could be more than one).
- Combine all the sorted subarrays to obtain the final sorted array.



The pivot should typically be one of the values in the array. How should we determine which one?

Code 3.5: Class ArrayIns (from book)

```

1 package Ch03_QuickSort;
2
3 class ArrayIns
4 {
5     private long[] theArray;           // ref to array theArray
6     private int nElems;                // number of data items
7     //-----
8     public ArrayIns(int max)           // constructor
9     {
10         theArray = new long[max];      // create the array
11         nElems = 0;                   // no items yet
12     }
13     //-----
14     public void insert(long value)     // put element into array
15     {
16         theArray[nElems] = value;      // insert it
17         nElems++;                      // increment size
18     }
19     //-----
20     public void display()              // displays array contents
21     {
22         System.out.print("A=");
23         for(int j=0; j<nElems; j++)    // for each element,
24             System.out.print(theArray[j] + " "); // display it
25         System.out.println("");
26     }
27     //-----
28     public void quickSort()
29     {
30         recQuickSort(0, nElems-1);
31     }
32     //-----
33     public void recQuickSort(int left, int right)
34     {
35         int size = right-left+1;
36         if(size <= 3)                 // manual sort if small
37             manualSort(left, right);
38         else                           // quicksort if large
39         {
40             long median = medianOf3(left, right);
41             int partition = partitionIt(left, right, median);
42             recQuickSort(left, partition-1);
43             recQuickSort(partition+1, right);
44         }
45     } // end recQuickSort()
46     //-----
47     public long medianOf3(int left, int right)
48     {

```

```

49     int center = (left+right)/2;
50     // order left & center
51     if( theArray[left] > theArray[center] )
52         swap(left , center);
53     // order left & right
54     if( theArray[left] > theArray[right] )
55         swap(left , right);
56     // order center & right
57     if( theArray[center] > theArray[right] )
58         swap(center , right);
59
60     swap(center , right-1);           // put pivot on right
61     return theArray[right-1];        // return median value
62 } // end medianOf3 ()
63 //-----
64 public void swap(int dex1 , int dex2) // swap two elements
65 {
66     long temp = theArray[dex1];      // A into temp
67     theArray[dex1] = theArray[dex2]; // B into A
68     theArray[dex2] = temp;           // temp into B
69 } // end swap(
70 //-----
71 public int partitionIt(int left , int right , long pivot)
72 {
73     int leftPtr = left;               // right of first elem
74     int rightPtr = right - 1;        // left of pivot
75
76     while(true)
77     {
78         while( theArray[++leftPtr] < pivot ) // find bigger
79             ;                               // (nop)
80         while( theArray[--rightPtr] > pivot ) // find smaller
81             ;                               // (nop)
82         if(leftPtr >= rightPtr)           // if pointers cross ,
83             break;                       // partition done
84         else                             // not crossed , so
85             swap(leftPtr , rightPtr);    // swap elements
86     } // end while(true)
87     swap(leftPtr , right-1);             // restore pivot
88     return leftPtr;                     // return pivot location
89 } // end partitionIt ()
90 //-----
91 public void manualSort(int left , int right)
92 {
93     int size = right-left+1;
94     if(size <= 1)
95         return;                        // no sort necessary
96     if(size == 2)
97     {
98         // 2-sort left and right

```



```

98         if( theArray[ left ] > theArray[ right ] )
99             swap( left , right );
100         return ;
101     }
102     else // size is 3
103     { // 3-sort left , center , & right
104         if( theArray[ left ] > theArray[ right -1 ] )
105             swap( left , right -1); // left , center
106         if( theArray[ left ] > theArray[ right ] )
107             swap( left , right ); // left , right
108         if( theArray[ right -1 ] > theArray[ right ] )
109             swap( right -1, right ); // center , right
110     }
111 } // end manualSort()
112 //-----
113 } // end class ArrayIns

```



← Get code here.

Code 3.6: Class QuickSort2App (from book)

```

1  class QuickSort2App
2  {
3      public static void main( String [] args )
4      {
5          int maxSize = 16; // array size
6          ArrayIns arr; // reference to array
7          arr = new ArrayIns( maxSize ); // create the array
8          for( int j=0; j<maxSize; j++) // fill array with
9              { // random numbers
10                 long n = (int)( java.lang.Math.random()*99 );
11                 arr.insert( n );
12             }
13             arr.display(); // display items
14             arr.quickSort(); // quicksort them
15             arr.display(); // display them again
16         } // end main()
17     } // end class QuickSort2App

```



← Get code here.



Analyze the efficiency of Quick Sort.

3.6 Practice Questions

1. (from book) **True or False:** The bubble sort always ends up comparing every item with every other item.
2. (from book) **True or False:** If there are N items, the bubble sort makes exactly $N*N$ comparisons.
3. (from book) **True or False:** If, in a particular sorting situation, swaps take much longer than comparisons, the selection sort is about twice as fast as the bubble sort.
4. **True or False:** The best-case time complexity of bubble sort is $O(n)$ when the input array is already sorted.
5. **True or False:** The space complexity of bubble sort is $O(1)$, meaning that it requires constant extra memory regardless of the size of the input array.
6. **True or False:** Bubble sort is a simple sorting algorithm that repeatedly swaps adjacent elements if they are in the wrong order.
7. **True or False:** The worst-case time complexity of bubble sort is $O(n^2)$.
8. **True or False:** Insertion sort performs better than selection sort when the input array is already partially sorted.
9. **True or False:** Bubble sort can be optimized by adding a flag that indicates whether any swaps were made during a pass through the array, in order to terminate early if the array is already sorted.
10. **True or False:** Selection sort only works on numbers.
11. **True or False:** Selection sort is an in-place sorting algorithm, meaning that it sorts the input array without using extra memory.
12. **True or False:** Quick Sort is an in-place sorting algorithm, meaning it doesn't require additional memory proportional to the size of the input array.

-
13. **True or False:** The worst-case time complexity of Quick Sort is $O(n^2)$.
14. **True or False:** Quick Sort can be implemented iteratively using a stack instead of recursion.
15. **True or False:** Quick Sort can be optimized by using a randomized pivot selection strategy.
16. **True or False:** Quick Sort exhibits a best-case time complexity of $O(n \log n)$ when the pivot element consistently divides the input into approximately equal-sized partitions.
17. (from book) Computer sorting algorithms are more limited than humans in that
- (a) humans are better at inventing new algorithms.
 - (b) computers can handle only a fixed amount of data.
 - (c) humans know what to sort, whereas computers need to be told.
 - (d) computers can compare only two things at a time.
18. (from book) The bubble sort algorithm alternates between
- (a) comparing and swapping.
 - (b) moving and copying.
 - (c) moving and comparing.
 - (d) copying and comparing.
19. (from book) In the selection sort,
- (a) the largest keys accumulate on the left (low indices).
 - (b) a minimum key is repeatedly discovered.
 - (c) a number of items must be shifted to insert each item in its correctly sorted position.
 - (d) the sorted items accumulate on the right.
20. (from book) In the insertion sort, after an item is inserted in the partially sorted group, it will
- (a) never be moved again.

- (b) never be shifted to the left.
 - (c) often be moved out of this group.
 - (d) find that its group is steadily shrinking.
21. (from book) In the BubbleSort program, the in index always goes from left to right, finding the largest item and carrying it toward out on the right. Modify the **bubbleSort()** method so that it's bidirectional. This means the in index will first carry the largest item from left to right as before, but when it reaches out, it will reverse and carry the smallest item from right to left. You'll need two outer indexes, one on the right (the old out) and another on the left.
22. (from book) Add a method called **median()** to the InsertSort program. This method should return the median value in the array. (Recall that in a group of numbers half are larger than the median and half are smaller.) Do it the easy way.
23. (from book) To the InsertSort program, add a method called **noDups()** that removes duplicates from a previously sorted array without disrupting the order. (You can use the **insertionSort()** method to sort the data, or you can simply use **main()** to insert the data in sorted order.) One can imagine schemes in which all the items from the place where a duplicate was discovered to the end of the array would be shifted down one space every time a duplicate was discovered, but this would lead to slow $O(N^2)$ time, at least when there were a lot of duplicates. In your algorithm, make sure no item is moved more than once, no matter how many duplicates there are. This will give you an algorithm with $O(N)$ time.
24. Write a Java program that sorts an array of integers using a modified selection sort algorithm. In this algorithm, instead of finding the minimum element in the unsorted portion of the array and swapping it with the first element, you find both the minimum and maximum elements in the unsorted portion of the array and swap them with the first and last elements, respectively. The program should take user input to determine the size of the array and the elements to be sorted. The sorted array should be displayed to the user.
25. Write a Java program that sorts an array of integers using a modified bubble sort algorithm. In the modified algorithm, the sort will end early if there are no swaps made during a full pass through the array. This can be done because if there are no swaps made, the array is already sorted. The program should take user input to determine the size of the array and the elements to be sorted. The sorted array should be displayed to the user.

26. Sort the following array using the Quick Sort algorithm. Show each step of the sorting process, including partitioning and choosing the pivot using the median of three strategy. Use manual sorting when encountering sub-arrays of size 3 or less. Additionally, provide the state of the array after each partitioning step.

Array: [5, 3, 8, 2, 9, 1, 7, 6, 4]

0	1	2	3	4	5	6	7	8
5	3	8	2	9	1	7	6	4

4. Stacks and Queues

Stacks, queues, and priority queues are more abstract entities than arrays and many other data storage structures.

4.1 Stacks

A stack allows access to only one data item: the last item inserted. If you remove this item, you can access the next-to-last item inserted, and so on.



In which situations can a Stack be useful?

Code 4.1: Class StackX (from book)

```
1 class StackX
2 {
3     private int maxSize;           // size of stack array
4     private long[] stackArray;
5     private int top;               // top of stack
6     //-----
7     public StackX(int s)           // constructor
8     {
9         maxSize = s;               // set array size
10        stackArray = new long[maxSize]; // create array
11        top = -1;                   // no items yet
12    }
13    //-----
14    public void push(long j)        // put item on top of stack
15    {
16        stackArray[++top] = j;      // increment top, insert item
17    }
18    //-----
19    public long pop()               // take item from top of stack
20    {
21        return stackArray[top--];  // access item, decrement top
22    }
23    //-----
24    public long peek()              // peek at top of stack
25    {
26        return stackArray[top];
27    }
28    //-----
29    public boolean isEmpty()        // true if stack is empty
30    {
31        return (top == -1);
32    }
33    //-----
34    public boolean isFull()         // true if stack is full
35    {
36        return (top == maxSize - 1);
37    }
38    //-----
39 } // end class StackX
```



← Get code here.

Code 4.2: Class StackApp (from book)

```
1 class StackApp
2 {
3     public static void main( String [] args )
4     {
5         StackX theStack = new StackX(10); // make new stack
6         theStack.push(20);                // push items onto stack
7         theStack.push(40);
8         theStack.push(60);
9         theStack.push(80);
10
11         while( !theStack.isEmpty() )      // until it's empty,
12         {                                  // delete item from stack
13             long value = theStack.pop();
14             System.out.print( value );    // display it
15             System.out.print( " " );
16         } // end while
17         System.out.println( " " );
18     } // end main()
19 } // end class StackApp
```



← Get code here.

Stack Example: Delimiter Matching

One common use for stacks is to parse certain kinds of text strings. Typically, the strings are lines of code in a computer language, and the programs parsing them are compilers.

To give the flavor of what's involved, we'll show a program that checks the delimiters in a line of text typed by the user. This text doesn't need to be a line of real Java code (although it could be), but it should use delimiters the same way Java does. The delimiters are the braces { and }, brackets [and], and parentheses (and).

Code 4.3: Class BracketChecker (from book)

```

1  class BracketChecker {
2  private String input;           // input string
3  //-----
4  public BracketChecker(String in) // constructor
5      { input = in; }
6  //-----
7  public void check() {
8      int stackSize = input.length(); // get max stack size
9      StackX theStack = new StackX(stackSize); // make stack
10
11     for(int j=0; j<input.length(); j++) // get chars in turn
12     {
13         char ch = input.charAt(j); // get char
14         switch(ch)
15         {
16             case '{': // opening symbols
17             case '[':
18             case '(':
19                 theStack.push(ch); // push them
20                 break;
21             case '}': // closing symbols
22             case ']':
23             case ')':
24                 if( !theStack.isEmpty() ) // if stack not empty,
25                 {
26                     char chx = (char) theStack.pop();
27                     // pop and check
28                     if( (ch=='}' && chx!='{') ||
29                        (ch==']' && chx!='[') ||
30                        (ch==')' && chx!='(') )
31                         System.out.println("Error: "+ch+" at "+j);
32                 }
33             else // prematurely empty
34                 System.out.println("Error: "+ch+" at "+j);
35             break;
36             default: // no action on other characters
37                 break;
38         } // end switch
39     } // end for
40     // at this point, all characters have been processed
41     if( !theStack.isEmpty() )
42         System.out.println("Error: missing right delimiter");
43 } // end check()
44 //-----
45 } // end class BracketChecker

```



← Get code here.

Code 4.4: Class BracketApp (from book)

```

1
2 import java.io.*;
3
4 class BracketsApp
5 {
6     public static void main(String[] args) throws IOException
7     {
8         String input;
9         while(true)
10         {
11             System.out.print(
12                 "Enter string containing delimiters: ");
13             System.out.flush();
14             input = getString(); // read a string from kbd
15             if( input.equals("") ) // quit if [Enter]
16                 break;
17                                     // make a BracketChecker
18             BracketChecker theChecker = new BracketChecker(input);
19             theChecker.check(); // check brackets
20         } // end while
21     } // end main()
22 //-----
23     public static String getString() throws IOException
24     {
25         InputStreamReader isr = new InputStreamReader(System.in);
26         BufferedReader br = new BufferedReader(isr);
27         String s = br.readLine();
28         return s;
29     }
30 //-----
31 } // end class BracketsApp

```



← Get code here.



Analyze the efficiency of StackX.

4.2 Queues

In computer science a queue is a data structure that is somewhat like a stack, except that in a queue the first item inserted is the first to be removed (First-In-First-Out, FIFO), while in a stack, as we've seen, the last item inserted is the first to be removed (LIFO).

Code 4.5: Class Queue (from book)

```

1  class Queue {
2  private long[] queArray;
3  private int maxSize, front, rear, nItems;
4  //-----
5  public Queue(int s) {           // constructor
6      maxSize = s;
7      queArray = new long[maxSize];
8      front = 0;
9      rear = -1;
10     nItems = 0;
11 }
12 //-----
13 public void insert(long j) { // put item at rear of queue
14     if(rear == maxSize-1)    // deal with wraparound
15         rear = -1;
16     queArray[++rear] = j;    // increment rear and insert
17     nItems++;                // one more item
18 }
19 //-----
20 public long remove() {        // take item from front of queue
21     long temp = queArray[front++]; // get value and incr front
22     if(front == maxSize)        // deal with wraparound
23         front = 0;
24     nItems--;                  // one less item
25     return temp;
26 }
27 //-----
28 public long peekFront() {     // peek at front of queue
29     return queArray[front];
30 }
31 //-----
32 public boolean isEmpty() {    // true if queue is empty
33     return (nItems==0);
34 }
35 //-----
36 public boolean isFull() { return (nItems==maxSize); }
37 //-----
38 public int size() { return nItems; }
39 //-----
40 } // end class Queue

```



← Get code here.

Code 4.6: Class QueueApp (from book)

```
1  class QueueApp
2  {
3  public static void main( String [] args)
4  {
5      Queue theQueue = new Queue(5);    // queue holds 5 items
6
7      theQueue.insert(10);               // insert 4 items
8      theQueue.insert(20);
9      theQueue.insert(30);
10     theQueue.insert(40);
11
12     theQueue.remove();                 // remove 3 items
13     theQueue.remove();                 //      (10, 20, 30)
14     theQueue.remove();
15
16     theQueue.insert(50);               // insert 4 more items
17     theQueue.insert(60);               //      (wraps around)
18     theQueue.insert(70);
19     theQueue.insert(80);
20
21     while( !theQueue.isEmpty() )       // remove and display
22     {                                   //      all items
23         long n = theQueue.remove();    // (40, 50, 60, 70, 80)
24         System.out.print(n);
25         System.out.print(" ");
26     }
27     System.out.println("");
28 } // end main()
29 } // end class QueueApp
```



← Get code here.



Analyze the efficiency of Queue.

4.3 Priority Queues

A priority queue is a more specialized data structure than a stack or a queue. Like an ordinary queue, a priority queue has a front and a rear, and items are removed from the front. However, in a priority queue, items are ordered by key value so that the item with the lowest key (or in some implementations the highest key) is always at the front. Items are inserted in the proper position to maintain the order.

Code 4.7: Class PriorityQApp (from book)

```
1
2 import java.io.*;
3
4 class PriorityQApp
5 {
6     public static void main(String[] args) throws IOException
7     {
8         PriorityQ thePQ = new PriorityQ(5);
9         thePQ.insert(30);
10        thePQ.insert(50);
11        thePQ.insert(10);
12        thePQ.insert(40);
13        thePQ.insert(20);
14
15        while( !thePQ.isEmpty() )
16        {
17            long item = thePQ.remove();
18            System.out.print(item + " "); // 10, 20, 30, 40, 50
19        } // end while
20        System.out.println("");
21    } // end main()
22 //-----
23 } // end class PriorityQApp
```



← Get code here.

Code 4.8: Class PriorityQ (from book)

```

1  class PriorityQ {
2  // array in sorted order, from max at 0 to min at size-1
3  private int maxSize;
4  private long[] queArray;
5  private int nItems;
6  //-----
7  public PriorityQ(int s) {           // constructor
8      maxSize = s;
9      queArray = new long[maxSize];
10     nItems = 0;
11 }
12 //-----
13 public void insert(long item) { // insert item
14     int j;
15     if(nItems==0)                // if no items,
16         queArray[nItems++] = item; // insert at 0
17     else {                        // if items,
18         for(j=nItems-1; j>=0; j--) { // start at end,
19             if( item > queArray[j] ) // if new item larger,
20                 queArray[j+1] = queArray[j]; // shift upward
21             else // if smaller,
22                 break; // done shifting
23         } // end for
24         queArray[j+1] = item; // insert it
25         nItems++;
26     } // end else (nItems > 0)
27 } // end insert()
28 //-----
29 public long remove() // remove minimum item
30 { return queArray[--nItems]; }
31 //-----
32 public long peekMin() // peek at minimum item
33 { return queArray[nItems-1]; }
34 //-----
35 public boolean isEmpty() // true if queue is empty
36 { return (nItems==0); }
37 //-----
38 public boolean isFull() // true if queue is full
39 { return (nItems == maxSize); }
40 //-----
41 } // end class PriorityQ

```



← Get code here.



Analyze the efficiency of Priority Queues.

4.4 Practice Questions

1. **True or False:** In Java, a stack is a data structure that follows the LIFO (Last In First Out) principle.
2. **True or False:** The push method in Java's Stack class inserts an element at the top of the stack.
3. **True or False:** The pop method in Java's Stack class removes an element from the bottom of the stack.
4. **True or False:** The peek method in Java's Stack class returns the top element of the stack without removing it.
5. **True or False:** A queue in Java is a data structure that follows the FIFO (First In First Out) principle.
6. **True or False:** The remove method in Java's Queue removes an element from the front of the queue.
7. **True or False:** The offer method in Java's Queue inserts an element at the end of the queue.
8. **True or False:** The peek method in Java's Queue interface returns, but does not remove, the head of the queue, returning null if the queue is empty.
9. **True or False:** A priority queue in Java is a special type of queue wherein all elements are ordered according to specific criteria.
10. **True or False:** In a priority queue, elements are removed from the front based on their priority instead of their insertion order.
11. (from book) Suppose you push 10, 20, 30, and 40 onto the stack. Then you pop three items. Which one is left on the stack?
12. (from book) Which of the following is true?
 - (a) The pop operation on a stack is considerably simpler than the remove operation on a queue.

- (b) The contents of a queue can wrap around, while those of a stack cannot.
 - (c) The top of a stack corresponds to the front of a queue.
 - (d) In both the stack and the queue, items removed in sequence are taken from increasingly high index cells in the array.
13. (from book) As other items are inserted and removed, does a particular item in a queue move along the array from lower to higher indices, or higher to lower?
14. (from book) Suppose you insert 15, 25, 35, and 45 into a queue. Then you remove three items. Which one is left?
15. (from book) **True or False:** Pushing and popping items on a stack and inserting and removing items in a queue all take $O(N)$ time.
16. (from book) A queue might be used to hold
- (a) the items to be sorted in an insertion sort.
 - (b) reports of a variety of imminent attacks on the star ship Enterprise.
 - (c) keystrokes made by a computer user writing a letter.
 - (d) symbols in an algebraic expression being evaluated.
17. (from book) Inserting an item into a typical priority queue takes what big O time?

18. (from book) The term priority in a priority queue means that
 - (a) the highest priority items are inserted first.
 - (b) the programmer must prioritize access to the underlying array.
 - (c) the underlying array is sorted by the priority of the items.
 - (d) the lowest priority items are deleted first.
19. (from book) A priority queue might be used to hold
 - (a) passengers to be picked up by a taxi from different parts of the city.
 - (b) keystrokes made at a computer keyboard.
 - (c) squares on a chessboard in a game program.
 - (d) planets in a solar system simulation.
20. (from book) Think about how you remember the events in your life. Are there times when they seem to be stored in your brain in a stack? In a queue? In a priority queue?
21. (from book) Write a method for the Queue class that displays the contents of the queue. Note that this does not mean simply displaying the contents of the underlying array. You should show the queue contents from the first item inserted to the last, without indicating to the viewer whether the sequence is broken by wrapping around the end of the array. Be careful that one item and no items display properly, no matter where **front** and **rear** are.
22. (from book) Create a Deque class based on the discussion of dequeues (double-ended queues) in this chapter. It should include insertLeft(), insertRight(), removeLeft(), removeRight(), isEmpty(), and isFull() methods. It will need to support wraparound at the end of the array, as queues do.
23. (from book) Write a program with a revised PriorityQ class that has fast $O(1)$ insertion time but slower removal of the high-priority item. Include a method that displays the contents of the priority queue.
24. Consider the StackX java code from this chapter. Add the following methods to the class StackX.
 - (a) **size():** Write a method that returns the number of elements in the stack. The method signature should be public int size(). The method should return the number of elements

currently in the stack. Test your method by pushing some elements to the stack and then calling your `size()` method. Verify that the returned value is correct.

- (b) **capacity():** Write a method that returns the maximum number of elements the stack can hold. The method signature should be `public int capacity()`. The method should return the maximum size of the stack. Test your method by creating a stack with a certain capacity and then calling your `capacity()` method. Verify that the returned value is correct.
- (c) **clear():** Write a method that removes all elements from the stack. The method signature should be `public void clear()`. After calling this method, the stack should be empty. Test your method by pushing some elements to the stack, calling your `clear()` method, and then checking if the stack is empty.
- (d) **contains(long j):** Write a method that checks if a specific element is in the stack. The method signature should be `public boolean contains(long j)`. The method should return `true` if the element is in the stack, and `false` otherwise. Test your method by pushing some elements to the stack, and then checking if these elements are contained in the stack.

25. Consider the Queue java code from this chapter. Add the following methods to the class Queue.

- (a) **rear():** Write a method that returns the rear element of the queue without removing it. The method signature should be `public long rear()`. The method should return the rear element of the queue. Test your method by inserting some elements to the queue, calling your `rear()` method, and then verifying that the returned element is at the rear of the queue.
- (b) **contains(long j):** Write a method that checks if a specific element is in the queue. The method signature should be `public boolean contains(long j)`. The method should return `true` if the element is in the queue, and `false` otherwise. Test your method by inserting some elements to the queue, and then checking if these elements are contained in the queue.
- (c) **clear():** Write a method that removes all elements from the queue. The method signature should be `public void clear()`. After calling this method, the queue should be empty. Test your method by inserting some elements to the queue, calling your `clear()` method, and then checking if the queue is empty.
- (d) **replace(long oldVal, long newVal):** Write a method that replaces all occurrences of a specified element (`oldVal`) in the queue with another (`newVal`). The method signature should be `public void replace(long oldVal, long newVal)`. Test your method by inserting some elements to the queue, calling your `replace(long oldVal, long newVal)` method with some old and new values, and then verifying that all occurrences of the old value have been replaced with the new value.

5. Linked Lists

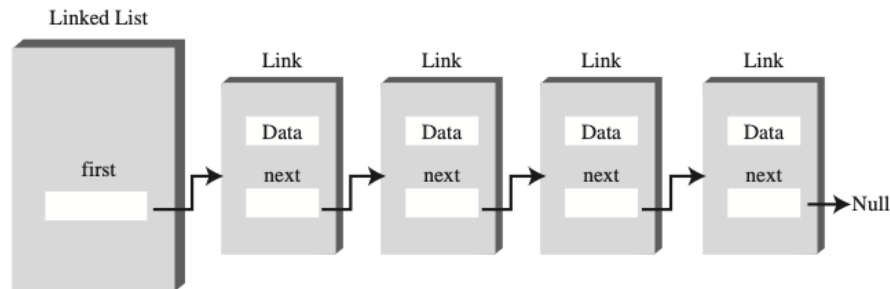
The linked list is a versatile mechanism suitable for use in many kinds of general-purpose databases. It can also replace an array as the basis for other storage structures such as stacks and queues. In fact, you can use a linked list in many cases in which you use an array, unless you need frequent random access to individual items using an index.



What are some of the advantages and disadvantages of using arrays?

5.1 Simple Linked Lists

In a linked list, each data item is embedded in a link. A link is an object of a class called something like **Link**. Each Link object contains a reference (usually called next) to the next link in the list.



Code 5.1: Class Link (from book)

```

1  class Link
2  {
3      public int iData;           // data item
4      public Link next;          // next link in list
5      //-----
6      public Link(int id) // constructor
7      {
8          iData = id;             // initialize data
9      }                          // ('next' is automatically // set to null)
10     //-----
11     public void displayLink()   // display yourself
12     {
13         System.out.print("(" + iData + ")");
14     }
15 } // end class Link

```



← Get code here.

Code 5.2: Class LinkList (from book)

```

1  class LinkList {
2      private Link first;           // ref to first link on list
3      //-----
4      public LinkList()             // constructor
5      {
6          first = null;              // no links on list yet
7      }
8      //-----
9      public void insertFirst(int id)
10     {                             // make new link
11         Link newLink = new Link(id);
12         newLink.next = first;       // it points to old first link
13         first = newLink;            // now first points to this
14     }
15     //-----
16     public Link find(int key)       // find link with given key
17     {                             // (assumes non-empty list)
18         Link current = first;       // start at 'first'
19         while(current.iData != key) // while no match,
20         {
21             if(current.next == null) // if end of list,
22                 return null;         // didn't find it
23             else                      // not end of list,
24                 current = current.next; // go to next link
25         }
26         return current;              // found it
27     }
28     //-----
29     public Link delete(int key)     // delete link with given key
30     {                             // (assumes non-empty list)
31         Link current = first;       // search for link
32         Link previous = first;
33         while(current.iData != key)
34         {
35             if(current.next == null)
36                 return null;        // didn't find it
37             else
38             {
39                 previous = current; // go to next link
40                 current = current.next;
41             }
42         }                          // found it
43         if(current == first)        // if first link,
44             first = first.next;     // change first
45         else                        // otherwise,
46             previous.next = current.next; // bypass it
47         return current;
48     }

```

```

49 //-----
50 public void displayList()    // display the list
51 {
52     System.out.print("List (first-->last): ");
53     Link current = first;    // start at beginning of list
54     while(current != null)    // until end of list,
55     {
56         current.displayLink(); // print data
57         current = current.next; // move to next link
58     }
59     System.out.println("");
60 }
61 //-----
62 } // end class LinkedList

```



← Get code here.

Code 5.3: Class LinkedListApp (from book)

```

1 class LinkedListApp {
2     public static void main(String[] args) {
3         LinkedList theList = new LinkedList(); // make list
4
5         theList.insertFirst(22);    // insert 4 items
6         theList.insertFirst(44);
7         theList.insertFirst(66);
8         theList.insertFirst(88);
9
10        theList.displayList();        // display list
11
12        Link f = theList.find(44);    // find item
13        if( f != null)
14            System.out.println("Found link with key " + f.iData);
15        else
16            System.out.println("Can't find link");
17
18        Link d = theList.delete(66);    // delete item
19        if( d != null )
20            System.out.println("Deleted link with key " + d.iData);
21        else
22            System.out.println("Can't delete link");
23
24        theList.displayList();        // display list
25    } // end main()
26 } // end class LinkedListApp

```



← Get code here.



Add a method **isEmpty()** to the `LinkedList` class that returns true if the list is empty, false otherwise. Test the method in a program.



Add a method **deleteFirst()** to the `LinkedList` class that removes the first element on the list, if the list is not empty. The method returns a reference to the deleted link. Test the method in a program.

5.2 Double Ended Lists

A double-ended list is similar to an ordinary linked list, but it has one additional feature: a reference to the last link as well as to the first. The reference to the last link permits you to insert a new link directly at the end of the list as well as at the beginning. Access to the end of the list as well as the beginning makes the double-ended list suitable for certain situations that a single-ended list can't handle efficiently (e.g. Queues). Check the book for an implementation of the Double Ended List.



Modify the previous code for a simple Linked List to implement a Double Ended List. Use a program to test the changes.



Analyze the efficiency of Linked Lists.

5.3 A Stack Implemented by a Linked List

Previously we used an array to store and retrieve the data belonging to a Stack. Here we will use a Linked List instead.

NOTE: The program that follows uses the class Link defined earlier in this document.

Code 5.4: Class Link (from book)

```
1 class Link
2 {
3     public long dData;           // data item
4     public Link next;           // next link in list
5     //-----
6     public Link(long dd)         // constructor
7     { dData = dd; }
8     //-----
9     public void displayLink()    // display myself
10    { System.out.print(dData + " "); }
11 } // end class Link
```



← Get code here.

Code 5.5: Class LinkedList (from book)

```

1  class LinkedList
2  {
3      private Link first;           // ref to first item on list
4      //-----
5      public LinkedList()           // constructor
6      { first = null; }             // no items on list yet
7      //-----
8      public boolean isEmpty()       // true if list is empty
9      { return (first==null); }
10     //-----
11     public void insertFirst(long dd) // insert at start of list
12     {                               // make new link
13         Link newLink = new Link(dd);
14         newLink.next = first;       // newLink --> old first
15         first = newLink;           // first --> newLink
16     }
17     //-----
18     public long deleteFirst()       // delete first item
19     {                               // (assumes list not empty)
20         Link temp = first;          // save reference to link
21         first = first.next;         // delete it: first-->old next
22         return temp.dData;          // return deleted link
23     }
24     //-----
25     public void displayList()
26     {
27         Link current = first;       // start at beginning of list
28         while(current != null)      // until end of list ,
29         {
30             current.displayLink();  // print data
31             current = current.next;  // move to next link
32         }
33         System.out.println("");
34     }
35     //-----
36 } // end class LinkedList

```



← Get code here.

Code 5.6: Class LinkStack (from book)

```
1  class LinkStack
2  {
3      private LinkedList theList;
4      //-----
5      public LinkStack()           // constructor
6      {
7          theList = new LinkedList();
8      }
9      //-----
10     public void push(long j)      // put item on top of stack
11     {
12         theList.insertFirst(j);
13     }
14     //-----
15     public long pop()             // take item from top of stack
16     {
17         return theList.deleteFirst();
18     }
19     //-----
20     public boolean isEmpty()      // true if stack is empty
21     {
22         return ( theList.isEmpty() );
23     }
24     //-----
25     public void displayStack()
26     {
27         System.out.print("Stack (top-->bottom): ");
28         theList.displayList();
29     }
30     //-----
31 } // end class LinkStack
```



← Get code here.

Code 5.7: Class LinkStackApp (from book)

```
1  class LinkStackApp
2  {
3      public static void main(String [] args)
4      {
5          LinkStack theStack = new LinkStack(); // make stack
6
7          theStack.push(20);                      // push items
8          theStack.push(40);
9
10         theStack.displayStack();                 // display stack
11
12         theStack.push(60);                      // push items
13         theStack.push(80);
14
15         theStack.displayStack();                 // display stack
16
17         theStack.pop();                          // pop items
18         theStack.pop();
19
20         theStack.displayStack();                 // display stack
21     } // end main()
22 } // end class LinkStackApp
```



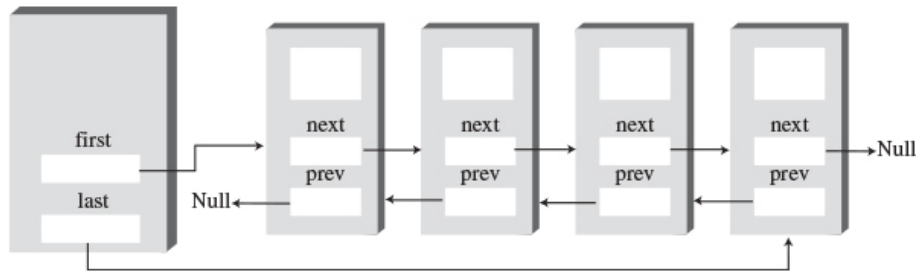
← Get code here.



What are the differences and similarities between the array and linked list implementations of a Stack?

5.4 Doubly Linked Lists

Let's examine another variation on the linked list: the doubly linked list (not to be confused with the double-ended list). The doubly linked list allows you to traverse backward as well as forward through the list. The secret is that each link has two references to other links instead of one. The first is to the next link, as in ordinary lists. The second is to the previous link.



Code 5.8: Class Link (from book)

```

1  class Link
2  {
3      public long dData;           // data item
4      public Link next;           // next link in list
5      public Link previous;       // previous link in list
6      //-----
7      public Link(long d)         // constructor
8      { dData = d; }
9      //-----
10     public void displayLink()    // display this link
11     { System.out.print(dData + " "); }
12     //-----
13 } // end class Link
  
```



← Get code here.

Code 5.9: Class DoublyLinkedList (from book)

```

1  class DoublyLinkedList
2  {
3      private Link first;           // ref to first item
4      private Link last;           // ref to last item
5      //-----
6      public DoublyLinkedList()     // constructor
7      {
8          first = null;             // no items on list yet
9          last = null;
10     }
11     //-----
12     public boolean isEmpty()       // true if no links
13     { return first==null; }
14     //-----
15     public void insertFirst(long dd) // insert at front of list
16     {
17         Link newLink = new Link(dd); // make new link
18
19         if( isEmpty() )             // if empty list ,
20             last = newLink;         // newLink <-- last
21         else
22             first.previous = newLink; // newLink <-- old first
23         newLink.next = first;       // newLink --> old first
24         first = newLink;            // first --> newLink
25     }
26     //-----
27     public void insertLast(long dd) // insert at end of list
28     {
29         Link newLink = new Link(dd); // make new link
30         if( isEmpty() )             // if empty list ,
31             first = newLink;         // first --> newLink
32         else
33         {
34             last.next = newLink;     // old last --> newLink
35             newLink.previous = last; // old last <-- newLink
36         }
37         last = newLink;              // newLink <-- last
38     }
39     //-----
40     public Link deleteFirst()       // delete first link
41     {                               // (assumes non-empty list)
42         Link temp = first;
43         if( first.next == null )    // if only one item
44             last = null;            // null <-- last
45         else
46             first.next.previous = null; // null <-- old next
47         first = first.next;         // first --> old next
48         return temp;

```

```

49     }
50     //-----
51     public Link deleteLast()           // delete last link
52     {                                  // (assumes non-empty list)
53         Link temp = last;
54         if(first.next == null)         // if only one item
55             first = null;              // first --> null
56         else
57             last.previous.next = null; // old previous --> null
58         last = last.previous;           // old previous <-- last
59         return temp;
60     }
61     //-----
62     // insert dd just after key
63     public boolean insertAfter(long key, long dd)
64     {                                  // (assumes non-empty list)
65         Link current = first;          // start at beginning
66         while(current.dData != key)    // until match is found,
67         {
68             current = current.next;    // move to next link
69             if(current == null)
70                 return false;          // didn't find it
71         }
72         Link newLink = new Link(dd);    // make new link
73
74         if(current==last)               // if last link,
75         {
76             newLink.next = null;        // newLink --> null
77             last = newLink;             // newLink <-- last
78         }
79         else                             // not last link,
80         {
81             newLink.next = current.next; // newLink --> old next
82             // newLink <-- old next
83             current.next.previous = newLink;
84         }
85         newLink.previous = current;     // old current <-- newLink
86         current.next = newLink;         // old current --> newLink
87         return true;                    // found it, did insertion
88     }
89     //-----
90     public Link deleteKey(long key)     // delete item w/ given key
91     {                                  // (assumes non-empty list)
92         Link current = first;          // start at beginning
93         while(current.dData != key)    // until match is found,
94         {
95             current = current.next;    // move to next link
96             if(current == null)
97                 return null;           // didn't find it

```



```

98     }
99     if (current == first)           // found it; first item?
100         first = current.next;      // first --> old next
101     else                             // not first
102         // old previous --> old next
103         current.previous.next = current.next;
104
105     if (current == last)           // last item?
106         last = current.previous;    // old previous <-- last
107     else                             // not last
108         // old previous <-- old next
109         current.next.previous = current.previous;
110     return current;                // return value
111 }
112 //-----
113 public void displayForward()
114 {
115     System.out.print("List (first --> last): ");
116     Link current = first;           // start at beginning
117     while (current != null)         // until end of list,
118     {
119         current.displayLink();      // display data
120         current = current.next;      // move to next link
121     }
122     System.out.println("");
123 }
124 //-----
125 public void displayBackward()
126 {
127     System.out.print("List (last --> first): ");
128     Link current = last;            // start at end
129     while (current != null)         // until start of list,
130     {
131         current.displayLink();      // display data
132         current = current.previous; // move to previous link
133     }
134     System.out.println("");
135 }
136 //-----
137 } // end class DoublyLinkedList

```



← Get code here.

Code 5.10: Class DoublyLinkedListApp (from book)

```
1 class DoublyLinkedListApp
2 {
3     public static void main( String [] args)
4     {
5         // make a new list
6         DoublyLinkedList theList = new DoublyLinkedList();
7
8         theList.insertFirst(22);    // insert at front
9         theList.insertFirst(44);
10        theList.insertFirst(66);
11
12        theList.insertLast(11);     // insert at rear
13        theList.insertLast(33);
14        theList.insertLast(55);
15
16        theList.displayForward();   // display list forward
17        theList.displayBackward(); // display list backward
18
19        theList.deleteFirst();      // delete first item
20        theList.deleteLast();       // delete last item
21        theList.deleteKey(11);      // delete item with key 11
22
23        theList.displayForward();   // display list forward
24
25        theList.insertAfter(22, 77); // insert 77 after 22
26        theList.insertAfter(33, 88); // insert 88 after 33
27
28        theList.displayForward();   // display list forward
29    } // end main()
30 } // end class DoublyLinkedListApp
```



← Get code here.

5.5 Iterators

Objects containing references to items in data structures, used to traverse these structures, are commonly called iterators.



← Get code here.

5.6 Practice Questions

1. **True or False:** In a singly linked list, each node contains a reference to both the next node and the previous node.
2. **True or False:** Linked lists have constant-time access to any element in the list.
3. **True or False:** Adding an element to the end of a singly linked list requires traversing the entire list.
4. **True or False:** In a doubly linked list, each node contains a reference to both the next node and the previous node.
5. **True or False:** Deleting the head node of a singly linked list requires traversing the entire list.
6. **True or False:** Singly linked lists require less memory compared to doubly linked lists.
7. **True or False:** Linked lists are a suitable data structure for implementing a stack.
8. **True or False:** Linked lists are inherently sorted data structures.
9. **True or False:** Deleting the tail node of a doubly linked list requires traversing the entire list. **True or False:** In Java, an iterator is an interface that allows traversal of elements in a collection sequentially.
10. Which of the following is not true? A reference to a class object
 - (a) can be used to access public methods in the object.
 - (b) has a size dependant on its class.
 - (c) has the data type of the class.
 - (d) does not hold the object itself.

-
11. Access to the links in a linked list is usually through the link.
 12. When you create a reference to a link in a linked list, it
 - (a) must refer to the first link.
 - (b) must refer to the link pointed to by current.
 - (c) must refer to the link pointed to by next.
 - (d) can refer to any link you want.
 13. How many references must you change to insert a link in the middle of a singly linked list?
 14. How many references must you change to insert a link at the end of a singly linked list?
 15. In the **insertFirst()** method in the **linkList.java** program, the statement *newLink.next=first;* means that
 - (a) the *next* new link to be inserted will refer to *first*.
 - (b) *first* will refer to the new link.
 - (c) the *next* field of the new link will refer to the old first link.
 - (d) *newLink.next* will refer to the new first link in the list.
 16. Assuming *current* points to the next-to-last link in a singly linked list, what statement will delete the last link from the list?
 17. When all references to a link are changed to refer to something else, what happens to the link?
 18. A double-ended list
 - (a) can be accessed from either end.
 - (b) is a different name for a doubly linked list.
 - (c) has pointers running both forward and backward between links.
 - (d) has its first link connected to its last link.

19. A special case often occurs for insertion and deletion routines when a list is
20. Assuming a copy takes longer than a comparison, is it faster to delete an item with a certain key from a linked list or from an unsorted array?
21. Which of the following is not true? Iterators would be useful if you wanted to
 - (a) do an insertion sort on a linked list.
 - (b) insert a new link at the beginning of a list.
 - (c) swap two links at arbitrary locations.
 - (d) delete all links with a certain key value.
22. Which do you think would be a better choice to implement a stack: a singly linked list or an array?
23. Implement a priority queue based on a sorted linked list. The remove operation on the priority queue should remove the item with the smallest key.
24. A circular list is a linked list in which the last link points back to the first link. There are many ways to design a circular list. Sometimes there is a pointer to the 'start' of the list. However, this makes the list less like a real circle and more like an ordinary list that has its end attached to its beginning. Make a class for a singly linked circular list that has no end and no beginning. The only access to the list is a single reference, current, that can point to any link on the list. This reference can move around the list as needed. Your list should handle insertion, searching, and deletion. You may find it convenient if these operations take place one link downstream of the link pointed to by current. (Because the upstream link is singly linked, you can't get at it without going all the way around the circle.) You should also be able to display the list (although you'll need to break the circle at some arbitrary point to print it on the screen). A step() method that moves current along to the next link might come in handy too.
25. Implement a stack class based on the circular list from the previous exercise. This exercise is not too difficult. (However, implementing a queue can be harder, unless you make the circular list doubly linked.)
26. The Josephus Problem is a famous mathematical puzzle that goes back to ancient times. There are many stories to go with the puzzle. One is that Josephus was one of a group of Jews who were about to be captured by the Romans. Rather than be enslaved, they chose to commit suicide. They arranged themselves in a circle and, starting at a certain person,

started counting off around the circle. Every n th person had to leave the circle and commit suicide. Josephus decided he didn't want to die, so he arranged the rules so he would be the last person left. If there were (say) 20 people, and he was the seventh person from the start of the circle, what number should he tell them to use for counting off? The problem is made much more complicated because the circle shrinks as the counting continues.

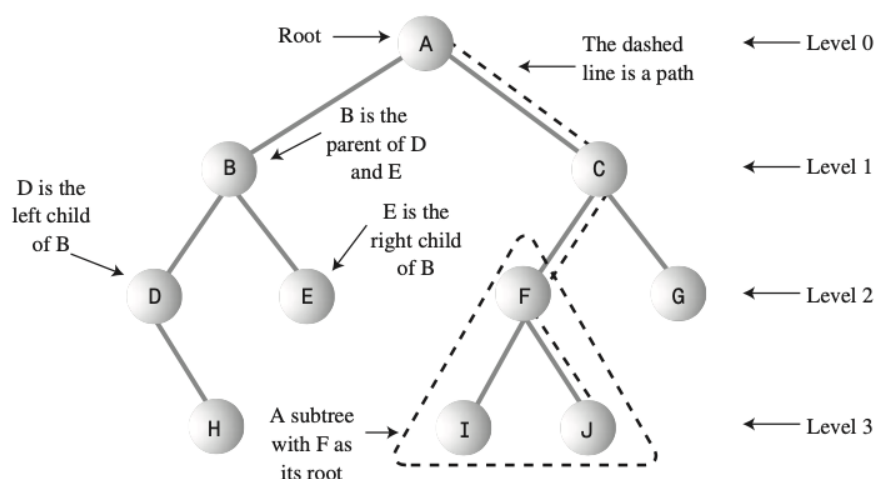
Create an application that uses a circular linked list to model this problem. Inputs are the number of people in the circle, the number used for counting off, and the number of the person where counting starts (usually 1). The output is the list of persons being eliminated. When a person drops out of the circle, counting starts again from the person who was on his left (assuming you go around clockwise). Here's an example. There are seven people numbered 1 through 7, and you start at 1 and count off by threes. People will be eliminated in the order 4, 1, 6, 5, 7, 3. Number 2 will be left.

27. **Code Question:** Implement a method called `deleteAll()` in the `LinkedList` class that deletes all links from the list. Write the code for this method.
28. **Code Question:** Extend the functionality of the `LinkedList` class by adding a method named `deleteAllByKey(int key)` that removes all links with the specified key from the list. Write the implementation code for this method.
29. **Code Question:** Implement a method called `insertAtIndex(int index, long dd)` in the `DoublyLinkedList` class that inserts a new link with the given data at the specified index position in the list. Provide the implementation code for this method.
30. **Code Question:** Add a method named `get(int index)` to the `DoublyLinkedList` class that retrieves the data stored in the link at the specified index position in the list. Write the code for this method.
31. **Code Question:** Extend the functionality of the `DoublyLinkedList` class by adding a method named `reverse()` that reverses the order of the links in the list. Write the implementation code for this method.

6. Binary Trees

Binary trees are one of the fundamental data storage structures used in programming. They provide advantages that the data structures we've seen so far cannot. A binary tree is very useful because it combines the advantages of two other structures: an ordered array and a linked list. You can search a tree quickly, as you can an ordered array, and you can also insert and delete items quickly, as you can with a linked list.

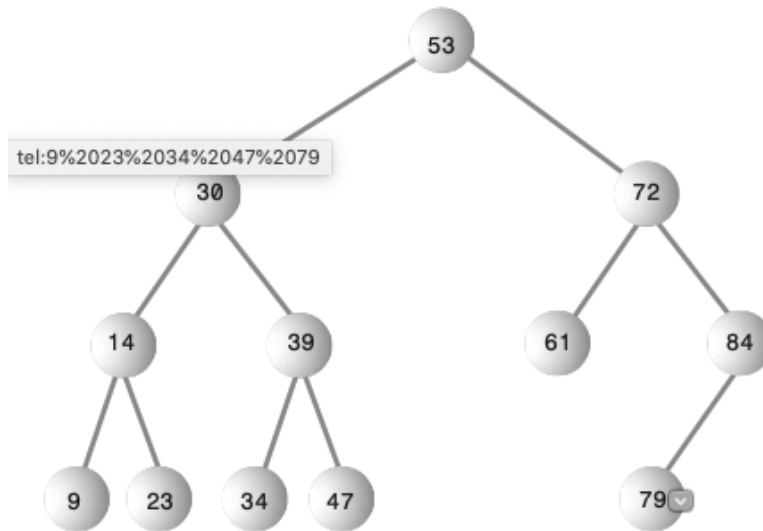
6.1 Tree Terminology



If every node in a tree can have at most two children, the tree is called a **binary tree**.

The two children of each node in a binary tree are called the **left child** and the **right child**.

In a binary search tree a node's left child must have a key less than its parent, and a node's right child must have a key greater than or equal to its parent.



A binary search tree.

6.2 Java Code for a Binary Tree

Code 6.1: Class Node (from book)

```

1  class Node
2  {
3      public int iData;           // data item (key)
4      // data item
5      public Node leftChild;      // this node's left child
6      public Node rightChild;    // this node's right child
7
8      public void displayNode()   // display ourself
9      {
10         System.out.print('(');
11         System.out.print(iData);
12         System.out.print(") ");
13     }
14 } // end class Node

```



← Get code here.

Code 6.2: Class Tree (from book)

```

1  import java.util.*;           // for Stack class
2
3  class Tree
4  {
5      private Node root;        // first node of tree
6
7      //-----
8      public Tree()              // constructor
9      { root = null; }           // no nodes in tree yet
10     //-----
11     public Node find(int key)   // find node with given key
12     {                           // (assumes non-empty tree)
13         Node current = root;    // start at root
14         while(current.iData != key) // while no match,
15         {
16             if(key < current.iData) // go left?
17                 current = current.leftChild;
18             else // or go right?
19                 current = current.rightChild;
20             if(current == null) // if no child,
21                 return null;   // didn't find it
22         }
23         return current;         // found it
24     } // end find()
25     //-----

```

```

26 public void insert(int id)
27 {
28     Node newNode = new Node(); // make new node
29     newNode.iData = id; // insert data
30     if (root == null) // no node in root
31         root = newNode;
32     else // root occupied
33     {
34         Node current = root; // start at root
35         Node parent;
36         while (true) // (exits internally)
37         {
38             parent = current;
39             if (id < current.iData) // go left?
40             {
41                 current = current.leftChild;
42                 if (current == null) // if end of the line,
43                     // insert on left
44                     parent.leftChild = newNode;
45                 return;
46             }
47             // end if go left
48             else // or go right?
49             {
50                 current = current.rightChild;
51                 if (current == null) // if end of the line
52                     // insert on right
53                     parent.rightChild = newNode;
54                 return;
55             }
56             // end else go right
57         } // end while
58     } // end else not root
59 } // end insert()
60 //-----
61 public boolean delete(int key) // delete node with given key
62 { // (assumes non-empty list)
63     Node current = root;
64     Node parent = root;
65     boolean isLeftChild = true;
66
67     while (current.iData != key) // search for node
68     {
69         parent = current;
70         if (key < current.iData) // go left?
71         {
72             isLeftChild = true;
73             current = current.leftChild;
74         }

```

```
75         else // or go right?
76         {
77             isLeftChild = false;
78             current = current.rightChild;
79         }
80         if(current == null) // end of the line ,
81             return false; // didn't find it
82     } // end while
83     // found node to delete
84
85     // if no children , simply delete it
86     if(current.leftChild==null &&
87        current.rightChild==null)
88     {
89         if(current == root) // if root ,
90             root = null; // tree is empty
91         else if(isLeftChild)
92             parent.leftChild = null; // disconnect
93         else // from parent
94             parent.rightChild = null;
95     }
96
97     // if no right child , replace with left subtree
98     else if(current.rightChild==null)
99         if(current == root)
100             root = current.leftChild;
101         else if(isLeftChild)
102             parent.leftChild = current.leftChild;
103         else
104             parent.rightChild = current.leftChild;
105
106     // if no left child , replace with right subtree
107     else if(current.leftChild==null)
108         if(current == root)
109             root = current.rightChild;
110         else if(isLeftChild)
111             parent.leftChild = current.rightChild;
112         else
113             parent.rightChild = current.rightChild;
114
115     else // two children , so replace with inorder successor
116     {
117         // get successor of node to delete (current)
118         Node successor = getSuccessor(current);
119
120         // connect parent of current to successor instead
121         if(current == root)
122             root = successor;
123         else if(isLeftChild)
```

```

124         parent.leftChild = successor;
125     else
126         parent.rightChild = successor;
127
128         // connect successor to current's left child
129         successor.leftChild = current.leftChild;
130     } // end else two children
131     // (successor cannot have a left child)
132     return true; // success
133 } // end delete()
134 //-----
135 // returns node with next-highest value after delNode
136 // goes to right child, then right child's left descendents
137 private Node getSuccessor(Node delNode)
138 {
139     Node successorParent = delNode;
140     Node successor = delNode;
141     Node current = delNode.rightChild; // go to right child
142     while(current != null) // until no more
143     { // left children,
144         successorParent = successor;
145         successor = current;
146         current = current.leftChild; // go to left child
147     }
148     // if successor not
149     if(successor != delNode.rightChild) // right child,
150     { // make connections
151         successorParent.leftChild = successor.rightChild;
152         successor.rightChild = delNode.rightChild;
153     }
154     return successor;
155 }
156 //-----
157 public void traverse(int traverseType)
158 {
159     switch(traverseType)
160     {
161     case 1: System.out.print("\nPreorder traversal: ");
162         preOrder(root);
163         break;
164     case 2: System.out.print("\nInorder traversal: ");
165         inOrder(root);
166         break;
167     case 3: System.out.print("\nPostorder traversal: ");
168         postOrder(root);
169         break;
170     }
171     System.out.println();
172 }

```

```
173 //-----
174 private void preOrder(Node localRoot)
175 {
176     if(localRoot != null)
177     {
178         System.out.print(localRoot.iData + " ");
179         preOrder(localRoot.leftChild);
180         preOrder(localRoot.rightChild);
181     }
182 }
183 //-----
184 private void inOrder(Node localRoot)
185 {
186     if(localRoot != null)
187     {
188         inOrder(localRoot.leftChild);
189         System.out.print(localRoot.iData + " ");
190         inOrder(localRoot.rightChild);
191     }
192 }
193 //-----
194 private void postOrder(Node localRoot)
195 {
196     if(localRoot != null)
197     {
198         postOrder(localRoot.leftChild);
199         postOrder(localRoot.rightChild);
200         System.out.print(localRoot.iData + " ");
201     }
202 }
203 //-----
204 public void displayTree()
205 {
206     Stack globalStack = new Stack();
207     globalStack.push(root);
208     int nBlanks = 32;
209     boolean isRowEmpty = false;
210     System.out.println(
211         " ..... ");
212     while(isRowEmpty==false)
213     {
214         Stack localStack = new Stack();
215         isRowEmpty = true;
216
217         for(int j=0; j<nBlanks; j++)
218             System.out.print(' ');
219
220         while(globalStack.isEmpty()==false)
221         {
```

```

222         Node temp = (Node) globalStack.pop();
223         if (temp != null)
224         {
225             System.out.print(temp.iData);
226             localStack.push(temp.leftChild);
227             localStack.push(temp.rightChild);
228
229             if (temp.leftChild != null ||
230                 temp.rightChild != null)
231                 isRowEmpty = false;
232         }
233         else
234         {
235             System.out.print("--");
236             localStack.push(null);
237             localStack.push(null);
238         }
239         for (int j=0; j<nBlanks*2-2; j++)
240             System.out.print(' ');
241     } // end while globalStack not empty
242     System.out.println();
243     nBlanks /= 2;
244     while (localStack.isEmpty() == false)
245         globalStack.push( localStack.pop() );
246 } // end while isRowEmpty is false
247 System.out.println(
248     " ..... ");
249 } // end displayTree()
250 //-----
251 } // end class Tree

```



← Get code here.

Code 6.3: Class TreeApp (from book)

```
1 import java.io.*;
2 class TreeApp
3 {
4     public static void main(String[] args) throws IOException
5     {
6         int value;
7         Tree theTree = new Tree();
8         theTree.insert(50);
9         theTree.insert(25);
10        theTree.insert(75);
11        theTree.insert(12);
12        theTree.insert(37);
13        theTree.insert(43);
14        theTree.insert(30);
15        theTree.insert(33);
16        theTree.insert(87);
17        theTree.insert(93);
18        theTree.insert(97);
19
20        while(true)
21        {
22            System.out.print("Enter first letter of show, ");
23            System.out.print("insert, find, delete, or traverse: ");
24            int choice = getChar();
25            switch(choice)
26            {
27                case 's':
28                    theTree.displayTree();
29                    break;
30                case 'i':
31                    System.out.print("Enter value to insert: ");
32                    value = getInt();
33                    theTree.insert(value);
34                    break;
35                case 'f':
36                    System.out.print("Enter value to find: ");
37                    value = getInt();
38                    Node found = theTree.find(value);
39                    if(found != null)
40                    {
41                        System.out.print("Found: ");
42                        found.displayNode();
43                        System.out.print("\n");
44                    }
45                else
46                    System.out.print("Could not find ");
47                System.out.print(value + '\n');
48                break;
49            }
50        }
51    }
52 }
```

```

49         case 'd':
50             System.out.print("Enter value to delete: ");
51             value = getInt();
52             boolean didDelete = theTree.delete(value);
53             if(didDelete)
54                 System.out.print("Deleted " + value + '\n');
55             else
56                 System.out.print("Could not delete ");
57             System.out.print(value + '\n');
58             break;
59         case 't':
60             System.out.print("Enter type 1, 2 or 3: ");
61             value = getInt();
62             theTree.traverse(value);
63             break;
64         default:
65             System.out.print("Invalid entry\n");
66     } // end switch
67 } // end while
68 } // end main()
69 //-----
70 public static String getString() throws IOException
71 {
72     InputStreamReader isr = new InputStreamReader(System.in);
73     BufferedReader br = new BufferedReader(isr);
74     String s = br.readLine();
75     return s;
76 }
77 //-----
78 public static char getChar() throws IOException
79 {
80     String s = getString();
81     return s.charAt(0);
82 }
83 //-----
84 public static int getInt() throws IOException
85 {
86     String s = getString();
87     return Integer.parseInt(s);
88 }
89 //-----
90 } // end class TreeApp

```



← Get code here.



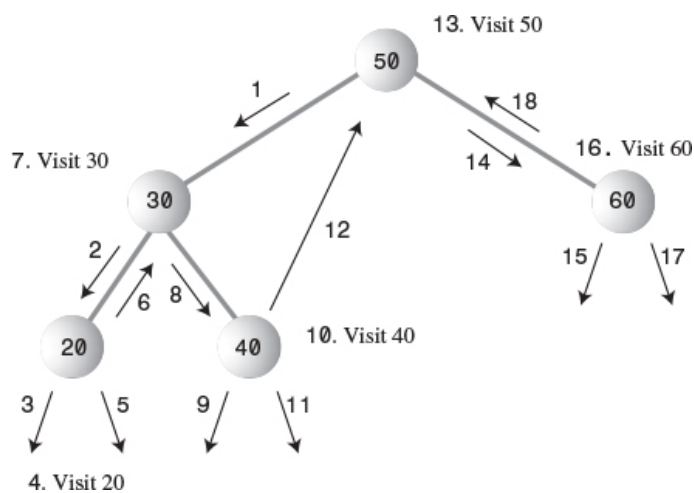
Analyze the complexity of each operation on a Binary Search Tree.

6.3 Traversing a Tree

Traversing a tree means visiting each node in a specified order. The simplest way to carry out a traversal is the use of recursion.

6.3.1 In Order Traversal

- Call itself to traverse the node's left subtree.
- Visit the node.
- Call itself to traverse the node's right subtree.



6.3.2 Pre Order Traversal

- Visit the node.
- Call itself to traverse the node's left subtree.
- Call itself to traverse the node's right subtree.

6.3.3 Post Order Traversal

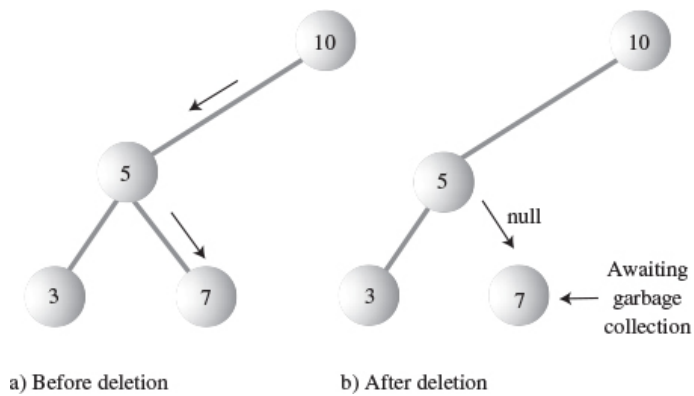
- Call itself to traverse the node's left subtree.
- Call itself to traverse the node's right subtree.
- Visit the node.

6.4 Deleting a Node from a Tree

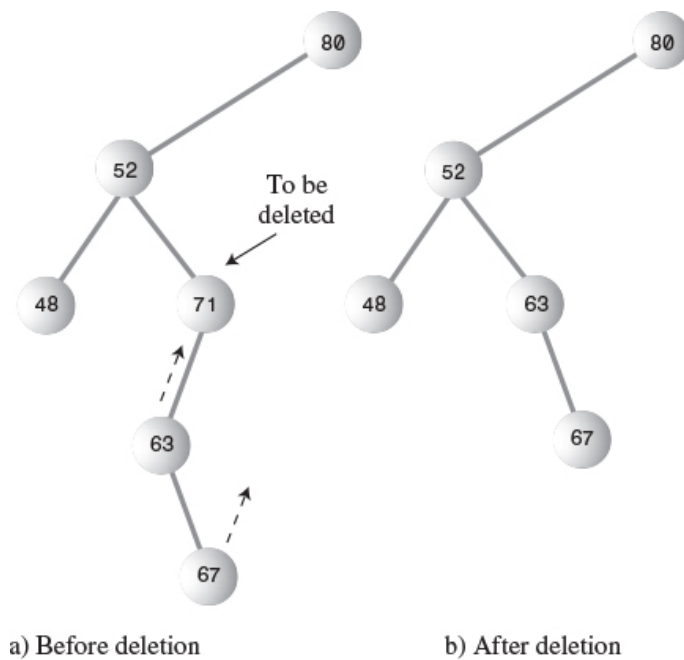
Deleting a node is the most complicated common operation required for binary search trees. Once we find the node to be deleted, there are three cases to consider:

- The node to be deleted is a leaf (has no children).
- The node to be deleted has one child.
- The node to be deleted has two children.

6.4.1 Case 1: No Children

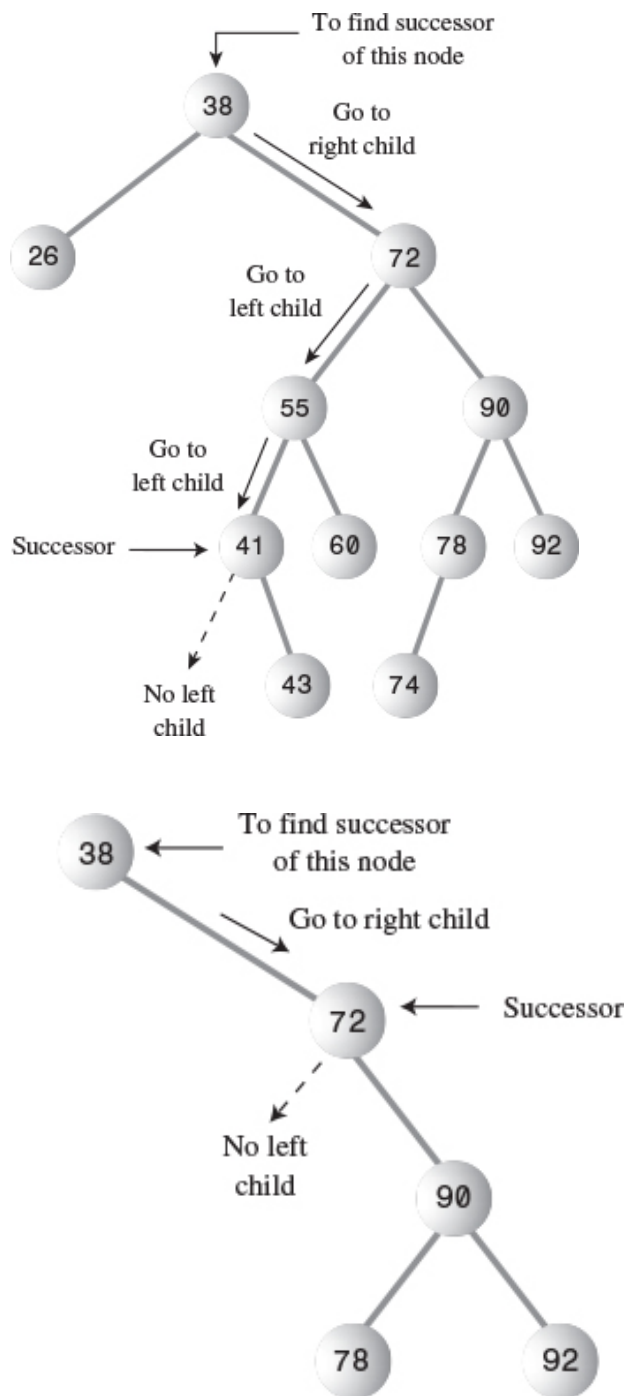


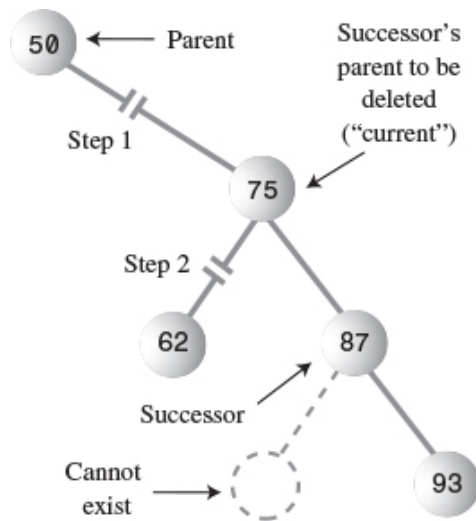
6.4.2 Case 2: One Child



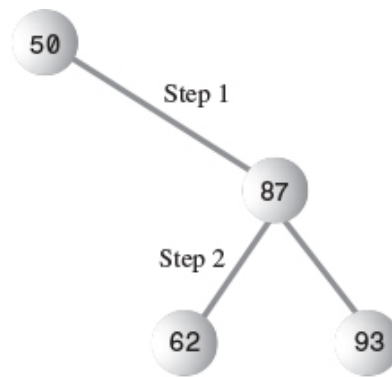
6.4.3 Case 3: Two Children

Find Successor

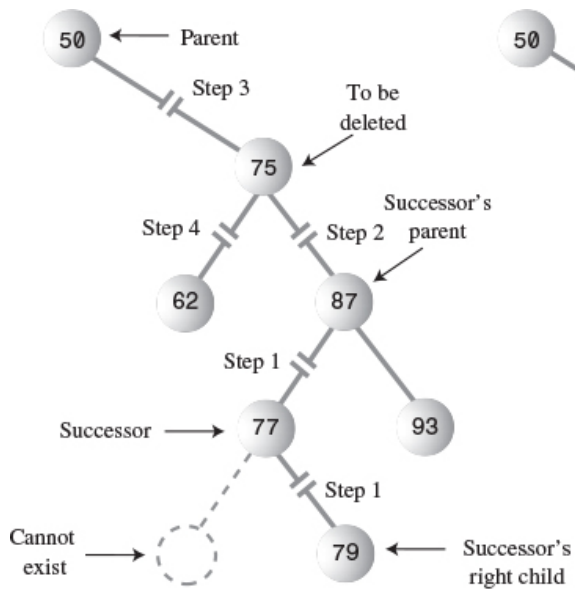


No Left Successor

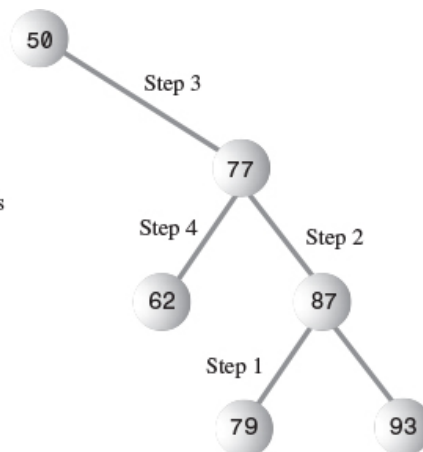
a) Before deletion



b) After deletion

With Left Successor

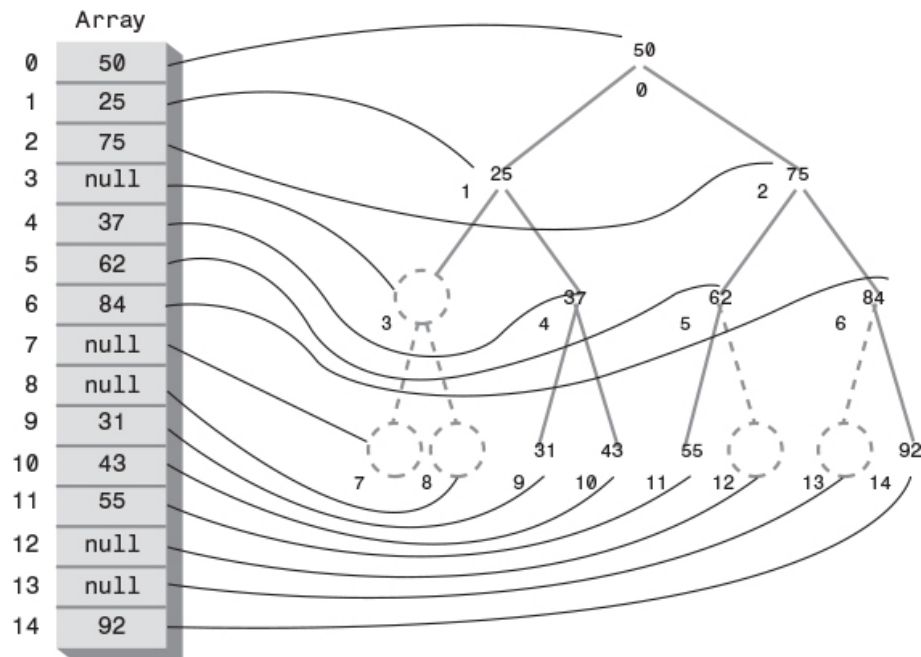
a) Before deletion



b) After deletion

6.5 Trees Represented as Arrays

In the array approach, the nodes are stored in an array and are not linked by references. The position of the node in the array corresponds to its position in the tree. The node at index 0 is the root, the node at index 1 is the root's left child, and so on, progressing from left to right along each level of the tree.



Insert the following nodes in a binary tree: 15, 9, 11, 18, 25, 6, 4, 28. Use an array to store the information.



How large should the array be if we wish to insert 500 nodes into the binary tree?



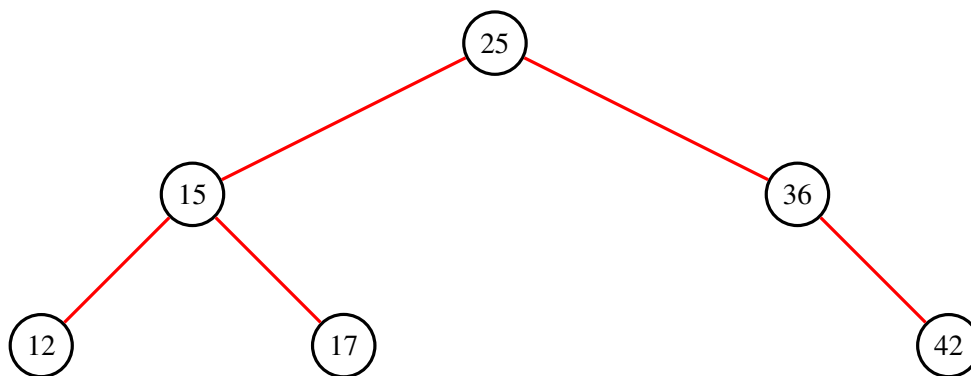
What are the advantages or disadvantages of using an array based binary tree?

6.6 Practice Questions

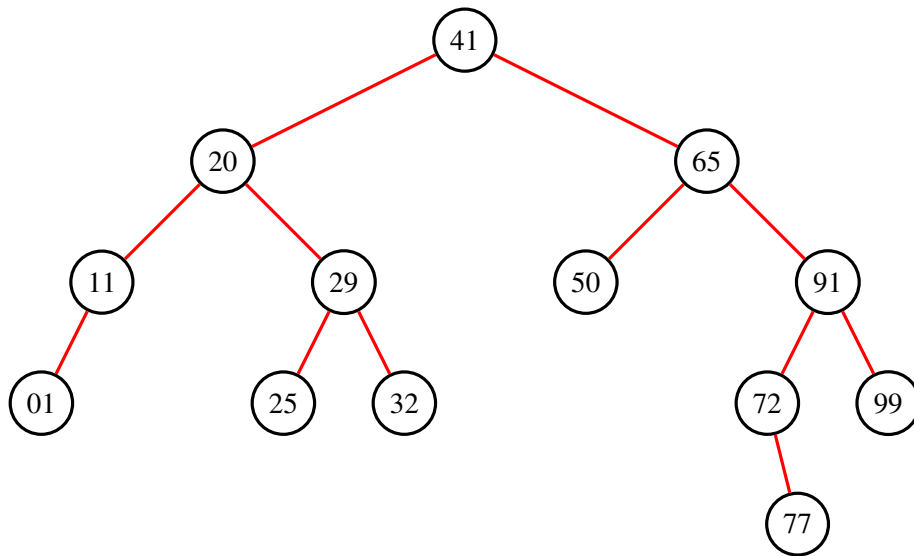
1. **True or False:** In a binary search tree (BST), the left subtree of a node contains only nodes with keys less than the node's key.
2. **True or False:** The height of a binary search tree is always $O(\log n)$ when the tree is balanced.
3. **True or False:** In a binary search tree, the maximum number of children a node can have is two.
4. **True or False:** A binary search tree with only one node can be considered a valid BST.
5. **True or False:** A binary search tree can have duplicate values as long as they are properly arranged within the tree.
6. **True or False:** A binary search tree with a skewed structure, where all nodes have only right children, has a height of $O(n)$.
7. **True or False:** The inorder traversal of a binary search tree results in a sorted list of its elements.
8. **True or False:** The worst-case time complexity of searching for an element in a balanced binary search tree is $O(\log n)$.
9. **True or False:** The time complexity of inserting a node into a binary search tree is $O(\log n)$ in the average case.
10. **True or False:** In a binary search tree, the maximum element is always the rightmost leaf node.
11. Insertion and deletion in a tree require what big O time?
12. A binary tree is a search tree if
 - (a) every non-leaf node has children whose key values are less than (or equal to) the parent.

- (b) every left child has a key less than the parent and every right child has a key greater than (or equal to) the parent.
 - (c) in the path from the root to every leaf node, the key of each node is greater than (or equal to) the key of its parent.
 - (d) a node can have a maximum of two children.
13. **True or False:** Not all trees are binary trees.
14. In a complete binary tree with 20 nodes, and the root considered to be at level 0, how many nodes are there at level 4?
15. A subtree of a binary tree always has
- (a) a root that is a child of the main tree's root.
 - (b) a root unconnected to the main tree's root.
 - (c) fewer nodes than the main tree.
 - (d) a sibling with the same number of nodes.
16. Finding a node in a binary search tree involves going from node to node, asking
- (a) how big the node's key is in relation to the search key.
 - (b) how big the node's key is compared to its right or left children.
 - (c) what leaf node we want to reach.
 - (d) what level we are on.

17. An unbalanced tree is one
- (a) in which most of the keys have values greater than the average.
 - (b) whose behavior is unpredictable.
 - (c) in which the root or some other node has many more left children than right children, or vice versa.
 - (d) that is shaped like an umbrella.
18. **True or False:** Deleting a node with one child from a binary search tree involves finding that node's successor.
19. **True or False:** The following is a binary search tree:



20. Show the list of numbers printed by the **Pre-Order Traversal**, and **Post-Order Traversal** on the tree below:



21. Assume we wish to delete node 20 from the graph in the previous question. Which one will be the **successor** node? How about 65, or 41?

7. Hash Tables

7.1 Introduction

A hash table is a data structure that offers very fast insertion and searching. No matter how many data items there are, insertion and searching (and sometimes deletion) can take close to constant time: $O(1)$ in big O notation.

Hashing is the process of mapping a set of keys to a set of array indexes.



Why use array indexes to refer to data in memory?



How can we store student records in computer memory for very fast access?



What if we wish to store non-numerical keys, such as a dictionary of 50000 entries? Which hash function could be a good solution? How big should be the corresponding hash table?

7.2 Open Addressing

In open addressing, when a data item can't be placed at the index calculated by the hash function, another location in the array is sought.

7.2.1 Linear Probing

In linear probing we search sequentially for vacant cells. If 5,421 is occupied when we try to insert a data item there, we go to 5,422, then 5,423, and so on, incrementing the index until we find an empty cell.

Code 7.1: Class DataItem (from book)

```
1  class DataItem
2  {
3      private int iData;           // (could have more data)
4      //----- // data item (key)
5      public DataItem(int ii)      // constructor
6      { iData = ii; }
7      //-----
8      public int getKey()
9      { return iData; }
10     //-----
11 } // end class DataItem
```



← Get code here.

Code 7.2: Class HashTable (from book)

```

1  class HashTable
2  {
3      private DataItem[] hashArray;    // array holds hash table
4      private int arraySize;
5      private DataItem nonItem;        // for deleted items
6      //-----
7      public HashTable(int size)        // constructor
8      {
9          arraySize = size;
10         hashArray = new DataItem[ arraySize ];
11         nonItem = new DataItem(-1);    // deleted item key is -1
12     }
13     //-----
14     public void displayTable()
15     {
16         System.out.print("Table: ");
17         for(int j=0; j<arraySize; j++)
18         {
19             if(hashArray[j] != null)
20                 System.out.print(hashArray[j].getKey() + " ");
21             else
22                 System.out.print("** ");
23         }
24         System.out.println("");
25     }
26     //-----
27     public int hashFunc(int key)
28     {
29         return key % arraySize;        // hash function
30     }
31     //-----
32     public void insert(DataItem item) // insert a DataItem
33     // (assumes table not full)
34     {
35         int key = item.getKey();        // extract key
36         int hashVal = hashFunc(key);    // hash the key
37         // until empty cell or -1,
38         while(hashArray[hashVal] != null &&
39             hashArray[hashVal].getKey() != -1)
40         {
41             ++hashVal;                  // go to next cell
42             hashVal %= arraySize;        // wraparound if necessary
43         }
44         hashArray[hashVal] = item;      // insert item
45     } // end insert()
46     //-----
47     public DataItem delete(int key)    // delete a DataItem
48     {

```



```

49     int hashVal = hashFunc(key); // hash the key
50
51     while(hashArray[hashVal] != null) // until empty cell,
52     {                                  // found the key?
53         if(hashArray[hashVal].getKey() == key)
54         {
55             DataItem temp = hashArray[hashVal]; // save item
56             hashArray[hashVal] = null;          // delete item
57             return temp;                        // return item
58         }
59         ++hashVal;                             // go to next cell
60         hashVal %= arraySize;                   // wraparound if necessary
61     }
62     return null;                               // can't find item
63 } // end delete()
64 //-----
65 public DataItem find(int key) // find item with key
66 {
67     int hashVal = hashFunc(key); // hash the key
68
69     while(hashArray[hashVal] != null) // until empty cell,
70     {                                  // found the key?
71         if(hashArray[hashVal].getKey() == key)
72             return hashArray[hashVal]; // yes, return item
73         ++hashVal;                     // go to next cell
74         hashVal %= arraySize;          // wraparound if necessary
75     }
76     return null;                       // can't find item
77 }
78 //-----
79 } // end class HashTable

```



← Get code here.

Code 7.3: Class HashTableApp (from book)

```
1 import java.io.*;
2
3 class HashTableApp
4 {
5     public static void main(String[] args) throws IOException
6     {
7         DataItem aDataItem;
8         int aKey, size, n, keysPerCell;
9         // get sizes
10        System.out.print("Enter size of hash table: ");
11        size = getInt();
12        System.out.print("Enter initial number of items: ");
13        n = getInt();
14        keysPerCell = 10;
15        // make table
16        HashTable theHashTable = new HashTable(size);
17
18        for(int j=0; j<n; j++)          // insert data
19        {
20            aKey = (int)(java.lang.Math.random() *
21                      keysPerCell * size);
22            aDataItem = new DataItem(aKey);
23            theHashTable.insert(aDataItem);
24        }
25
26        while(true)                    // interact with user
27        {
28            System.out.print("Enter first letter of ");
29            System.out.print("show, insert, delete, or find: ");
30            char choice = getChar();
31            switch(choice)
32            {
33                case 's':
34                    theHashTable.displayTable();
35                    break;
36                case 'i':
37                    System.out.print("Enter key value to insert: ");
38                    aKey = getInt();
39                    aDataItem = new DataItem(aKey);
40                    theHashTable.insert(aDataItem);
41                    break;
42                case 'd':
43                    System.out.print("Enter key value to delete: ");
44                    aKey = getInt();
45                    theHashTable.delete(aKey);
46                    break;
47                case 'f':
48                    System.out.print("Enter key value to find: ");
```

```

49         aKey = getInt();
50         aDataItem = theHashTable.find(aKey);
51         if(aDataItem != null)
52         {
53             System.out.println("Found " + aKey);
54         }
55         else
56             System.out.println("Could not find " + aKey);
57         break;
58     default:
59         System.out.print("Invalid entry\n");
60     } // end switch
61 } // end while
62 } // end main()
63 //-----
64 public static String getString() throws IOException
65 {
66     InputStreamReader isr = new InputStreamReader(System.in);
67     BufferedReader br = new BufferedReader(isr);
68     String s = br.readLine();
69     return s;
70 }
71 //-----
72 public static char getChar() throws IOException
73 {
74     String s = getString();
75     return s.charAt(0);
76 }
77 //-----
78 public static int getInt() throws IOException
79 {
80     String s = getString();
81     return Integer.parseInt(s);
82 }
83 //-----
84 } // end class HashTableApp

```



← Get code here.



What could we do if we run out of space using the current array?

7.2.2 Quadratic Probing

One of the issues with linear probing is *clustering*. Quadratic probing is an attempt to keep clusters from forming. The idea is to probe more widely separated cells, instead of those adjacent to the primary hash site.

In a linear probe, if the primary hash index is x , subsequent probes go to $x + 1$, $x + 2$, $x + 3$, and so on. In quadratic probing, probes go to $x + 1$, $x + 4$, $x + 9$, $x + 16$, $x + 25$, and so on. The distance from the initial probe is the square of the step number: $x + 1^2$, $x + 2^2$, $x + 3^2$, $x + 4^2$, $x + 5^2$, and so on.



What are possible issues with quadratic probing?

7.2.3 Double Hashing

To eliminate secondary clustering as well as primary clustering, we can use another approach: *double hashing*. The solution is to hash the key a second time, using a different hash function, and use the result as the step size. For a given key the step size remains constant throughout a probe, but it's different for different keys.

Experience has shown that this secondary hash function must have certain characteristics:

- It must not be the same as the primary hash function.
- It must never output a 0. (Why?)

Experts have discovered that functions of the following form work well:

$$stepSize = constant - (key \% constant);$$

where *constant* is prime and smaller than the array size.

Code 7.4: Class Double Hash Table (from book)

```

1  class HashTable
2  {
3      private DataItem[] hashArray;      // array is the hash table
4      private int arraySize;
5      private DataItem nonItem;          // for deleted items
6      //-----
7      HashTable(int size)                  // constructor
8      {
9          arraySize = size;
10         hashArray = new DataItem[ arraySize ];
11         nonItem = new DataItem(-1);
12     }
13     //-----
14     public void displayTable()
15     {
16         System.out.print("Table: ");
17         for(int j=0; j<arraySize; j++)
18         {
19             if(hashArray[j] != null)
20                 System.out.print(hashArray[j].getKey()+ " ");
21             else
22                 System.out.print("** ");
23         }
24         System.out.println("");
25     }
26     //-----
27     public int hashFunc1(int key)
28     {
29         return key % arraySize;
30     }
31     //-----
32     public int hashFunc2(int key)
33     {
34         // non-zero, less than array size, different from hF1
35         // array size must be relatively prime to 5, 4, 3, and 2
36         return 5 - key % 5;
37     }
38     //-----
39     // insert a DataItem
40     public void insert(int key, DataItem item)
41     // (assumes table not full)
42     {
43         int hashVal = hashFunc1(key); // hash the key
44         int stepSize = hashFunc2(key); // get step size
45         // until empty cell or -1
46         while(hashArray[hashVal] != null &&
47             hashArray[hashVal].getKey() != -1)
48         {

```

```

49         hashVal += stepSize;           // add the step
50         hashVal %= arraySize;          // for wraparound
51     }
52     hashArray[hashVal] = item;          // insert item
53 } // end insert()
54 //-----
55 public DataItem delete(int key)        // delete a DataItem
56 {
57     int hashVal = hashFunc1(key);       // hash the key
58     int stepSize = hashFunc2(key);      // get step size
59
60     while(hashArray[hashVal] != null)   // until empty cell,
61     {                                   // is correct hashVal?
62         if(hashArray[hashVal].getKey() == key)
63         {
64             DataItem temp = hashArray[hashVal]; // save item
65             hashArray[hashVal] = nonItem;        // delete item
66             return temp;                         // return item
67         }
68         hashVal += stepSize;                   // add the step
69         hashVal %= arraySize;                  // for wraparound
70     }
71     return null;                             // can't find item
72 } // end delete()
73 //-----
74 public DataItem find(int key)           // find item with key
75 // (assumes table not full)
76 {
77     int hashVal = hashFunc1(key);        // hash the key
78     int stepSize = hashFunc2(key);       // get step size
79
80     while(hashArray[hashVal] != null)    // until empty cell,
81     {                                   // is correct hashVal?
82         if(hashArray[hashVal].getKey() == key)
83         {
84             return hashArray[hashVal];    // yes, return item
85         }
86         hashVal += stepSize;              // add the step
87         hashVal %= arraySize;             // for wraparound
88     }
89     return null;                         // can't find item
90 } // end class HashTable

```



← Get code here.

Code 7.5: Class HashDoubleApp (from book)

```
1 import java.io.*;
2
3 class HashDoubleApp
4 {
5     public static void main(String[] args) throws IOException
6     {
7         int aKey;
8         DataItem aDataItem;
9         int size, n;
10        // get sizes
11        System.out.print("Enter size of hash table2: ");
12        size = getInt();
13        System.out.print("Enter initial number of items: ");
14        n = getInt();
15        // make table
16        HashTable theHashTable = new HashTable(size);
17
18        for(int j=0; j<n; j++)           // insert data
19        {
20            aKey = (int)(java.lang.Math.random() * 2 * size);
21            aDataItem = new DataItem(aKey);
22            theHashTable.insert(aKey, aDataItem);
23        }
24
25        while(true)                     // interact with user
26        {
27            System.out.print("Enter first letter of ");
28            System.out.print("show, insert, delete, or find: ");
29            char choice = getChar();
30            switch(choice)
31            {
32                case 's':
33                    theHashTable.displayTable();
34                    break;
35                case 'i':
36                    System.out.print("Enter key value to insert: ");
37                    aKey = getInt();
38                    aDataItem = new DataItem(aKey);
39                    theHashTable.insert(aKey, aDataItem);
40                    break;
41                case 'd':
42                    System.out.print("Enter key value to delete: ");
43                    aKey = getInt();
44                    theHashTable.delete(aKey);
45                    break;
46                case 'f':
47                    System.out.print("Enter key value to find: ");
48                    aKey = getInt();
```

```

49         aDataItem = theHashTable.find(aKey);
50         if(aDataItem != null)
51             System.out.println("Found " + aKey);
52         else
53             System.out.println("Could not find " + aKey);
54         break;
55     default:
56         System.out.print("Invalid entry\n");
57     } // end switch
58 } // end while
59 } // end main()
60 //-----
61 public static String getString() throws IOException
62 {
63     InputStreamReader isr = new InputStreamReader(System.in);
64     BufferedReader br = new BufferedReader(isr);
65     String s = br.readLine();
66     return s;
67 }
68 //-----
69 public static char getChar() throws IOException
70 {
71     String s = getString();
72     return s.charAt(0);
73 }
74 //-----
75 public static int getInt() throws IOException
76 {
77     String s = getString();
78     return Integer.parseInt(s);
79 }
80 //-----
81 } // end class HashDoubleApp

```



← Get code here.

7.3 Separate Chaining

In open addressing, collisions are resolved by looking for an open cell in the hash table. A different approach is to install a linked list at each index in the hash table. A data item's key is hashed to the index in the usual way, and the item is inserted into the linked list at that index. Other items that hash to the same index are simply added to the linked list.

Code 7.6: Class Separate Chaining Link (from book)

```
1  class Link
2  {                                     // (could be other items)
3      private int iData;               // data item
4      public Link next;                // next link in list
5      //-----
6      public Link(int it)              // constructor
7      { iData= it; }
8      //-----
9      public int getKey()
10     { return iData; }
11     //-----
12     public void displayLink()         // display this link
13     { System.out.print(iData + " "); }
14 } // end class Link
```



← Get code here.

Code 7.7: Class Separate Chaining Sorted List (from book)

```

1  class SortedList
2  {
3      private Link first;           // ref to first list item
4      //-----
5      public void SortedList()      // constructor
6      { first = null; }
7      //-----
8      public void insert(Link theLink) // insert link, in order
9      {
10         int key = theLink.getKey();
11         Link previous = null;      // start at first
12         Link current = first;
13         // until end of list,
14         while( current != null && key > current.getKey() )
15         {                          // or current > key,
16             previous = current;
17             current = current.next; // go to next item
18         }
19         if(previous==null)          // if beginning of list,
20             first = theLink;        // first --> new link
21         else                        // not at beginning,
22             previous.next = theLink; // prev --> new link
23         theLink.next = current;     // new link --> current
24     } // end insert()
25     //-----
26     public void delete(int key)     // delete link
27     {                               // (assumes non-empty list)
28         Link previous = null;      // start at first
29         Link current = first;
30         // until end of list,
31         while( current != null && key != current.getKey() )
32         {                          // or key == current,
33             previous = current;
34             current = current.next; // go to next link
35         }
36         // disconnect link
37         if(previous==null)          // if beginning of list
38             first = first.next;     // delete first link
39         else                        // not at beginning
40             previous.next = current.next; // delete current link
41     } // end delete()
42     //-----
43     public Link find(int key)       // find link
44     {
45         Link current = first;      // start at first
46         // until end of list,
47         while(current != null && current.getKey() <= key)
48         {                          // or key too small,

```

```
49         if(current.getKey() == key)    // is this the link?
50             return current;           // found it, return link
51         current = current.next;         // go to next item
52     }
53     return null;                       // didn't find it
54 } // end find()
55 //-----
56 public void displayList()
57 {
58     System.out.print("List (first-->last): ");
59     Link current = first;              // start at beginning of list
60     while(current != null)             // until end of list,
61     {
62         current.displayLink();         // print data
63         current = current.next;        // move to next link
64     }
65     System.out.println("");
66 }
67 } // end class SortedList
```



<— Get code here.

Code 7.8: Class Separate Chaining Hash Table (from book)

```

1  class HashTable
2  {
3      private SortedList[] hashArray;    // array of lists
4      private int arraySize;
5      //-----
6      public HashTable(int size)          // constructor
7      {
8          arraySize = size;
9          hashArray = new SortedList[arraySize]; // create array
10         for(int j=0; j<arraySize; j++)      // fill array
11             hashArray[j] = new SortedList(); // with lists
12     }
13     //-----
14     public void displayTable()
15     {
16         for(int j=0; j<arraySize; j++) { // for each cell,
17             System.out.print(j + ". "); // display cell number
18             hashArray[j].displayList(); // display list
19         }
20     }
21     //-----
22     public int hashFunc(int key)          // hash function
23     { return key % arraySize; }
24     //-----
25     public void insert(Link theLink)     // insert a link
26     {
27         int key = theLink.getKey();
28         int hashVal = hashFunc(key);    // hash the key
29         hashArray[hashVal].insert(theLink); // insert at hashVal
30     } // end insert()
31     //-----
32     public void delete(int key)          // delete a link
33     {
34         int hashVal = hashFunc(key);    // hash the key
35         hashArray[hashVal].delete(key); // delete link
36     } // end delete()
37     //-----
38     public Link find(int key)            // find link
39     {
40         int hashVal = hashFunc(key);    // hash the key
41         Link theLink = hashArray[hashVal].find(key); // get link
42         return theLink;                 // return link
43     }
44 } // end class HashTable

```



← Get code here.

Code 7.9: Class Separate Chaining App (from book)

```
1 import java.io.*;
2
3 class HashChainApp
4 {
5     public static void main(String[] args) throws IOException
6     {
7         int aKey;
8         Link aDataItem;
9         int size, n, keysPerCell = 100;
10        // get sizes
11        System.out.print("Enter size of hash table: ");
12        size = getInt();
13        System.out.print("Enter initial number of items: ");
14        n = getInt();
15        // make table
16        HashTable theHashTable = new HashTable(size);
17
18        for(int j=0; j<n; j++)           // insert data
19        {
20            aKey = (int)(java.lang.Math.random() *
21                      keysPerCell * size);
22            aDataItem = new Link(aKey);
23            theHashTable.insert(aDataItem);
24        }
25        while(true)                     // interact with user
26        {
27            System.out.print("Enter first letter of ");
28            System.out.print("show, insert, delete, or find: ");
29            char choice = getChar();
30            switch(choice)
31            {
32                case 's':
33                    theHashTable.displayTable();
34                    break;
35                case 'i':
36                    System.out.print("Enter key value to insert: ");
37                    aKey = getInt();
38                    aDataItem = new Link(aKey);
39                    theHashTable.insert(aDataItem);
40                    break;
41                case 'd':
42                    System.out.print("Enter key value to delete: ");
43                    aKey = getInt();
44                    theHashTable.delete(aKey);
45                    break;
46                case 'f':
47                    System.out.print("Enter key value to find: ");
48                    aKey = getInt();
```

```

49         aDataItem = theHashTable.find(aKey);
50         if(aDataItem != null)
51             System.out.println("Found " + aKey);
52         else
53             System.out.println("Could not find " + aKey);
54         break;
55     default:
56         System.out.print("Invalid entry\n");
57     } // end switch
58 } // end while
59 } // end main()
60 //-----
61 public static String getString() throws IOException
62 {
63     InputStreamReader isr = new InputStreamReader(System.in);
64     BufferedReader br = new BufferedReader(isr);
65     String s = br.readLine();
66     return s;
67 }
68 //-----
69 public static char getChar() throws IOException
70 {
71     String s = getString();
72     return s.charAt(0);
73 }
74 //-----
75 public static int getInt() throws IOException
76 {
77     String s = getString();
78     return Integer.parseInt(s);
79 }
80 //-----
81 } // end class HashChainApp

```



← Get code here.



Do a comparative analysis of various hashing techniques discussed in this chapter.

Practice Questions

1. Using big O notation, say how long it takes (ideally) to find an item in a hash table.
2. A transforms a range of key values into a range of index values.
3. Open addressing refers to
 - (a) keeping many of the cells in the array unoccupied.
 - (b) keeping an open mind about which address to use.
 - (c) probing at cell $x+1$, $x+2$, and so on until an empty cell is found.
 - (d) looking for another location in the array when the one you want is occupied.
4. What are the first five step sizes in quadratic probing?
5. Secondary clustering occurs because
 - (a) many keys hash to the same location.
 - (b) the sequence of step lengths is always the same.
 - (c) too many items with the same key are inserted.
 - (d) the hash function is not perfect.
6. Separate chaining involves the use of a at each location.
7. **True or False:** A possible hash function for strings involves multiplying each character by an ever-increasing power.
8. The best technique when the amount of data is not well known is
 - (a) linear probing.
 - (b) quadratic probing.
 - (c) double hashing.
 - (d) separate chaining.

9. **True or False:** In linear probing an unsuccessful search takes longer than a successful search.
10. In separate chaining the time to insert a new item
- (a) increases linearly with the load factor.
 - (b) is proportional to the number of items in the table.
 - (c) is proportional to the number of lists.
 - (d) is proportional to the percentage of full cells in the array.
11. Modify the Linear Hashing program to use quadratic probing.
12. Implement a linear probe hash table that stores strings. You'll need a hash function that converts a string to an index number; see the section "Hashing Strings" in the book. Assume the strings will be lowercase words, so 26 characters will suffice.
13. Write a *rehash()* method for the hash.java program. It should be called by *insert()* to move the entire hash table to an array about twice as large whenever the load factor exceeds 0.5. The new array size should be a prime number. Refer to the section "Expanding the Array" in the book. Don't forget you'll need to handle items that have been "deleted," that is, written over with -1.
14. Instead of using a linked list to resolve collisions, as in separate chaining, use a binary search tree. That is, create a hash table that is an array of trees. To display a small tree-based hash table, you could use an in order traversal of each tree.
- The advantage of a tree over a linked list is that it can be searched in $O(\log N)$ instead of $O(N)$ time. This time savings can be a significant advantage if very high load factors are encountered. Checking 15 items takes a maximum of 15 comparisons in a list but only 4 in a tree.
- Duplicates can present problems in both trees and hash tables, so add some code that prevents a duplicate key from being inserted in the hash table. (Beware: The *find()* method in Tree assumes a non-empty tree.) To shorten the listing for this program, you can forget about deletion, which for trees requires a lot of code.

15. Assume the array below is used to implement a hash table. In which array index would the number 30 be inserted using linear probing? How about 59, or 76?

0	1	2	3	4	5	6	7	8	9	10	11	12
		15	54			45	20				37	63

16. Assume the array below is used to implement a hash table. In which array index would the number 29 be inserted using quadratic probing? How about 40, or 66?

0	1	2	3	4	5	6	7	8	9	10	11	12
	62					45	32	34		75	76	12

17. Assume the array below is used to implement a hash table. What would be the content of the array after inserting the following list of numbers using double hashing? Use $(\% \text{ arraysize})$ as the first function, and $(5 - (\text{key} \% 5))$ as the second function.

List: 55, 34, 23,31,39,25, 47,38

[illegible]

8. Heaps

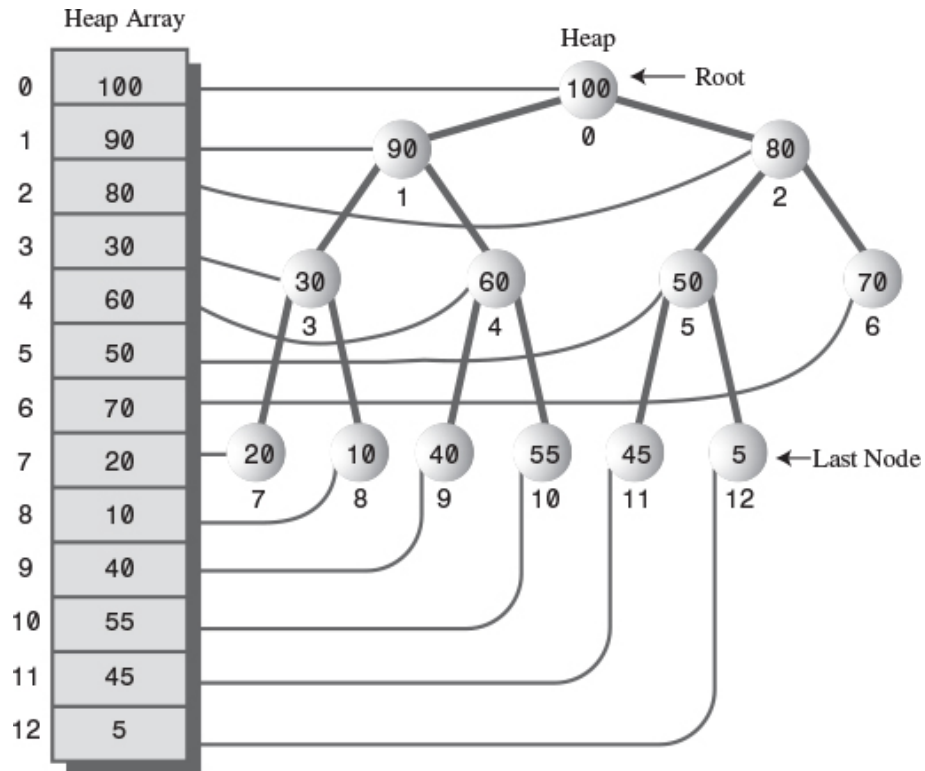
8.1 Introduction

Previously we saw a priority queue implemented as an ordered array. It had a fast removal of the largest item in $O(1)$ time, but insertion requires slow $O(N)$ time.

There is another alternative to implementing priority queues, using **heaps**.

A heap is a binary tree with these characteristics:

- It's complete. This means it's completely filled in, reading from left to right across each row, although the last row need not be full.
- It's (usually) implemented as an array.
- Each node in a heap satisfies the heap condition, which states that every nodes key is larger than (or equal to) the keys of its children.
- A priority queue based on such a heap is a *descending-priority queue*.

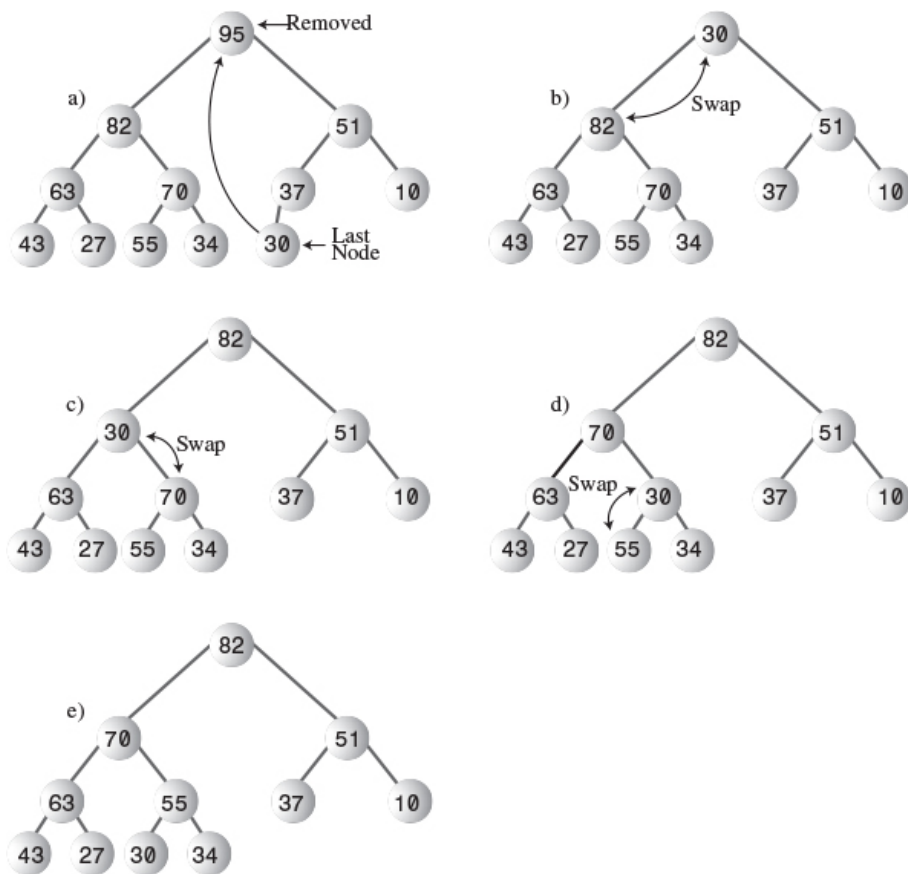


8.2 Heap Operations

8.2.1 Removal

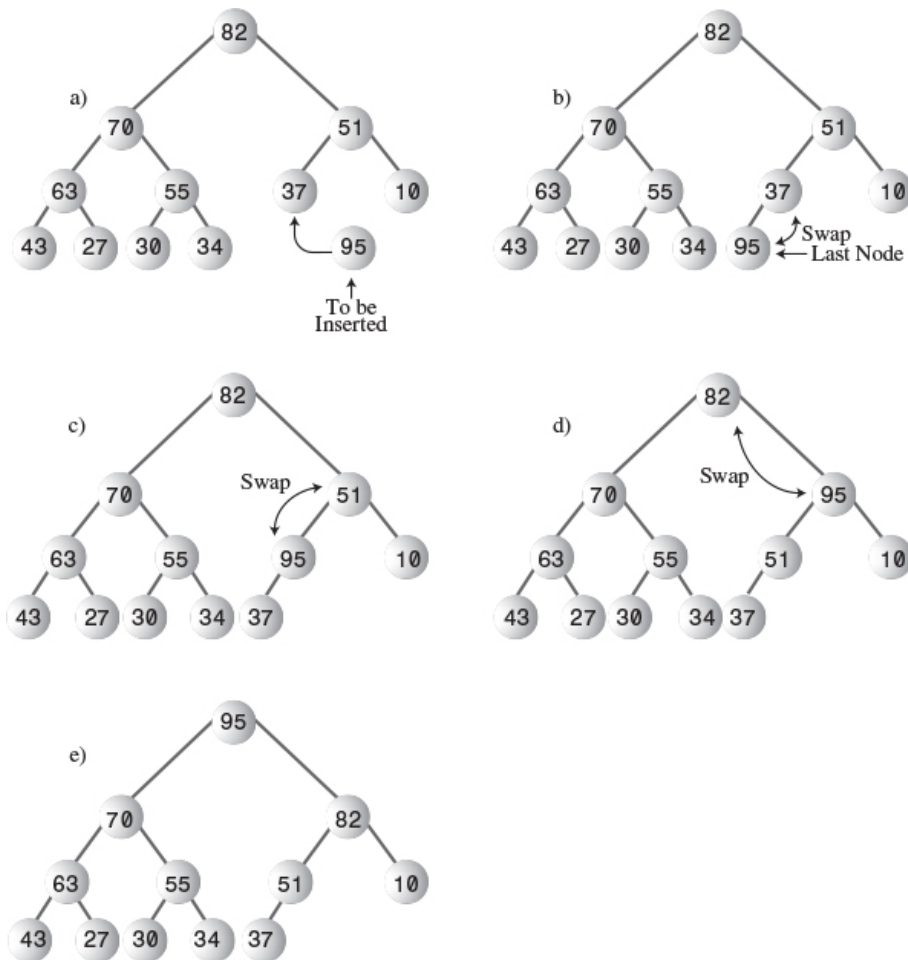
Removal means removing the node with the maximum key. This node is always the root, so removing it is easy. Here are the steps for removing the maximum node:

- Remove the root.
- Move the last node into the root.
- Trickle the last node down until it's below a larger node and above a smaller one.



8.2.2 Insertion

Inserting a node is also easy. Insertion uses trickle up, rather than trickle down.



8.3 Java Code

Here are some points to remember about representing a tree as an array. For a node at index x in the array,

- Its parent is $(x - 1)/2$.
- Its left child is $2 * x + 1$.
- Its right child is $2 * x + 2$.

Code 8.1: Class Heap Node (from book)

```
1  class Node
2  {
3      private int iData;           // data item (key)
4      //-----
5      public Node(int key)         // constructor
6      { iData = key; }
7      //-----
8      public int getKey()
9      { return iData; }
10     //-----
11     public void setKey(int id)
12     { iData = id; }
13     //-----
14 } // end class Node
```



← Get code here.

Code 8.2: Class Heap (from book)

```

1  class Heap
2  {
3      private Node[] heapArray;
4      private int maxSize;           // size of array
5      private int currentSize;       // number of nodes in array
6      //-----
7      public Heap(int mx)            // constructor
8      {
9          maxSize = mx;
10         currentSize = 0;
11         heapArray = new Node[maxSize]; // create array
12     }
13     //-----
14     public boolean isEmpty()
15     { return currentSize==0; }
16     //-----
17     public boolean insert(int key)
18     {
19         if(currentSize==maxSize)
20             return false;
21         Node newNode = new Node(key);
22         heapArray[currentSize] = newNode;
23         trickleUp(currentSize++);
24         return true;
25     } // end insert()
26     //-----
27     public void trickleUp(int index)
28     {
29         int parent = (index-1) / 2;
30         Node bottom = heapArray[index];
31
32         while( index > 0 &&
33             heapArray[parent].getKey() < bottom.getKey() )
34         {
35             heapArray[index] = heapArray[parent]; // move it down
36             index = parent;
37             parent = (parent-1) / 2;
38         } // end while
39         heapArray[index] = bottom;
40     } // end trickleUp()
41     //-----
42     public Node remove()            // delete item with max key
43     {                               // (assumes non-empty list)
44         Node root = heapArray[0];
45         heapArray[0] = heapArray[--currentSize];
46         trickleDown(0);
47         return root;
48     } // end remove()

```



```

49 //-----
50 public void trickleDown(int index)
51 {
52     int largerChild;
53     Node top = heapArray[index];           // save root
54     while(index < currentSize/2)           // while node has at
55     {                                       // least one child,
56         int leftChild = 2*index+1;
57         int rightChild = leftChild+1;
58         // find larger child
59         if(rightChild < currentSize && // (rightChild exists?)
60             heapArray[leftChild].getKey() <
61             heapArray[rightChild].getKey())
62             largerChild = rightChild;
63         else
64             largerChild = leftChild;
65         // top >= largerChild?
66         if( top.getKey() >= heapArray[largerChild].getKey() )
67             break;
68         // shift child up
69         heapArray[index] = heapArray[largerChild];
70         index = largerChild;               // go down
71     } // end while
72     heapArray[index] = top;                 // root to index
73 } // end trickleDown()
74 //-----
75 public boolean change(int index, int newValue)
76 {
77     if(index < 0 || index >= currentSize)
78         return false;
79     int oldValue = heapArray[index].getKey(); // remember old
80     heapArray[index].setKey(newValue); // change to new
81
82     if(oldValue < newValue)                 // if raised,
83         trickleUp(index);                 // trickle it up
84     else                                   // if lowered,
85         trickleDown(index);               // trickle it down
86     return true;
87 } // end change()
88 //-----
89 public void displayHeap()
90 {
91     System.out.print("heapArray: "); // array format
92     for(int m=0; m<currentSize; m++)
93         if(heapArray[m] != null)
94             System.out.print( heapArray[m].getKey() + " ");
95     else
96         System.out.print( "-- ");
97     System.out.println();

```

```

98      // heap format
99      int nBlanks = 32;
100     int itemsPerRow = 1;
101     int column = 0;
102     int j = 0;                                // current item
103     String dots = ".....";
104     System.out.println(dots+dots);            // dotted top line
105
106     while(currentSize > 0)                    // for each heap item
107     {
108         if(column == 0)                      // first item in row?
109             for(int k=0; k<nBlanks; k++)    // preceding blanks
110                 System.out.print(' ');
111         // display item
112         System.out.print(heapArray[j].getKey());
113
114         if(++j == currentSize)               // done?
115             break;
116
117         if(++column==itemsPerRow)            // end of row?
118         {
119             nBlanks /= 2;                    // half the blanks
120             itemsPerRow *= 2;                // twice the items
121             column = 0;                      // start over on
122             System.out.println();            // new row
123         }
124         else                                  // next item on row
125             for(int k=0; k<nBlanks*2-2; k++)
126                 System.out.print(' ');      // interim blanks
127     } // end for
128     System.out.println("\n"+dots+dots);    // dotted bottom line
129 } // end displayHeap()
130 //-----
131 } // end class Heap

```



← Get code here.


```

49         value2 = getInt();
50         success = theHeap.change(value, value2);
51         if( !success )
52             System.out.println("Invalid index");
53         break;
54     default:
55         System.out.println("Invalid entry\n");
56     } // end switch
57 } // end while
58 } // end main()
59 //-----
60 public static String getString() throws IOException
61 {
62     InputStreamReader isr = new InputStreamReader(System.in);
63     BufferedReader br = new BufferedReader(isr);
64     String s = br.readLine();
65     return s;
66 }
67 //-----
68 public static char getChar() throws IOException
69 {
70     String s = getString();
71     return s.charAt(0);
72 }
73 //-----
74 public static int getInt() throws IOException
75 {
76     String s = getString();
77     return Integer.parseInt(s);
78 }
79 //-----
80 } // end class HeapApp

```



← Get code here.



Analyze the efficiency of the binary heap data structure.

8.4 Heapify Methods: Top-Down vs Bottom-Up

When building a heap from an unordered array, there are two primary strategies: **top-down** (also known as successive insertions) and **bottom-up** (also known as heapify). Each has different performance characteristics and use cases.

8.4.1 Top-Down Heapify (Successive Insertions)

This method builds the heap by inserting elements one at a time into an initially empty heap, using the `trickleUp()` operation after each insertion.

- Start with an empty heap.
- Insert each element from the array into the heap.
- After each insertion, restore the heap property by trickling the new element up.

This approach is simple and intuitive, but it has a time complexity of $O(N \log N)$ in the worst case, since each insertion may require up to $\log N$ comparisons.

8.4.2 Bottom-Up Heapify

This method transforms an entire array into a heap in-place by working from the bottom of the tree upward, using the `trickleDown()` operation.

- Start from the last non-leaf node (at index $\lfloor N/2 \rfloor - 1$).
- Apply `trickleDown()` to each node in reverse level order.
- Continue until the root is processed.

This method is more efficient, with a time complexity of $O(N)$, because it performs fewer comparisons overall. It is the preferred method when building a heap from an existing array of elements.

Summary:

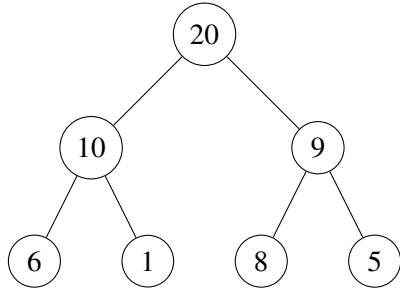
- **Top-Down:** Easy to implement, good for dynamic insertions, $O(N \log N)$.
- **Bottom-Up:** More efficient for bulk heap construction, $O(N)$.

Example: Top-Down Heapify

Given the array: [10, 20, 5, 6, 1, 8, 9]

- Start with an empty heap.
- Insert 10 → Heap: [10]
- Insert 20 → Trickle up → Heap: [20, 10]
- Insert 5 → Heap: [20, 10, 5]
- Insert 6 → Heap: [20, 10, 5, 6]
- Insert 1 → Heap: [20, 10, 5, 6, 1]
- Insert 8 → Trickle up → Heap: [20, 10, 8, 6, 1, 5]
- Insert 9 → Trickle up → Heap: [20, 10, 9, 6, 1, 5, 8]

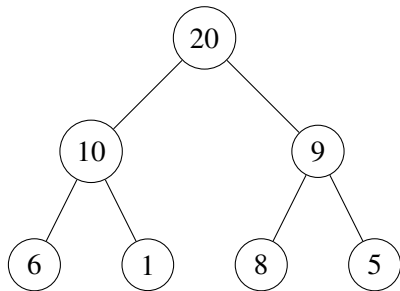
Each insertion may require a trickle-up operation, leading to a total time of $O(N \log N)$.

Top-Down Heapify (Final Heap)**Example: Bottom-Up Heapify**

Given the same array: [10, 20, 5, 6, 1, 8, 9]

- Treat the array as a complete binary tree.
- Start from the last non-leaf node: index 2 (value 5).
- Apply trickle-down:
 - Index 2 → Compare with children 8 and 9 → Swap with 9
 - Index 1 → Compare with children 6 and 1 → No swap
 - Index 0 → Compare with children 20 and 9 → Swap with 20
- Final heap: [20, 10, 9, 6, 1, 8, 5]

This method builds the heap in-place with fewer operations, resulting in $O(N)$ time.

Bottom-Up Heapify (Final Heap)**Question:**

Given the random array of values, build a max-heap using both top-down and bottom-up heapify methods. Will the final heap structures be the same?

8.5 Heapsort

The efficiency of the heap data structure lends itself to a surprisingly simple and very efficient sorting algorithm called **heapsort**.

The basic idea is to insert all the unordered items into a heap using the normal *insert()* routine. Repeated application of the *remove()* routine will then remove the items in sorted order.

Code 8.4: Class HeapSort (from book)

```

1  class Heap
2  {
3      private Node[] heapArray;
4      private int  maxSize;           // size of array
5      private int  currentSize;       // number of items in array
6      //-----
7      public Heap(int mx)             // constructor
8      {
9          maxSize = mx;
10         currentSize = 0;
11         heapArray = new Node[maxSize];
12     }
13     //-----
14     public Node remove()             // delete item with max key
15     {                               // (assumes non-empty list)
16         Node root = heapArray[0];
17         heapArray[0] = heapArray[--currentSize];
18         trickleDown(0);
19         return root;
20     } // end remove()
21     //-----
22     public void trickleDown(int index)
23     {
24         int largerChild;
25         Node top = heapArray[index]; // save root
26         while(index < currentSize/2) // not on bottom row
27         {
28             int leftChild = 2*index+1;
29             int rightChild = leftChild+1;
30             // find larger child
31             if(rightChild < currentSize && // right ch exists?
32                heapArray[leftChild].getKey() <
33                heapArray[rightChild].getKey())
34                 largerChild = rightChild;
35             else
36                 largerChild = leftChild;
37             // top >= largerChild?
38             if(top.getKey() >= heapArray[largerChild].getKey())

```

```

39         break;
40         // shift child up
41         heapArray[index] = heapArray[largerChild];
42         index = largerChild;           // go down
43     } // end while
44     heapArray[index] = top;           // root to index
45 } // end trickleDown()
46 //-----
47 public void displayHeap()
48 {
49     int nBlanks = 32;
50     int itemsPerRow = 1;
51     int column = 0;
52     int j = 0;                        // current item
53     String dots = ".....";
54     System.out.println(dots+dots);    // dotted top line
55
56     while(currentSize > 0)            // for each heap item
57     {
58         if(column == 0)               // first item in row?
59             for(int k=0; k<nBlanks; k++) // preceding blanks
60                 System.out.print(' ');
61         // display item
62         System.out.print(heapArray[j].getKey());
63
64         if(++j == currentSize)        // done?
65             break;
66
67         if(++column==itemsPerRow)     // end of row?
68         {
69             nBlanks /= 2;             // half the blanks
70             itemsPerRow *= 2;         // twice the items
71             column = 0;               // start over on
72             System.out.println();     // new row
73         }
74         else                          // next item on row
75             for(int k=0; k<nBlanks*2-2; k++)
76                 System.out.print(' '); // interim blanks
77     } // end for
78     System.out.println("\n"+dots+dots); // dotted bottom line
79 } // end displayHeap()
80 //-----
81 public void displayArray()
82 {
83     for(int j=0; j<maxSize; j++)
84         System.out.print(heapArray[j].getKey() + " ");
85     System.out.println("");
86 }
87 //-----

```



```

88     public void insertAt(int index , Node newNode)
89     { heapArray[index] = newNode; }
90     //-----
91     public void incrementSize()
92     { currentSize++; }
93     //-----
94 } // end class Heap

```



← Get code here.

Code 8.5: Class HeapSort App (from book)

```

1  import java.io.*;
2
3  class HeapSortApp
4  {
5      public static void main(String[] args) throws IOException
6      {
7          int size , j;
8
9          System.out.print("Enter number of items: ");
10         size = getInt();
11         Heap theHeap = new Heap(size);
12
13         for(j=0; j<size; j++)          // fill array with
14         {                               // random nodes
15             int random = (int)(java.lang.Math.random()*100);
16             Node newNode = new Node(random);
17             theHeap.insertAt(j , newNode);
18             theHeap.incrementSize();
19         }
20
21         System.out.print("Random: ");
22         theHeap.displayArray(); // display random array
23
24         for(j=size/2-1; j>=0; j--) // make random array into heap
25             theHeap.trickleDown(j);
26
27         System.out.print("Heap: ");
28         theHeap.displayArray(); // display heap array
29         theHeap.displayHeap(); // display heap
30
31         for(j=size-1; j>=0; j--) // remove from heap and
32         {                         // store at array end
33             Node biggestNode = theHeap.remove();
34             theHeap.insertAt(j , biggestNode);
35         }
36         System.out.print("Sorted: ");

```

```
37     theHeap.displayArray();    // display sorted array
38 } // end main()
39 //-----
40 public static String getString() throws IOException
41 {
42     InputStreamReader isr = new InputStreamReader(System.in);
43     BufferedReader br = new BufferedReader(isr);
44     String s = br.readLine();
45     return s;
46 }
47 //-----
48 public static int getInt() throws IOException
49 {
50     String s = getString();
51     return Integer.parseInt(s);
52 }
53 //-----
54 } // end class HeapSortApp
```



← Get code here.



Analyze the efficiency of HeapSort.

8.6 Practice Questions

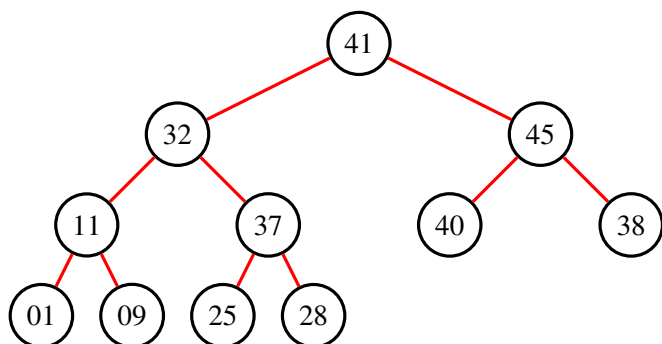
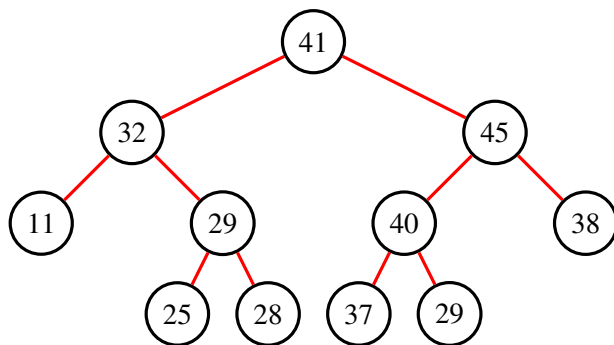
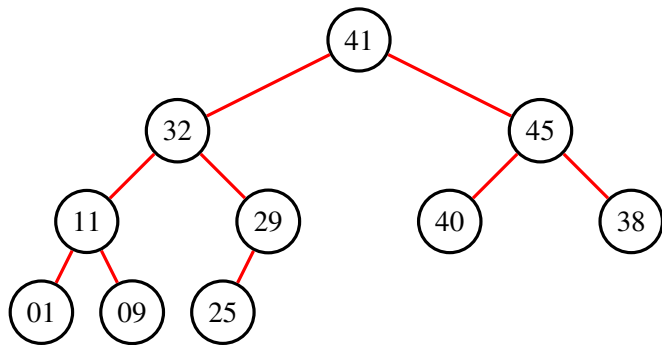
1. What does the term *complete* mean when applied to binary trees?
 - (a) All the necessary data has been inserted.
 - (b) All the rows are filled with nodes, except possibly the bottom one.
 - (c) All existing nodes contain data.
 - (d) The node arrangement satisfies the heap condition.
2. What does the term *weakly ordered* mean when applied to heaps?
3. To "trickle up" a node in a descending heap means
 - (a) to repeatedly exchange it with its parent until it's larger than its parent.
 - (b) to repeatedly exchange it with its child until it's larger than its child.
 - (c) to repeatedly exchange it with its child until it's smaller than its child.
 - (d) to repeatedly exchange it with its parent until it's smaller than its parent.
4. A heap can be represented by an array because a heap
 - (a) is complete.
 - (b) is weakly ordered.
 - (c) is a binary tree.
 - (d) satisfies the heap condition.
5. The last node in a heap is
 - (a) always a left child.
 - (b) always a right child.
 - (c) always on the bottom row.
 - (d) never less than its sibling.

6. A heap is to a priority queue as a(n) is to a stack.
7. A node is always removed from the
8. Insertion into a descending heap involves trickle
9. Heapsort involves
- (a) removing data from a heap and then inserting it again.
 - (b) inserting data into a heap and then removing it.
 - (c) copying data from one heap to another.
 - (d) copying data from the array representing a heap to the heap.
10. How many arrays, each big enough to hold all the data, does it take to sort a heap?
11. The array below is used to store the values from a heap. What would be the content of the array after we insert the value 99? How about 77?

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
95	79	68	40	52	49	55	37	28	21	36	24			

12. What would be the content of the previous array after we delete the maximum value?

13. Are the trees below, heap trees?

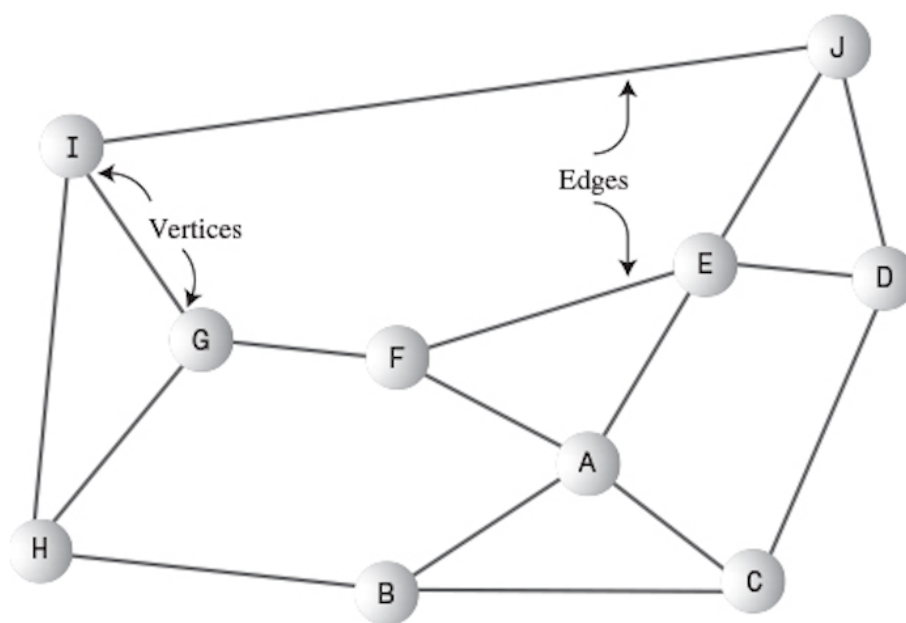


9. Graphs

9.1 Introduction and Review from MAC281

Graphs are one of the most versatile structures used in computer programming. Graphs are data structures rather like trees. In fact, in a mathematical sense, a tree is a kind of graph. In computer programming, however, graphs are used in different ways than trees.

Graphs are data structures that contain a set of vertices (nodes) and a set of edges (arcs).



Review of Terminology:

- **adjacent (neighbors):** two vertices connected by an edge
- **path:** a sequence of edges
- **connected graph:** there is at least one path from every vertex to every other vertex
- **directed / undirected graphs:** depending on whether edges can be traversed in one or both directions
- **weighted graphs:** each edge is assigned a value
- **adjacency matrix:** is a two-dimensional array in which the elements indicate whether an edge is present between two vertices.
- **adjacency list:** each vertex is associated with an individual list showing what vertices it is adjacent to.

9.2 Java Graph Code

Code 9.1: Class Vertex (from book)

```
1 class Vertex
2 {
3     public char label;           // label (e.g. 'A')
4     public boolean wasVisited;
5
6     public Vertex(char lab)      // constructor
7     {
8         label = lab;
9         wasVisited = false;
10    }
11 } // end class Vertex
```



← Get code here.

Code 9.2: Class Graph (from book)

```

1  class Graph
2  {
3      private final int MAX_VERTS = 20;
4      private Vertex vertexList[]; // array of vertices
5      private int adjMat[][];      // adjacency matrix
6      private int nVerts;          // current number of vertices
7      //-----
8      public Graph()               // constructor
9      {
10         vertexList = new Vertex[MAX_VERTS];
11         // adjacency matrix
12         adjMat = new int[MAX_VERTS][MAX_VERTS];
13         nVerts = 0;
14         for(int j=0; j<MAX_VERTS; j++) // set adjacency
15             for(int k=0; k<MAX_VERTS; k++) // matrix to 0
16                 adjMat[j][k] = 0;
17     } // end constructor
18     //-----
19     public void addVertex(char lab) // argument is label
20     {
21         vertexList[nVerts++] = new Vertex(lab);
22     }
23     //-----
24     public void addEdge(int start, int end)
25     {
26         adjMat[start][end] = 1;
27         adjMat[end][start] = 1;
28     }
29     //-----
30     public void displayVertex(int v)
31     {
32         System.out.print(vertexList[v].label);
33     }
34     //-----
35     public void displayMatrix()
36     {
37         System.out.println("Graph Matrix: ");
38         System.out.print(" ");
39         for(int i=0; i<nVerts; i++)
40             System.out.print(vertexList[i].label + " ");
41         System.out.println();
42
43         for(int i=0; i<nVerts; i++)
44         {
45             System.out.print(vertexList[i].label + " ");
46             for(int j=0; j<nVerts; j++)
47             {
48                 System.out.print(adjMat[i][j] + " ");

```

```
49         }
50         System.out.println();
51     }
52 }
53 //-----
54 } // end class Graph
```



← Get code here.

Code 9.3: Class GraphApp

```
1 public class GraphApp {
2     public static void main(String[] args)
3     {
4         Graph g = new Graph();
5
6         g.addVertex('A');
7         g.addVertex('B');
8         g.addVertex('C');
9         g.addVertex('D');
10        g.addEdge(1,2);
11        g.addEdge(0,1);
12
13        g.displayMatrix();
14    }
15 }
```



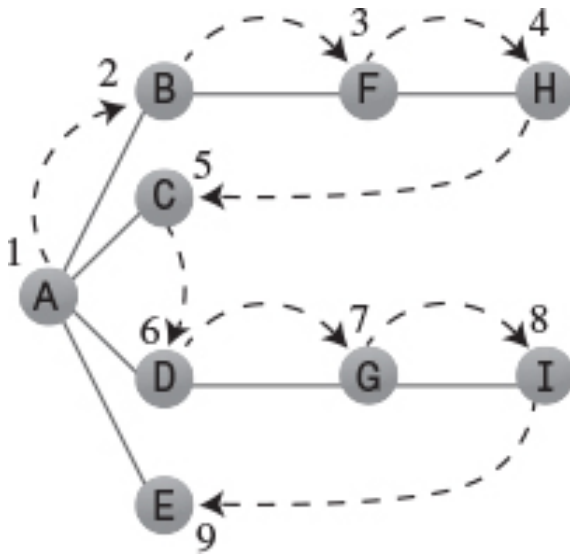
← Get code here.

9.3 Searches

One of the most fundamental operations to perform on a graph is finding which vertices can be reached from a specified vertex.

There are two common approaches to searching a graph: *depth-first search* (DFS) and *breadth-first search* (BFS). Both will eventually reach all connected vertices. The depth-first search is implemented with a stack, whereas the breadth-first search is implemented with a queue.

9.3.1 Depth-First Search



To carry out the depth-first search, you pick a starting point:

- **Rule 1:** If possible, visit an adjacent unvisited vertex, mark it, and push it on the stack.
- **Rule 2:** If you can't follow Rule 1, then, if possible, pop a vertex off the stack.
- **Rule 3:** If you can't follow Rule 1 or Rule 2, you're done.

Code 9.4: Class Vertex (from book)

```

1 class Vertex
2 {
3     public char label;           // label (e.g. 'A')
4     public boolean wasVisited;
5
6     public Vertex(char lab)      // constructor
7     {
8         label = lab;
9         wasVisited = false;
10    }
11 } // end class Vertex

```



← Get code here.

Code 9.5: Class StackX (from book)

```

1 class StackX
2 {
3     private final int SIZE = 20;
4     private int[] st;
5     private int top;
6     //-----
7     public StackX()           // constructor
8     {
9         st = new int[SIZE];    // make array
10        top = -1;
11    }
12    //-----
13    public void push(int j)    // put item on stack
14    { st[++top] = j; }
15    //-----
16    public int pop()           // take item off stack
17    { return st[top--]; }
18    //-----
19    public int peek()          // peek at top of stack
20    { return st[top]; }
21    //-----
22    public boolean isEmpty()    // true if nothing on stack -
23    { return (top == -1); }
24    //-----
25 } // end class StackX

```



← Get code here.

Code 9.6: Class Graph (from book)

```

1  class Graph
2  {
3      private final int MAX_VERTS = 20;
4      private Vertex vertexList[]; // list of vertices
5      private int adjMat[][];      // adjacency matrix
6      private int nVerts;          // current number of vertices
7      private StackX theStack;
8      //-----
9      public Graph()                // constructor
10     {
11         vertexList = new Vertex[MAX_VERTS];
12         // adjacency matrix
13         adjMat = new int[MAX_VERTS][MAX_VERTS];
14         nVerts = 0;
15         for(int j=0; j<MAX_VERTS; j++) // set adjacency
16             for(int k=0; k<MAX_VERTS; k++) // matrix to 0
17                 adjMat[j][k] = 0;
18         theStack = new StackX();
19     } // end constructor
20     //-----
21     public void addVertex(char lab)
22     { vertexList[nVerts++] = new Vertex(lab); }
23     //-----
24     public void addEdge(int start, int end)
25     {
26         adjMat[start][end] = 1;
27         adjMat[end][start] = 1;
28     }
29     //-----
30     public void displayVertex(int v)
31     { System.out.print(vertexList[v].label); }
32     //-----
33     public void dfs() // depth-first search
34     {
35         vertexList[0].wasVisited = true; // begin at vertex 0
36         displayVertex(0);                // mark it
37         theStack.push(0);                // display it
38                                         // push it
39         while( !theStack.isEmpty() )    // until stack empty,
40         {
41             // get an unvisited vertex adjacent to stack top
42             int v = getAdjUnvisitedVertex( theStack.peek() );
43             if(v == -1)                    // if no such vertex,
44                 theStack.pop();
45             else                            // if it exists,
46             {
47                 vertexList[v].wasVisited = true; // mark it
48                 displayVertex(v);                // display it

```

```

49         theStack.push(v);                // push it
50     }
51 } // end while
52
53 // stack is empty, so we're done
54 for(int j=0; j<nVerts; j++)                // reset flags
55     vertexList[j].wasVisited = false;
56 } // end dfs
57 //-----
58 // returns an unvisited vertex adj to v
59 public int getAdjUnvisitedVertex(int v)
60 {
61     for(int j=0; j<nVerts; j++)
62         if(adjMat[v][j]==1 && vertexList[j].wasVisited==false)
63             return j;
64     return -1;
65 } // end getAdjUnvisitedVertex()
66 //-----
67 } // end class Graph

```



← Get code here.

Code 9.7: Class DFSApp (from book)

```

1  class DFSApp
2  {
3      public static void main(String[] args)
4      {
5          Graph theGraph = new Graph();
6          theGraph.addVertex('A');    // 0  (start for dfs)
7          theGraph.addVertex('B');    // 1
8          theGraph.addVertex('C');    // 2
9          theGraph.addVertex('D');    // 3
10         theGraph.addVertex('E');    // 4
11
12         theGraph.addEdge(0, 1);      // AB
13         theGraph.addEdge(1, 2);      // BC
14         theGraph.addEdge(0, 3);      // AD
15         theGraph.addEdge(3, 4);      // DE
16
17         System.out.print("Visits: ");
18         theGraph.dfs();               // depth-first search
19         System.out.println();
20     } // end main()
21 } // end class DFSApp

```

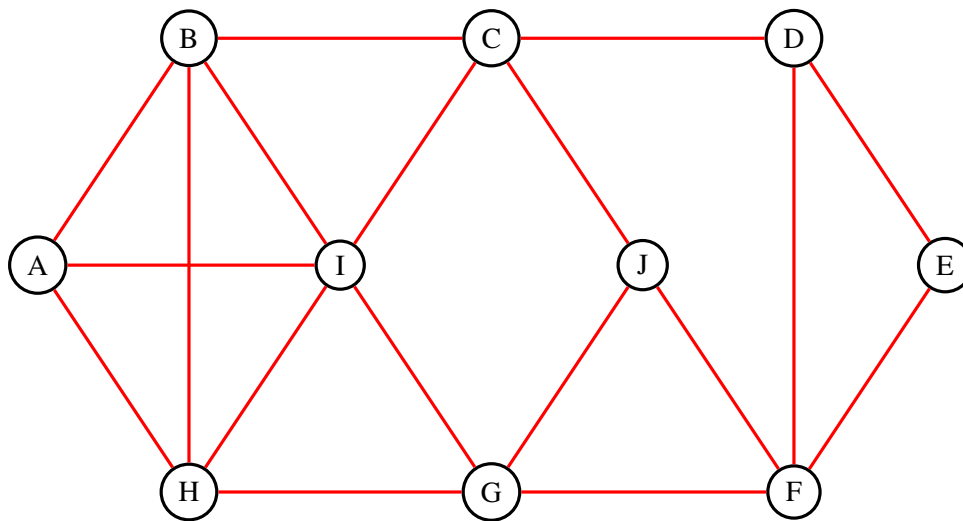


← Get code here.



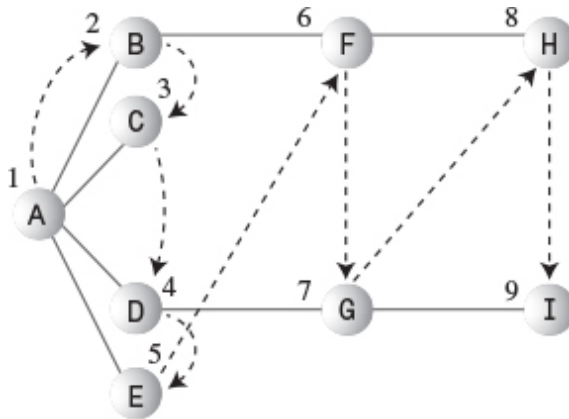
Consider the graph below. Determine the order in which the **DFS algorithm** will visit the vertices in the graph if we start from vertex A. Assume the usual alphabetical order when making choices.

Repeat the same algorithm starting from other nodes.



9.3.2 Breadth-First Search

In the breadth-first search, on the other hand, the algorithm likes to stay as close as possible to the starting point. It visits all the vertices adjacent to the starting vertex, and only then goes further afield. This kind of search is implemented using a queue instead of a stack.



To carry out the breadth-first search, you pick a starting point (current vertex):

- **Rule 1:** Visit the next unvisited vertex (if there is one) that's adjacent to the current vertex, mark it, and insert it into the queue.
- **Rule 2:** If you can't carry out Rule 1 because there are no more unvisited vertices, remove a vertex from the queue (if possible) and make it the current vertex.
- **Rule 3:** If you can't carry out Rule 2 because the queue is empty, you're done

Code 9.8: Class Vertex (from book)

```

1  class Vertex
2  {
3      public char label;           // label (e.g. 'A')
4      public boolean wasVisited;
5
6      public Vertex(char lab)     // constructor
7      {
8          label = lab;
9          wasVisited = false;
10     }
11 } // end class Vertex

```



← Get code here.

Code 9.9: Class Queue (from book)

```
1  class Queue
2  {
3      private final int SIZE = 20;
4      private int[] queArray;
5      private int front;
6      private int rear;
7      //-----
8      public Queue()           // constructor
9      {
10         queArray = new int[SIZE];
11         front = 0;
12         rear = -1;
13     }
14     //-----
15     public void insert(int j) // put item at rear of queue
16     {
17         if(rear == SIZE-1)
18             rear = -1;
19         queArray[++rear] = j;
20     }
21     //-----
22     public int remove()       // take item from front of queue
23     {
24         int temp = queArray[front++];
25         if(front == SIZE)
26             front = 0;
27         return temp;
28     }
29     //-----
30     public boolean isEmpty()  // true if queue is empty
31     {
32         return ( rear+1==front || (front+SIZE-1==rear) );
33     }
34     //-----
35 } // end class Queue
```



← Get code here.

Code 9.10: Class BFS Graph (from book)

```

1  class Graph
2  {
3      private final int MAX_VERTS = 20;
4      private Vertex vertexList[]; // list of vertices
5      private int adjMat[][];      // adjacency matrix
6      private int nVerts;           // current number of vertices
7      private Queue theQueue;
8      //-----
9      public Graph()                // constructor
10     {
11         vertexList = new Vertex[MAX_VERTS];
12         // adjacency matrix
13         adjMat = new int[MAX_VERTS][MAX_VERTS];
14         nVerts = 0;
15         for(int j=0; j<MAX_VERTS; j++) // set adjacency
16             for(int k=0; k<MAX_VERTS; k++) // matrix to 0
17                 adjMat[j][k] = 0;
18         theQueue = new Queue();
19     } // end constructor
20     //-----
21     public void addVertex(char lab)
22     { vertexList[nVerts++] = new Vertex(lab); }
23     //-----
24     public void addEdge(int start, int end)
25     {
26         adjMat[start][end] = 1;
27         adjMat[end][start] = 1;
28     }
29     //-----
30     public void displayVertex(int v)
31     { System.out.print(vertexList[v].label); }
32     //-----
33     public void bfs()                // breadth-first search
34     {                                // begin at vertex 0
35         vertexList[0].wasVisited = true; // mark it
36         displayVertex(0);                // display it
37         theQueue.insert(0);              // insert at tail
38         int v2;
39
40         while( !theQueue.isEmpty() )    // until queue empty,
41         {
42             int v1 = theQueue.remove(); // remove vertex at head
43             // until it has no unvisited neighbors
44             while( (v2=getAdjUnvisitedVertex(v1)) != -1 )
45             {                             // get one,
46                 vertexList[v2].wasVisited = true; // mark it
47                 displayVertex(v2);                // display it
48                 theQueue.insert(v2);              // insert it

```

```

49         } // end while
50     } // end while(queue not empty)
51
52     // queue is empty, so we're done
53     for(int j=0; j<nVerts; j++) // reset flags
54         vertexList[j].wasVisited = false;
55 } // end bfs()
56 //-----
57 // returns an unvisited vertex adj to v
58 public int getAdjUnvisitedVertex(int v)
59 {
60     for(int j=0; j<nVerts; j++)
61         if(adjMat[v][j]==1 && vertexList[j].wasVisited==false)
62             return j;
63     return -1;
64 } // end getAdjUnvisitedVertex()
65 //-----
66 } // end class Graph

```



← Get code here.

Code 9.11: Class BFS App (from book)

```

1  class BFSApp
2  {
3      public static void main(String[] args)
4      {
5          Graph theGraph = new Graph();
6          theGraph.addVertex('A'); // 0 (start for dfs)
7          theGraph.addVertex('B'); // 1
8          theGraph.addVertex('C'); // 2
9          theGraph.addVertex('D'); // 3
10         theGraph.addVertex('E'); // 4
11
12         theGraph.addEdge(0, 1); // AB
13         theGraph.addEdge(1, 2); // BC
14         theGraph.addEdge(0, 3); // AD
15         theGraph.addEdge(3, 4); // DE
16
17         System.out.print("Visits: ");
18         theGraph.bfs(); // breadth-first search
19         System.out.println();
20     } // end main()
21 } // end class BFSApp

```

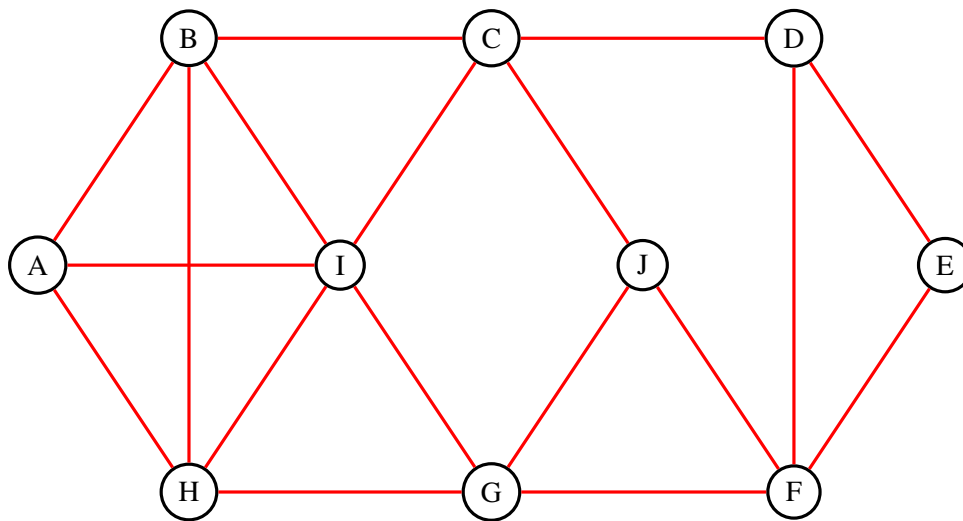


← Get code here.



Consider the graph below. Determine the order in which the **BFS algorithm** will visit the vertices in the graph if we start from vertex A. Assume the usual alphabetical order when making choices.

Repeat the same algorithm starting from other nodes.



9.4 Minimum Spanning Trees

Assume we have a connected graph. The goal is to find the minimum number of edges necessary to connect the vertices. This constitutes a *minimum spanning tree* (MST).

There are many possible minimum spanning trees for a given set of vertices.

The number of edges E in a minimum spanning tree is always one less than the number of vertices V : ($E = V - 1$)

The algorithm for creating the minimum spanning tree is almost identical to that used for searching. It can be based on either the depth-first search or the breadth-first search.

Note: The code below uses the Vertex class and the StackX class from the previous program.

Code 9.12: Class MST Graph (from book)

```

1  class Graph
2  {
3      private final int MAX_VERTS = 20;
4      private Vertex vertexList []; // list of vertices
5      private int adjMat [][];      // adjacency matrix
6      private int nVerts;           // current number of vertices
7      private StackX theStack;
8      //-----
9      public Graph()                // constructor
10     {
11         vertexList = new Vertex [MAX_VERTS];
12         // adjacency matrix
13         adjMat = new int [MAX_VERTS] [MAX_VERTS];
14         nVerts = 0;
15         for (int j=0; j<MAX_VERTS; j++) // set adjacency
16             for (int k=0; k<MAX_VERTS; k++) // matrix to 0
17                 adjMat[j][k] = 0;
18         theStack = new StackX ();
19     } // end constructor
20     //-----
21     public void addVertex (char lab)
22     {
23         vertexList[nVerts++] = new Vertex(lab);
24     }
25     //-----
26     public void addEdge (int start, int end)
27     {
28         adjMat[start][end] = 1;
29         adjMat[end][start] = 1;
30     }
31     //-----

```

```

32     public void displayVertex(int v)
33     {
34         System.out.print(vertexList[v].label);
35     }
36     //-----
37     public void mst() // minimum spanning tree (depth first)
38     {
39         // start at 0
40         vertexList[0].wasVisited = true; // mark it
41         theStack.push(0); // push it
42
43         while( !theStack.isEmpty() ) // until stack empty
44         { // get stack top
45             int currentVertex = theStack.peek();
46             // get next unvisited neighbor
47             int v = getAdjUnvisitedVertex(currentVertex);
48             if(v == -1) // if no more neighbors
49                 theStack.pop(); // pop it away
50             else // got a neighbor
51             {
52                 vertexList[v].wasVisited = true; // mark it
53                 theStack.push(v); // push it
54                 // display edge
55                 displayVertex(currentVertex); // from currentV
56                 displayVertex(v); // to v
57                 System.out.print(" ");
58             }
59         } // end while(stack not empty)
60
61         // stack is empty, so we're done
62         for(int j=0; j<nVerts; j++) // reset flags
63             vertexList[j].wasVisited = false;
64     } // end tree
65     //-----
66     // returns an unvisited vertex adj to v
67     public int getAdjUnvisitedVertex(int v)
68     {
69         for(int j=0; j<nVerts; j++)
70             if(adjMat[v][j]==1 && vertexList[j].wasVisited==false)
71                 return j;
72         return -1;
73     } // end getAdjUnvisitedVertex()
74     //-----
75 } // end class Graph

```



← Get code here.

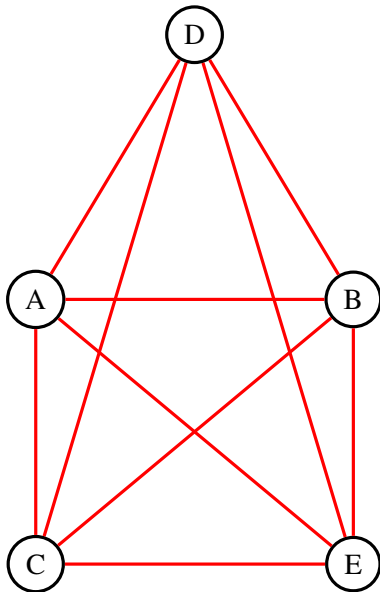
Code 9.13: Class MST App (from book)

```
1  class MSTApp
2  {
3      public static void main(String[] args)
4      {
5          Graph theGraph = new Graph();
6          theGraph.addVertex('A');    // 0  (start for mst)
7          theGraph.addVertex('B');    // 1
8          theGraph.addVertex('C');    // 2
9          theGraph.addVertex('D');    // 3
10         theGraph.addVertex('E');    // 4
11
12         theGraph.addEdge(0, 1);      // AB
13         theGraph.addEdge(0, 2);      // AC
14         theGraph.addEdge(0, 3);      // AD
15         theGraph.addEdge(0, 4);      // AE
16         theGraph.addEdge(1, 2);      // BC
17         theGraph.addEdge(1, 3);      // BD
18         theGraph.addEdge(1, 4);      // BE
19         theGraph.addEdge(2, 3);      // CD
20         theGraph.addEdge(2, 4);      // CE
21         theGraph.addEdge(3, 4);      // DE
22
23         System.out.print("Minimum spanning tree: ");
24         theGraph.mst();               // minimum spanning tree
25         System.out.println();
26     } // end main()
27 } // end class MSTApp
```

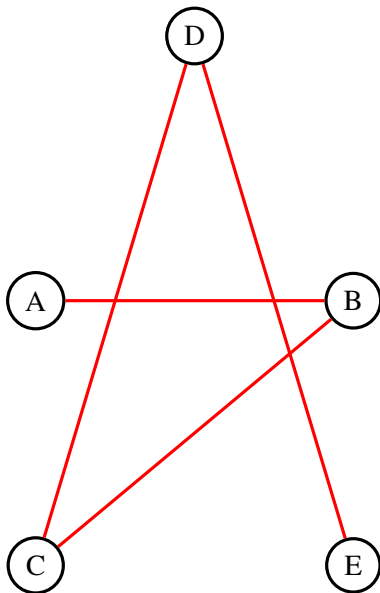


← Get code here.

Graph created by the code above.

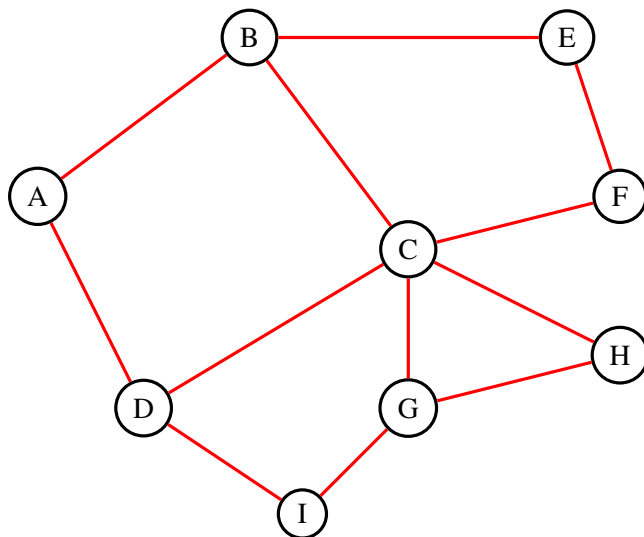


Minimum spanning tree from graph.





Find a minimum spanning tree for the graph below. Start with node A.



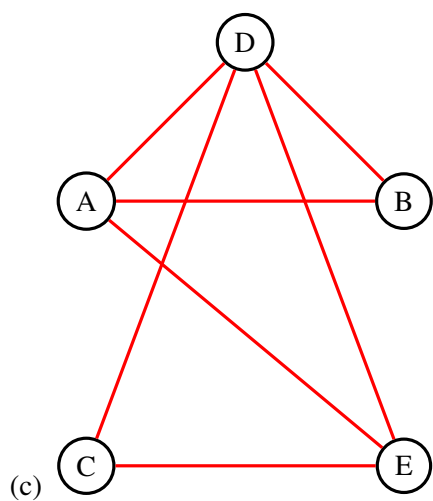
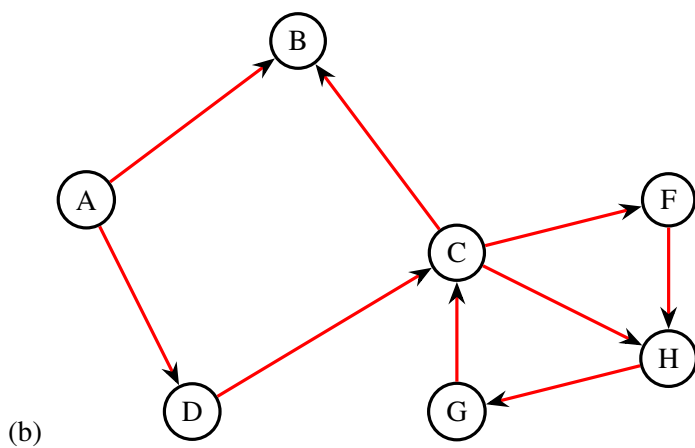
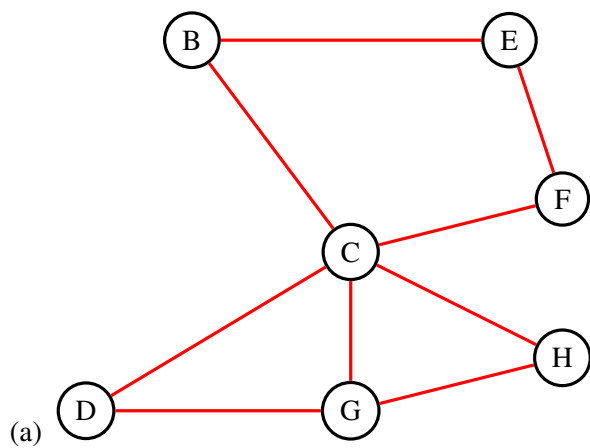
9.5 Practice Questions

1. (from book) How do you tell, by looking at its adjacency matrix, how many edges there are in an undirected graph?
2. (from book) In a game simulation, what graph entity corresponds to a choice about what move to make?
3. (from book) A directed graph is one in which
 - (a) you must follow the minimum spanning tree.
 - (b) you must go from vertex A to vertex B to vertex C and so on.
 - (c) you can go in only one direction from one given vertex to another.
 - (d) you can go in only one direction on any given path.
4. (from book) If an adjacency matrix has rows 0,1,0,0, 1,0,1,1, 0,1,0,0, and 0,1,0,0, what is the corresponding adjacency list?
5. (from book) A minimum spanning tree is a graph in which
 - (a) the number of edges connecting all the vertices is as small as possible.
 - (b) the number of edges is equal to the number of vertices.
 - (c) all unnecessary vertices have been removed.
 - (d) every combination of two vertices is connected by the minimum number of edges.
6. (from book) How many different minimum spanning trees are there in an undirected graph of three vertices and three edges?
7. (from book) An undirected graph must have a cycle if
 - (a) any vertex can be reached from some other vertex.
 - (b) the number of paths is greater than the number of vertices.
 - (c) the number of edges is equal to the number of vertices.
 - (d) the number of paths is less than the number of edges.

8. (from book) A is a graph with no cycles.
9. Can a minimum spanning tree for an undirected graph have cycles?
10. (from book) Can a tree have cycles?
11. Modify the BFS program to find the minimum spanning tree using a breadth-first search, rather than the depth-first search shown in the MST program. In `main()`, create a graph with 9 vertices and 12 edges, and find its minimum spanning tree.
12. (from book) Modify the DFS program to use adjacency lists rather than an adjacency matrix. You can obtain a list by adapting the `Link` and `LinkList` classes. Modify the `find()` routine from `LinkList` to search for an unvisited vertex rather than for a key value.
13. (from book) The Knight's Tour is an ancient and famous chess puzzle. The object is to move a knight from one square to another on an otherwise empty chess board until it has visited every square exactly once. Write a program that solves this puzzle using a depth-first search. It's best to make the board size variable so that you can attempt solutions for smaller boards. The regular 8x8 board can take years to solve on a desktop computer, but a 5x5 board takes only a minute or so.

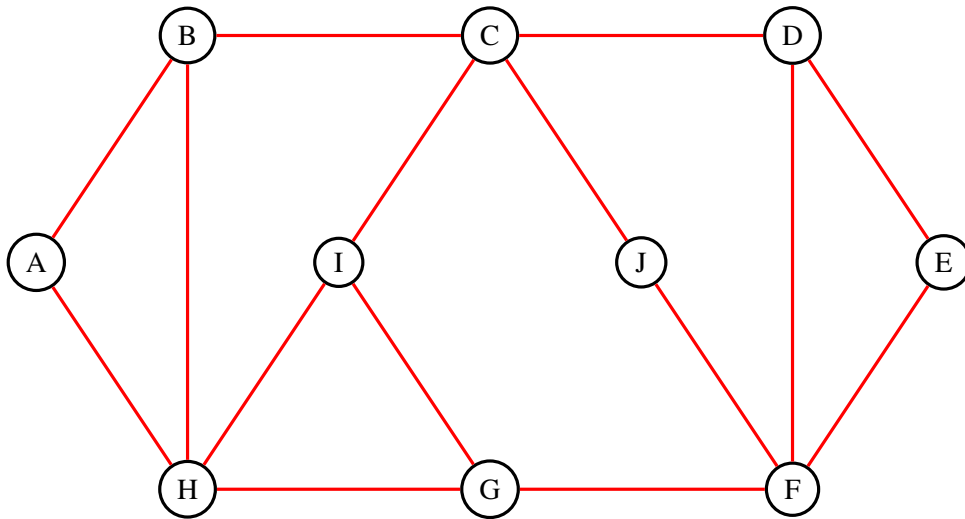
It may be easier to think of a new knight being created and remaining on the new square when a move is made. This way, a knight corresponds to a vertex, and a sequence of knights can be pushed onto the stack. When the board is completely filled with knights (the stack is full), you win. In this problem the board is traditionally numbered sequentially, from 1 at the upper-left corner to 64 at the lower-right corner (or 1 to 25 on a 5x5 board). When looking for its next move, a knight must not only make a legal knight's move, it must also not move off the board or onto an already-occupied (visited) square. If you make the program display the board and wait for a keypress after every move, you can watch the progress of the algorithm as it places more and more knights on the board, and then, when it gets boxed in, backtracks by removing some knights and trying a different series of moves.

14. Consider each graph below. Type in the corresponding zero-one matrix.



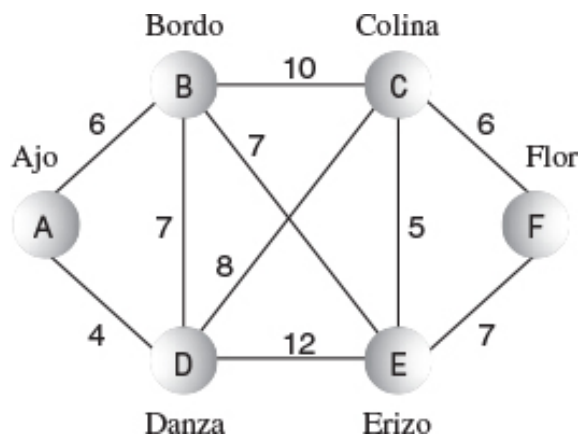
-
15. Consider the **Graph** program (see code 9.2 on page 137.) Add a method *countEdges()* that counts and returns the number of edges in the graph.
Suggestion: It may be easier if you use the adjacency matrix.

16. Find the minimum spanning tree for the graph below. Start at any node. Try both DFS and BFS approaches.



10. Weighted Graphs

In many cases it is useful to specify weights (numerical values) for each vertex in a graph. In real world applications weighted graphs are very common.



Assume the weights in the above graph represent the cost of building a cable between the towns. How can we pick a route that minimizes the cost of installing the cable system?

10.1 Minimum Spanning Trees for Weighted Graphs

Previously we discussed minimum spanning trees (MST) for unweighted graphs. Now the goal is to find the MST with the smallest sum of weights. The MST algorithm we describe next uses a priority queue. For simplicity we will use an array based priority queue.

Outline of the Algorithm

Start with a vertex, and put it in the tree. Then repeatedly do the following:

1. Find all the edges from the newest vertex to other vertices that aren't in the tree. Put these edges in the priority queue.
2. Pick the edge with the lowest weight, and add this edge and its destination vertex to the tree.

Repeat these steps until all the vertices are in the tree. At that point, you're done. In step 1, newest

means most recently installed in the tree. The edges for this step can be found in the adjacency matrix. After step 1, the list will contain all the edges from vertices in the tree to vertices on the fringe.



Use the algorithm to find the MST for the previous graph.

Code 10.1: Class Edge (from book)

```
1 class Edge
2 {
3     public int srcVert;    // index of a vertex starting edge
4     public int destVert;  // index of a vertex ending edge
5     public int distance;  // distance from src to dest
6     //-----
7     public Edge(int sv, int dv, int d) // constructor
8     {
9         srcVert = sv;
10        destVert = dv;
11        distance = d;
12    }
13    //-----
14 } // end class Edge
```



<— Get code here.

Code 10.2: Class Vertex (from book)

```
1 class Vertex
2 {
3     public char label;    // label (e.g. 'A')
4     public boolean isInTree;
5     //-----
6     public Vertex(char lab) // constructor
7     {
8         label = lab;
9         isInTree = false;
10    }
11    //-----
12 } // end class Vertex
```



<— Get code here.

Code 10.3: Class PriorityQ (from book)

```

1  class PriorityQ
2  {
3      // array in sorted order, from max at 0 to min at size-1
4      private final int SIZE = 20;
5      private Edge[] queArray;
6      private int size;
7      //-----
8      public PriorityQ()           // constructor
9      {
10         queArray = new Edge[SIZE];
11         size = 0;
12     }
13     //-----
14     public void insert(Edge item) // insert item in sorted order
15     {
16         int j;
17
18         for(j=0; j<size; j++)           // find place to insert
19             if( item.distance >= queArray[j].distance )
20                 break;
21
22         for(int k=size-1; k>=j; k--)     // move items up
23             queArray[k+1] = queArray[k];
24
25         queArray[j] = item;             // insert item
26         size++;
27     }
28     //-----
29     public Edge removeMin()           // remove minimum item
30     { return queArray[--size]; }
31     //-----
32     public void removeN(int n)        // remove item at n
33     {
34         for(int j=n; j<size-1; j++)    // move items down
35             queArray[j] = queArray[j+1];
36         size--;
37     }
38     //-----
39     public Edge peekMin()             // peek at minimum item
40     { return queArray[size-1]; }
41     //-----
42     public int size()                 // return number of items
43     { return size; }
44     //-----
45     public boolean isEmpty()          // true if queue is empty
46     { return (size==0); }
47     //-----
48     public Edge peekN(int n)          // peek at item n

```

```

49     { return queArray[n]; }
50     //-----
51     public int find(int findDex) // find item with specified
52     {                          // destVert value
53         for(int j=0; j<size; j++)
54             if(queArray[j].destVert == findDex)
55                 return j;
56         return -1;
57     }
58     //-----
59 } // end class PriorityQ

```



← Get code here.

Code 10.4: Class Graph (from book)

```

1  class Graph
2  {
3      private final int MAX_VERTS = 20;
4      private final int INFINITY = 1000000;
5      private Vertex vertexList[]; // list of vertices
6      private int adjMat[][];      // adjacency matrix
7      private int nVerts;          // current number of vertices
8      private int currentVert;
9      private PriorityQ thePQ;
10     private int nTree;            // number of verts in tree
11     //-----
12     public Graph()                // constructor
13     {
14         vertexList = new Vertex[MAX_VERTS];
15         // adjacency matrix
16         adjMat = new int[MAX_VERTS][MAX_VERTS];
17         nVerts = 0;
18         for(int j=0; j<MAX_VERTS; j++) // set adjacency
19             for(int k=0; k<MAX_VERTS; k++) // matrix to 0
20                 adjMat[j][k] = INFINITY;
21         thePQ = new PriorityQ();
22     } // end constructor
23     //-----
24     public void addVertex(char lab)
25     {
26         vertexList[nVerts++] = new Vertex(lab);
27     }
28     //-----
29     public void addEdge(int start, int end, int weight)
30     {
31         adjMat[start][end] = weight;
32         adjMat[end][start] = weight;

```

```

33     }
34     //-----
35     public void displayVertex(int v)
36     {
37         System.out.print(vertexList[v].label);
38     }
39     //-----
40     public void mstw()          // minimum spanning tree
41     {
42         currentVert = 0;        // start at 0
43
44         while(nTree < nVerts-1) // while not all verts in tree
45         {                       // put currentVert in tree
46             vertexList[currentVert].isInTree = true;
47             nTree++;
48
49             // insert edges adjacent to currentVert into PQ
50             for(int j=0; j<nVerts; j++) // for each vertex ,
51             {
52                 if(j==currentVert) // skip if it's us
53                     continue;
54                 if(vertexList[j].isInTree) // skip if in the tree
55                     continue;
56                 int distance = adjMat[currentVert][j];
57                 if( distance == INFINITY) // skip if no edge
58                     continue;
59                 putInPQ(j, distance); // put it in PQ (maybe)
60             }
61             if(thePQ.size()==0) // no vertices in PQ?
62             {
63                 System.out.println(" GRAPH NOT CONNECTED");
64                 return;
65             }
66             // remove edge with minimum distance , from PQ
67             Edge theEdge = thePQ.removeMin();
68             int sourceVert = theEdge.srcVert;
69             currentVert = theEdge.destVert;
70
71             // display edge from source to current
72             System.out.print( vertexList[sourceVert].label );
73             System.out.print( vertexList[currentVert].label );
74             System.out.print(" ");
75         } // end while(not all verts in tree)
76
77         // mst is complete
78         for(int j=0; j<nVerts; j++) // unmark vertices
79             vertexList[j].isInTree = false;
80     } // end mstw
81     //-----

```

```

82     public void putInPQ(int newVert, int newDist)
83     {
84         // is there another edge with the same destination vertex?
85         int queueIndex = thePQ.find(newVert);
86         if(queueIndex != -1)           // got edge's index
87         {
88             Edge tempEdge = thePQ.peekN(queueIndex); // get edge
89             int oldDist = tempEdge.distance;
90             if(oldDist > newDist)       // if new edge shorter,
91             {
92                 thePQ.removeN(queueIndex); // remove old edge
93                 Edge theEdge =
94                     new Edge(currentVert, newVert, newDist);
95                 thePQ.insert(theEdge);     // insert new edge
96             }
97             // else no action; just leave the old vertex there
98         } // end if
99         else // no edge with same destination vertex
100         {           // so insert new one
101             Edge theEdge = new Edge(currentVert, newVert, newDist);
102             thePQ.insert(theEdge);
103         }
104     } // end putInPQ()
105     //-----
106 } // end class Graph

```



← Get code here.

Code 10.5: Class MSTWApp (from book)

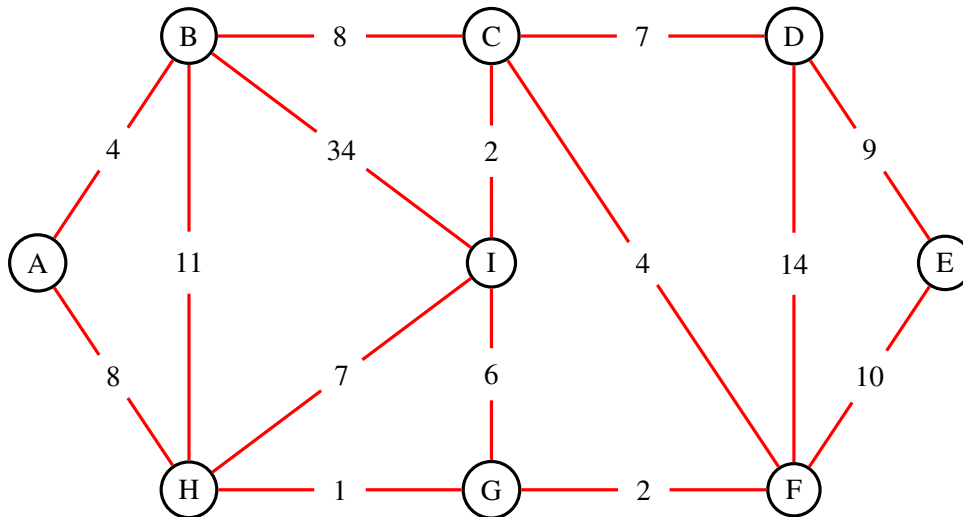
```
1 class MSTWApp
2 {
3     public static void main(String[] args)
4     {
5         Graph theGraph = new Graph();
6         theGraph.addVertex('A'); // 0 (start for mst)
7         theGraph.addVertex('B'); // 1
8         theGraph.addVertex('C'); // 2
9         theGraph.addVertex('D'); // 3
10        theGraph.addVertex('E'); // 4
11        theGraph.addVertex('F'); // 5
12
13        theGraph.addEdge(0, 1, 6); // AB 6
14        theGraph.addEdge(0, 3, 4); // AD 4
15        theGraph.addEdge(1, 2, 10); // BC 10
16        theGraph.addEdge(1, 3, 7); // BD 7
17        theGraph.addEdge(1, 4, 7); // BE 7
18        theGraph.addEdge(2, 3, 8); // CD 8
19        theGraph.addEdge(2, 4, 5); // CE 5
20        theGraph.addEdge(2, 5, 6); // CF 6
21        theGraph.addEdge(3, 4, 12); // DE 12
22        theGraph.addEdge(4, 5, 7); // EF 7
23
24        System.out.print("Minimum spanning tree: ");
25        theGraph.mstw(); // minimum spanning tree
26        System.out.println();
27    } // end main()
28 } // end class MSTWApp
```



← Get code here.



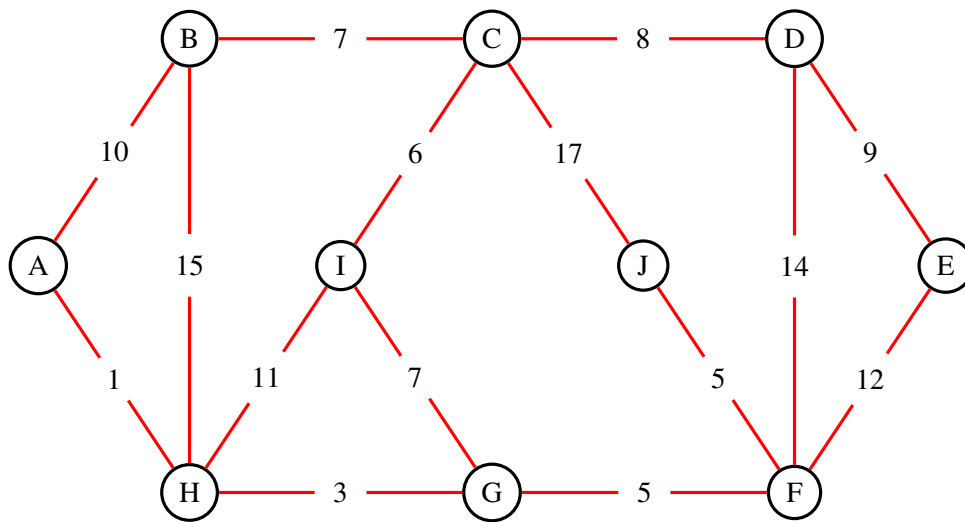
Find the minimum spanning tree for the graph below. Start from any node.



10.1.1 Practice Questions

1. **True or False:** The weight of the MST depends on the starting vertex.
2. In a weighted graph, the minimum spanning tree (MST) tries to minimize
 - (a) the number of edges from the starting vertex to a specified vertex.
 - (b) the number of edges connecting all the vertices.
 - (c) the total weight of the edges from the starting vertex to a specified vertex.
 - (d) the total weight of edges connecting all the vertices.
3. In the cable TV example, each edge added to the MST connects
 - (a) the starting vertex to an adjacent vertex.
 - (b) an already-connected city to an unconnected city.
 - (c) the current vertex to an adjacent vertex.
 - (d) two cities with offices.
4. In the MST algorithm, what is removed from the priority queue?
5. The MST algorithm "prunes" an edge from the list when the edge leads to a vertex that

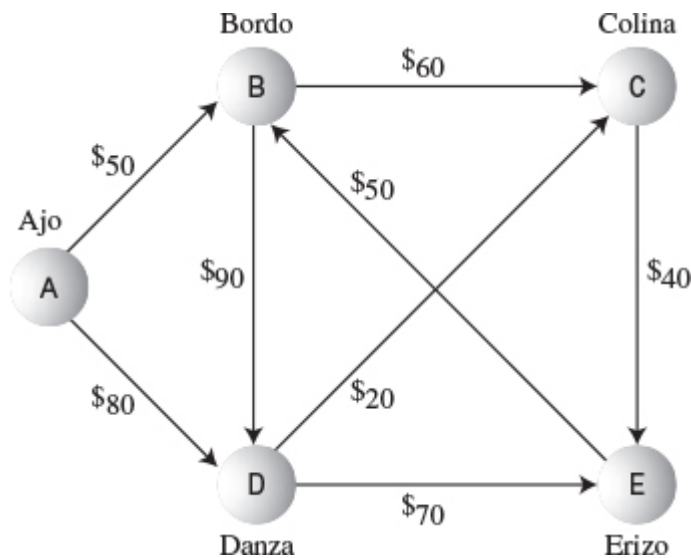
6. Find the minimum spanning tree for the weighted graph below. Start from any node.



10.2 The Shortest-Path Problem

Perhaps the most commonly encountered problem associated with weighted graphs is that of finding the shortest path between two given vertices. Consider the graph below. Each vertex represents a

railroad station. Each edge represents a one directional track connection between 2 stations. The weight for each edge represents the cost of taking the train between those two stations.



What would be the cheapest fare for traveling between two stations?

The solution we'll show for the shortest-path problem is called Dijkstra's algorithm, after Edsger Dijkstra, who first described it in 1959. This algorithm is based on the adjacency matrix representation of a graph.

The key points are:

- Each time you send an agent to a new town, you use the new information provided by that agent to revise your list of fares. Only the cheapest fare (that you know about) from the starting point to a given town is retained.
- You always send the new agent to the town that has the cheapest path from the starting point (not the cheapest edge from any town with an agent, as in the minimum spanning tree).

Code 10.6: Class DistPar (from book)

```
1 class DistPar           // distance and parent
2 {                       // items stored in sPath array
3     public int distance; // distance from start to this vertex
4     public int parentVert; // current parent of this vertex
5     //-----
6     public DistPar(int pv, int d) // constructor
7     {
8         distance = d;
9         parentVert = pv;
10    }
11 } // end class DistPar
```



← Get code here.

Code 10.7: Class Vertex (from book)

```
1 class Vertex
2 {
3     public char label; // label (e.g. 'A')
4     public boolean isInTree;
5     //-----
6     public Vertex(char lab) // constructor
7     {
8         label = lab;
9         isInTree = false;
10    }
11    //-----
12 } // end class Vertex
```



← Get code here.

Code 10.8: Class Graph (from book)

```

1  class Graph
2  {
3      private final int MAX_VERTS = 20;
4      private final int INFINITY = 1000000;
5      private Vertex vertexList[]; // list of vertices
6      private int adjMat[][];      // adjacency matrix
7      private int nVerts;          // current number of vertices
8      private int nTree;           // number of verts in tree
9      private DistPar sPath[];     // array for shortest-path data
10     private int currentVert;      // current vertex
11     private int startToCurrent;   // distance to currentVert
12     //-----
13     public Graph()                // constructor
14     {
15         vertexList = new Vertex[MAX_VERTS];
16         // adjacency matrix
17         adjMat = new int[MAX_VERTS][MAX_VERTS];
18         nVerts = 0;
19         nTree = 0;
20         for(int j=0; j<MAX_VERTS; j++)    // set adjacency
21             for(int k=0; k<MAX_VERTS; k++) //      matrix
22                 adjMat[j][k] = INFINITY; //      to infinity
23         sPath = new DistPar[MAX_VERTS];    // shortest paths
24     } // end constructor
25     //-----
26     public void addVertex(char lab)
27     {
28         vertexList[nVerts++] = new Vertex(lab);
29     }
30     //-----
31     public void addEdge(int start, int end, int weight)
32     {
33         adjMat[start][end] = weight; // (directed)
34     }
35     //-----
36     public void path()              // find all shortest paths
37     {
38         int startTree = 0;          // start at vertex 0
39         vertexList[startTree].isInTree = true;
40         nTree = 1;                  // put it in tree
41
42         // transfer row of distances from adjMat to sPath
43         for(int j=0; j<nVerts; j++)
44         {
45             int tempDist = adjMat[startTree][j];
46             sPath[j] = new DistPar(startTree, tempDist);
47         }
48     }

```

```

49     // until all vertices are in the tree
50     while(nTree < nVerts)
51     {
52         int indexMin = getMin();    // get minimum from sPath
53         int minDist = sPath[indexMin].distance;
54
55         if(minDist == INFINITY)    // if all infinite
56         {                          // or in tree ,
57             System.out.println("There are unreachable vertices");
58             break;                // sPath is complete
59         }
60         else
61         {                          // reset currentVert
62             currentVert = indexMin; // to closest vert
63             startToCurrent = sPath[indexMin].distance;
64             // minimum distance from startTree is
65             // to currentVert, and is startToCurrent
66         }
67         // put current vertex in tree
68         vertexList[currentVert].isInTree = true;
69         nTree++;
70         adjust_sPath();            // update sPath[] array
71     } // end while(nTree<nVerts)
72
73     displayPaths();               // display sPath[] contents
74
75     nTree = 0;                    // clear tree
76     for(int j=0; j<nVerts; j++)
77         vertexList[j].isInTree = false;
78 } // end path()
79 //-----
80 public int getMin()               // get entry from sPath
81 {                                // with minimum distance
82     int minDist = INFINITY;      // assume minimum
83     int indexMin = 0;
84     for(int j=1; j<nVerts; j++)  // for each vertex ,
85     {                            // if it's in tree and
86         if( !vertexList[j].isInTree && // smaller than old one
87             sPath[j].distance < minDist )
88         {
89             minDist = sPath[j].distance;
90             indexMin = j;         // update minimum
91         }
92     } // end for
93     return indexMin;             // return index of minimum
94 } // end getMin()
95 //-----
96 public void adjust_sPath()
97 {

```

```

98     // adjust values in shortest-path array sPath
99     int column = 1;                // skip starting vertex
100    while(column < nVerts)          // go across columns
101    {
102        // if this column's vertex already in tree, skip it
103        if( vertexList[column].isInTree )
104        {
105            column++;
106            continue;
107        }
108        // calculate distance for one sPath entry
109        // get edge from currentVert to column
110        int currentToFringe = adjMat[currentVert][column];
111        // add distance from start
112        int startToFringe = startToCurrent + currentToFringe;
113        // get distance of current sPath entry
114        int sPathDist = sPath[column].distance;
115
116        // compare distance from start with sPath entry
117        if(startToFringe < sPathDist) // if shorter,
118        {                               // update sPath
119            sPath[column].parentVert = currentVert;
120            sPath[column].distance = startToFringe;
121        }
122        column++;
123    } // end while(column < nVerts)
124 } // end adjust_sPath()
125 //-----
126 public void displayPaths()
127 {
128     for(int j=0; j<nVerts; j++) // display contents of sPath[]
129     {
130         System.out.print(vertexList[j].label + "="); // B=
131         if(sPath[j].distance == INFINITY)
132             System.out.print("inf");                // inf
133         else
134             System.out.print(sPath[j].distance);      // 50
135         char parent = vertexList[ sPath[j].parentVert ].label;
136         System.out.print("(" + parent + ") ");        // (A)
137     }
138     System.out.println("");
139 }
140 //-----
141 } // end class Graph

```



← Get code here.

Code 10.9: Class PathApp (from book)

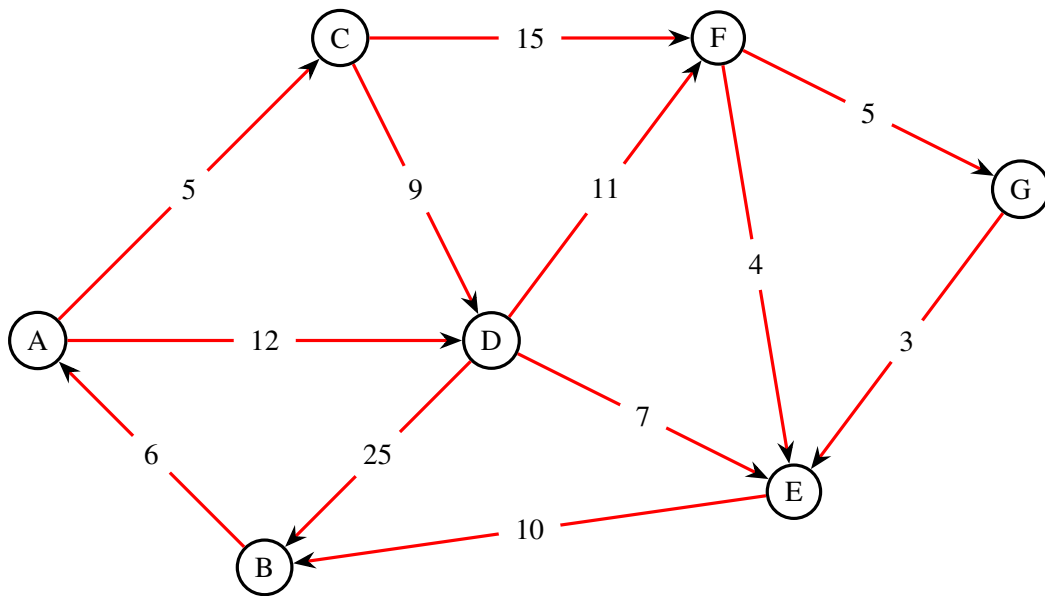
```
1  class PathApp
2  {
3      public static void main(String[] args)
4      {
5          Graph theGraph = new Graph();
6          theGraph.addVertex('A');    // 0  (start)
7          theGraph.addVertex('C');    // 2
8          theGraph.addVertex('B');    // 1
9          theGraph.addVertex('D');    // 3
10         theGraph.addVertex('E');    // 4
11
12         theGraph.addEdge(0, 1, 50);  // AB 50
13         theGraph.addEdge(0, 3, 80);  // AD 80
14         theGraph.addEdge(1, 2, 60);  // BC 60
15         theGraph.addEdge(1, 3, 90);  // BD 90
16         theGraph.addEdge(2, 4, 40);  // CE 40
17         theGraph.addEdge(3, 2, 20);  // DC 20
18         theGraph.addEdge(3, 4, 70);  // DE 70
19         theGraph.addEdge(4, 1, 50);  // EB 50
20
21         System.out.println("Shortest paths");
22         theGraph.path();              // shortest paths
23         System.out.println();
24     } // end main()
25 } // end class PathApp
```



← Get code here.



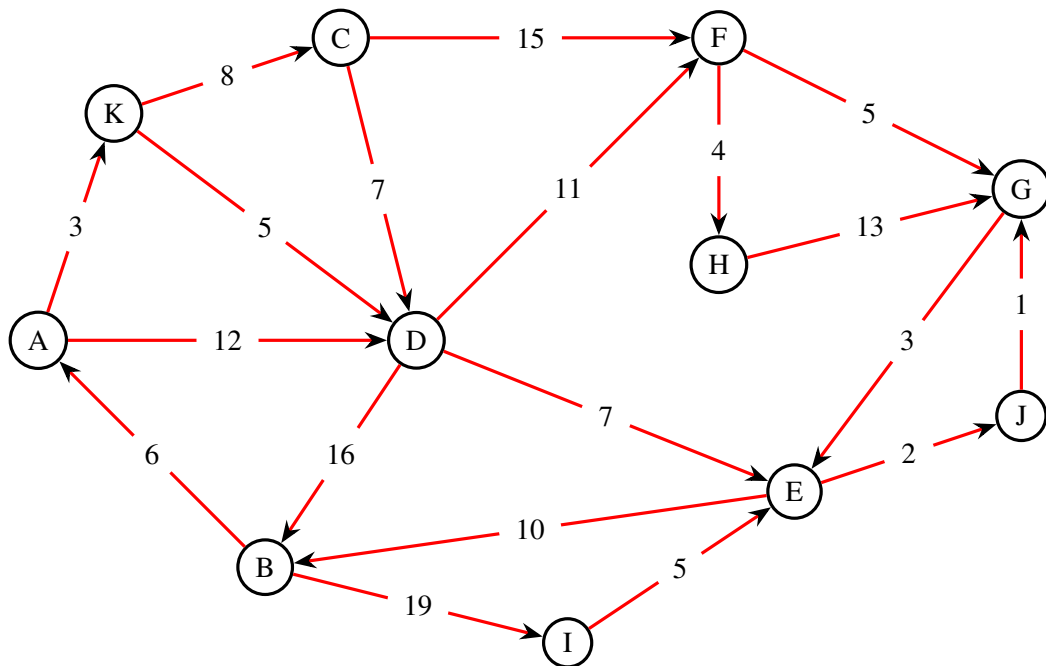
Use Dijkstra's algorithm to find all shortest path from node A in the graph below.



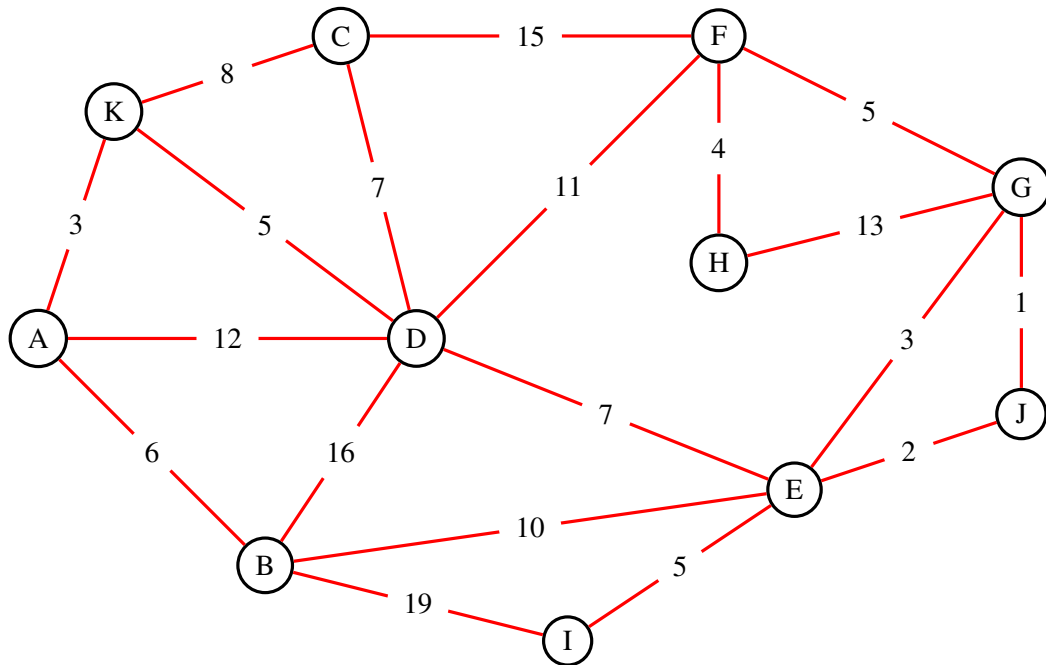
10.2.1 Practice Questions

1. **True or False:** The shortest-path problem (SPP) must be carried out on a directed graph.
2. Dijkstra's algorithm finds the shortest path
 - (a) from one specified vertex to all other vertices.
 - (b) from one specified vertex to another specified vertex.
 - (c) from all vertices to all other vertices that can be reached along one edge.
 - (d) from all vertices to all other vertices that can be reached along multiple edges.
3. **True or False:** The rule in Dijkstra's algorithm is to always put in the tree the vertex that is closest to the starting vertex.

4. Use Dijkstra's algorithm to find all shortest path from node A in the directed weighted graph below.



5. Use Dijkstra's algorithm to find all shortest path from node A in the **undirected** weighted graph below. Does the algorithm work for undirected graphs? Compare the results with the previous example of a directed graph.



11. When to Use What?

11.1 General Purpose Data Structures

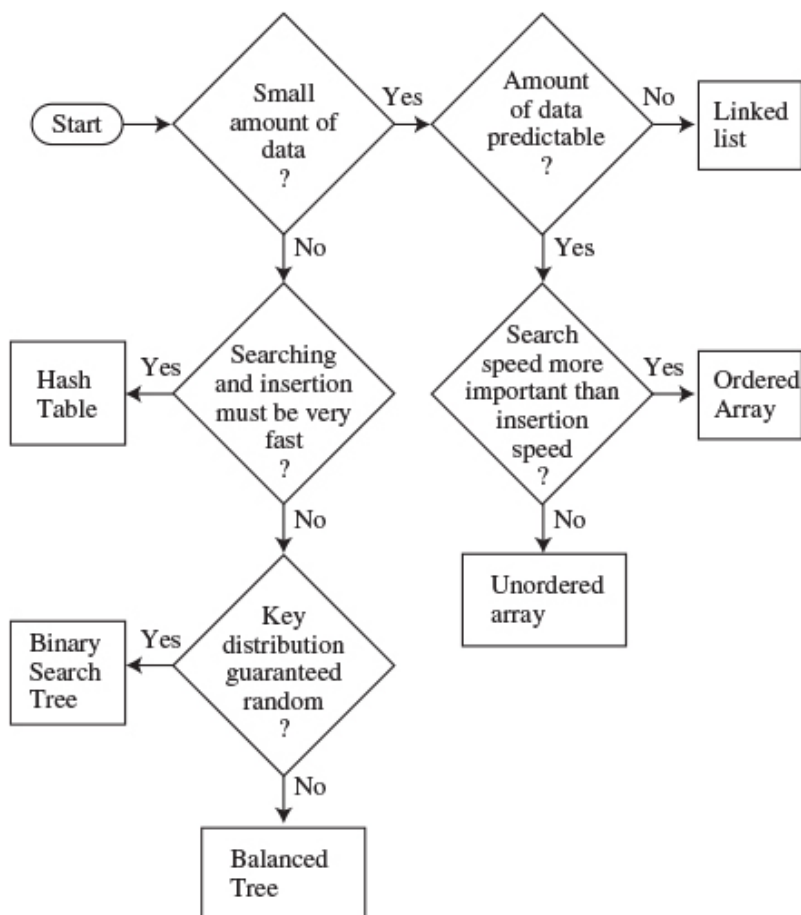


Table 11.1: Data Structure Operations

Data Structure	Search	Insertion	Deletion	Traversal
Array	$O(N)$	$O(1)$	$O(N)$	—
Ordered array	$O(\log N)$	$O(N)$	$O(N)$	$O(N)$
Linked list	$O(N)$	$O(1)$	$O(N)$	—
Ordered linked list	$O(N)$	$O(N)$	$O(N)$	$O(N)$
Binary tree (average)	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$
Binary tree (worst case)	$O(N)$	$O(N)$	$O(N)$	$O(N)$
Balanced tree (average and worst case)	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$
Hash table	$O(1)$	$O(1)$	$O(1)$	—

11.2 Special Purpose Data Structures

Table 11.2: Data Structure Operations

Data Structure	Insertion	Deletion	Comment
Stack (array or linked list)	$O(1)$	$O(1)$	Deletes most recently inserted item
Queue (array or linked list)	$O(1)$	$O(1)$	Deletes least recently inserted item
Priority queue (ordered array)	$O(N)$	$O(1)$	Deletes highest-priority item
Priority queue (heap)	$O(\log N)$	$O(\log N)$	Deletes highest-priority item

11.3 Sorting

Table 11.3: Sorting Algorithms

Sort	Average	Worst	Comparison	Extra Memory
Bubble	$O(N^2)$	$O(N^2)$	Poor	No
Selection	$O(N^2)$	$O(N^2)$	Fair	No
Insertion	$O(N^2)$	$O(N^2)$	Good	No
Shellsort	$O(N^{3/2})$	$O(N^{3/2})$	—	No
Quicksort	$O(N \log N)$	$O(N^2)$	Good	No
Mergesort	$O(N \log N)$	$O(N \log N)$	Fair	Yes
Heapsort	$O(N \log N)$	$O(N \log N)$	Fair	No

12. Sample Chapter

12.1 Sample section

A. Answers to Exercises

A.1 Chapter 1

→ Answer to question [2](#)

```
1 public class Rectangle {
2     private double length;
3     private double width;
4
5     public Rectangle() {
6         this.length = 0;
7         this.width = 0;
8     }
9
10    public Rectangle(double length, double width) {
11        this.length = length;
12        this.width = width;
13    }
14
15    public double getLength() {
16        return length;
17    }
18
19    public void setLength(double length) {
20        this.length = length;
21    }
22
23    public double getWidth() {
24        return width;
25    }
```

```
26
27     public void setWidth(double width) {
28         this.width = width;
29     }
30
31     public double calculateArea() {
32         return length * width;
33     }
34
35     public double calculatePerimeter() {
36         return 2 * (length + width);
37     }
38 }
```

```
1 public class Square extends Rectangle {
2     public Square() {
3         super();
4     }
5
6     public Square(double side) {
7         super(side, side);
8     }
9
10    public double getSide() {
11        return super.getLength();
12    }
13
14    public void setSide(double side) {
15        super.setLength(side);
16        super.setWidth(side);
17    }
18
19    @Override
20    public double calculateArea() {
21        return super.calculateArea();
22    }
23
24    @Override
25    public double calculatePerimeter() {
26        return 4 * super.getLength();
27    }
28 }
```

```
1 public class Main {
2     public static void main(String[] args) {
3         Rectangle rectangle = new Rectangle(5.0, 4.0);
4         System.out.println("Rectangle area: " + rectangle.calculateArea());
5         System.out.println("Rectangle perimeter: " + rectangle.calculatePerimeter());
6
7         Square square = new Square(5.0);
8         System.out.println("Square area: " + square.calculateArea());
9         System.out.println("Square perimeter: " + square.calculatePerimeter());
10    }
11 }
```

→ Answer to question 14

```
1 public long getMax() {
2     if (nElems == 0) {
3         return -1; // array is empty
4     }
5     long max = a[0];
6     for (int j = 1; j < nElems; j++) {
7         if (a[j] > max) {
8             max = a[j];
9         }
10    }
11    return max;
12 }
```

→ Answer to question 15

```
1 public long removeMax() {
2     if (nElems == 0) {
3         return -1;
4     }
5     long max = a[0];
6     int maxIndex = 0;
7     for (int j = 1; j < nElems; j++) {
8         if (a[j] > max) {
9             max = a[j];
10            maxIndex = j;
11        }
12    }
13    for (int k = maxIndex; k < nElems; k++) {
14        a[k] = a[k + 1];
15    }
16    nElems--;
17    return max;
18 }
```

→ Item 3

A.2 Chapter 2

→ Answer to question 10

Correct: takes the same time no matter how many items there are.

→ Answer to question 20

```
1 public int find(long searchKey) {
2     int lowerBound = 0;
3     int upperBound = nElems - 1;
4     int curIn;
5
6     while (true) {
7         curIn = (lowerBound + upperBound) / 2;
8         if (a[curIn] == searchKey)
9             return curIn; // found it
10        else if (lowerBound > upperBound)
11            return -1; // can't find it
12        else {
13            if (a[curIn] < searchKey)
14                lowerBound = curIn + 1; // it's in upper half
15            else
16                upperBound = curIn - 1; // it's in lower half
17        } // end else divide range
18    } // end while
19 } // end find()
20
21 public void insert(long value) {
22     int j = 0;
23     int lowerBound = 0;
24     int upperBound = nElems - 1;
25
26     while (lowerBound <= upperBound) {
27         int curIn = (lowerBound + upperBound) / 2;
28         if (a[curIn] < value)
29             lowerBound = curIn + 1;
30         else
31             upperBound = curIn - 1;
32     }
33
34     j = lowerBound;
35
36     for (int k = nElems; k > j; k--) // move bigger ones up
37         a[k] = a[k - 1];
38     a[j] = value; // insert it
39     nElems++; // increment size
```

```
40 } // end insert()
41
42 public boolean delete(long value) {
43     int j = find(value);
44     if (j == -1) // can't find it
45         return false;
46     else { // found it
47         for (int k = j; k < nElems; k++) // move bigger ones down
48             a[k] = a[k + 1];
49         nElems--; // decrement size
50         return true;
51     }
52 } // end delete()
```

→ Item 2

→ Item 3

A.3 Chapter 1

→ Item 1

→ Item 2

→ Item 3

Index

- if* and *if-else* Statements, 41
- Abstract Classes, 181
- Adding and Removing Nodes, 192
- Addition and Subtraction of Numbers at Different Bases, 20
- Arithmetic Operations, 29
- Array Size, 101
- Arrays and Pointer Arithmetic, 112
- Arrays and Pointers, 150
- Arrays of Structures, 118
- Background, 183
- Basic Exception Handling, 195
- C++ Loops. The *while* and *do...while* Statement., 55
- C++ Syntax Tokens, 25
- C-style String Functions, 127
- Characters, Strings and Arrays of Strings, 104
- Class Composition, 147
- Class Templates, 187
- Classes, 122
- Constant Objects, 145
- Constructors, 140
- Conversion Between Binary, Octal and Hexadecimal, 19
- Converting Binary, Octal and Hexadecimal to Decimal, 15
- Converting Decimal Numbers to Other Bases, 17
- Default and Static Variables, 90
- Default Function Arguments, 78
- Destructors, 142
- Function Overloading, 79
- Function Templates, 185
- Getting Started with C++, 11
- How C++ Stores Text?, 125
- Inheritance, 167
- Inheritance and Constructors, 170
- Inheriting Functionality, 172
- Initializing Arrays, 98
- Iterators, 201
- Linked Lists, 189
- Linked Lists using Classes, 191
- Local and Global Variables, 89
- Logical Operators, 46
- Member Initializers, 146
- More on Recursion, 159
- Multi-Dimensional Arrays, 105
- Multilevel Inheritance, 175
- Multiple Inheritance, 174
- Multiple Throws and Catches, 198
- Objects, 121
- Operator Overloading, 153

Passing a Parameter by Value or by Reference, 76

Passing Arrays to Functions, 102

Passing Structures to Functions, 119

Pointers and Some Special Operators, 150

Pointers Revisited, 149

Polymorphism and Virtual Functions, 177

Practice Questions 1, 13

Practice Questions 2, 22

Practice Questions 3, 32, 49

Practice Questions 5, 61

Practice Questions 6, 80

Practice Questions 7, 93, 162

Practice Questions 8, 106

Pre-defined Functions, 91

Primitive Data Types, 27

Queue Iterator Example, 203

Recursive Functions, 155

Relational and Comparison Operators, 41

Separating Content in Different Files, 143

Sorting Algorithms, 133

Storing Information in the Computer, 15

Stream Objects, 211

Structure Composition, 117

The `<string>` Class, 128

The *break* and *continue* Statements, 60

The *for* loop, 57

The *switch* Statement, 44

The First Example, 122

The Nested *if-else* Statements, 44

The **this** Keyword, 152

Throwing an Exception in a Function, 199

Vectors, 202

What are Arrays, 97

What are Functions?, 71

What are Pointers?, 111

What are Structures?, 115

What is Computer Science?, 9

What is Programming?, 10